



**The
University of
Faisalabad**

**COMPUTER ORGANIZATION AND ASSEMBLY
LANGUAGE**

LAB MANUAL

SUBMITTED BY : AYESHA IMRAN

REGISTRATION NO : 2023-BSAI-011

SUBMITTED TO : MS NOORHAN

DEPARTMENT : CS (AI)

LAB NO 1

INTRODUCTION AND NUMBER CONVERSION

Introduction

Computer Organization and Assembly Language is an important subject that helps us understand how a computer works at the hardware and software level. It explains the internal structure and functioning of a computer system — including how data is represented, processed, and controlled within the CPU. Computer organization focuses on the physical components of a computer, such as the memory, processor, registers, and buses, and how these parts interact to execute instructions.

Assembly Language, on the other hand, is a low-level programming language that allows programmers to write instructions in a form closer to machine code. It helps in understanding how programs actually run inside the computer by directly controlling hardware operations. Learning Assembly Language improves our knowledge of processor architecture, instruction sets, and memory management.

Together, Computer Organization and Assembly Language bridge the gap between hardware and software, providing the foundation for understanding how high-level programs are executed by the computer.

Task 1: Decimal Conversions

Number systems are used in computers to represent and process data. The decimal number system (**base 10**) is commonly used by humans, while computers use binary (**base 2**) internally. Octal (**base 8**) and hexadecimal (**base 16**) are used as compact representations of binary numbers.

1. Decimal to Binary Conversion

Binary number system uses only two digits: 0 and 1.

Method:

- Divide the decimal number repeatedly by 2
- Record the remainder at each step
- The binary equivalent is obtained by reading the remainders from bottom to top

Importance:

Binary representation is fundamental because computers operate using binary logic (ON/OFF states).

2. Decimal to Octal Conversion

Octal number system has a base of 8 and uses digits from 0 to 7.

Method:

- Divide the decimal number repeatedly by 8
- Record the remainder after each division
- Read the remainders from bottom to top to get the octal number

Importance:

Octal is used to represent large binary numbers in a shorter and more readable form.

3. Decimal to Hexadecimal Conversion

Hexadecimal number system has a base of 16 and uses digits:

0–9 and A–F

(where A = 10, B = 11, ..., F = 15)

Method:

- Divide the decimal number repeatedly by 16
- Record the remainder
- Replace remainders from 10 to 15 with letters A to F
- Read the remainders from bottom to top

Importance:

Hexadecimal is widely used in assembly language, memory addressing, and debugging, as it provides a compact representation of binary data.

Convert the following decimal number into binary , octal and hexadecimal

Question no 1 :

(1) $(25)_{10} \rightarrow \text{Decimal}$											
Binary											
$= (11001)_2$	<table> <tr><td>2</td><td>25</td></tr> <tr><td>2</td><td>12-1</td></tr> <tr><td>2</td><td>6-0</td></tr> <tr><td>2</td><td>3-0</td></tr> <tr><td></td><td>1-1</td></tr> </table>	2	25	2	12-1	2	6-0	2	3-0		1-1
2	25										
2	12-1										
2	6-0										
2	3-0										
	1-1										
Octal											
$= (31)_8$	<table> <tr><td>8</td><td>25</td></tr> <tr><td></td><td>3-1</td></tr> </table>	8	25		3-1						
8	25										
	3-1										
Hexadecimal											
$= (19)_{16}$	<table> <tr><td>16</td><td>25</td></tr> <tr><td></td><td>1-9</td></tr> </table>	16	25		1-9						
16	25										
	1-9										

Question no 2 :

(2) $(73)_{10} \rightarrow \text{Decimal}$

Binary
 $= (1001001)_2$

2	73
2	36 - 1
2	18 - 0
2	9 - 0
2	4 - 1
2	2 - 0
2	1 - 0

Octal
 $= (111)_8$

8	73
8	9 - 1
	1 - 1

Hexadecimal
 $= (49)_{16}$

16	73
	4 - 9

Question no 3 :

(3) $(255)_{10}$

Binary
 $= (11111111)_2$

2	255
2	127 - 1
2	63 - 1
2	31 - 1
2	15 - 1
2	7 - 1
2	3 - 1
2	1 - 1

Octal
 $= (377)_8$

8	255
8	32 - 7
	3 - 7

Hexadecimal
 $= (FF)_{16}$

16	255
	15 - 15

$\therefore 15 \rightarrow F$

Question no 4 :

(4) (1023)₁₀

Binary
= (11111111)₂

Octal
= (1777)₈

Hexadecimal
= (3FF)₁₆

2	1023 - 1
2	511 - 1
2	255 - 1
2	127 - 1
2	63 - 1
2	31 - 1
2	15 - 1
2	7 - 1
2	3 - 1
2	1 - 1

8	1023
8	127 - 7
8	15 - 7
8	1 - 7

16	1023
16	63 - 15 (F)
16	3 - 15 (F)

Task 2: Binary Conversions

The binary number system (base 2) uses only two digits: 0 and 1. Computers store and process all data in binary form. To understand and analyze computer operations, binary numbers are often converted into decimal and hexadecimal formats.

1. Binary to Decimal Conversion

The decimal number system (base 10) is the standard number system used by humans.

Method

- Each binary digit (bit) is multiplied by 2 raised to the power of its position
- Positions start from 0 at the rightmost bit
- Add all the resulting values to get the decimal equivalent

Explanation

Binary digits represent powers of 2. By summing these weighted values, the decimal number is obtained.

Importance

- Binary to decimal conversion helps in:
- Understanding computer calculations
- Verifying program outputs
- Debugging assembly language programs

2. Binary to Hexadecimal Conversion

The hexadecimal number system (base 16) uses digits 0–9 and letters A–F.

Method

- Group the binary number into sets of 4 bits starting from the right
- Convert each group into its hexadecimal equivalent
- Combine the results to form the final hexadecimal number

Explanation

Each hexadecimal digit represents 4 binary bits, making hexadecimal a compact and readable form of binary data.

Importance

- Binary to hexadecimal conversion is widely used in:
- Assembly language programming
- Memory addressing
- Debugging and machine-level operations

Convert the following binary numbers into decimal and hexadecimal

Question no 1 :

(i) $(101101)_2$

Decimal

$$= (1 \times 2^5) + (0 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$$

$$= (32) + 0 + 8 + 4 + 1 = (45)_{10}$$

Hexadecimal

8	4	2	1	8	4	2	1
0	0	1	0	1	1	0	1

$= 2 \ 13 \quad \therefore 13 = D$

$= (2D)_{16}$

Question no 2 :

(ii) $(11110000)_2$

Decimal

$$= (1 \times 2^7) + (1 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$$

$$= 128 + 64 + 32 + 16 + 0 + 0 + 0 + 0 = (240)_{10}$$

Hexadecimal

8	4	2	1	8	4	2	1
1	1	1	1	0	0	0	0

$= 15$

$= (F)_{16}$

Question no 3 :

(iii) $(10011011)_2$

Binary $\rightarrow$ Decimal

$$= (1 \times 2^7) + (1 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$$

$$= 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = (251)_{10}$$

Hexadecimal

8	4	2	1	8	4	2	1
1	0	0	1	1	0	1	1

$$= 911$$

$$= (9B)_{16}$$

Question no 4 :

(iv) $(110011001)_2$

Decimal

$$= (1 \times 2^8) + (1 \times 2^7) + (1 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$$

$$= 256 + 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = (509)_{10}$$

Hexadecimal

8	4	2	1	8	4	2	1	8	4	2	1
0	0	0	1	1	0	0	1	1	0	0	1

$$= (199)_{16}$$

Task 3: Hexadecimal Conversions

The hexadecimal number system (**base 16**) is commonly used in computer systems and assembly language programming because it provides a compact representation of binary data. Hexadecimal digits include 0–9 and A–F, where A = 10, B = 11, ..., F = 15.

1. Hexadecimal to Binary Conversion**Method**

- Convert each hexadecimal digit into its 4-bit binary equivalent
- Replace all hexadecimal digits with their corresponding binary values
- Combine the binary groups to obtain the final binary number

Explanation

Each hexadecimal digit represents exactly 4 binary bits, which makes this conversion direct and fast without calculations.

Importance

- Simplifies understanding of machine-level data
- Widely used in assembly language and memory addressing
- Makes binary values easier to read and interpret

2. Hexadecimal to Decimal Conversion

Method

- Write the positional weights of each digit as powers of 16
- Multiply each hexadecimal digit by its corresponding power of 16
- Add all the results to get the decimal equivalent

Explanation

Hexadecimal numbers follow the same positional notation as decimal, but with base 16 instead of base 10.

Importance

- Helps interpret memory addresses and register values
- Useful for debugging and verifying program outputs

Convert the following hexadecimal into binary and decimal

Question no 1 :

Handwritten solution for converting hexadecimal (2F)₁₆ to binary and decimal:

Binary

8	4	2	1	8	4	2	1
0	0	1	0	1	1	1	1

$= (00101111)_2$

Decimal

$$= 2 \times 16^1 + 15 \times 16^0$$

$$= 32 + 15 = (47)_{10}$$

Question no 2 :

(ii) $(7A)_{16}$

Binary

8	4	2	1	8	4	2	1
0	1	1	1	1	0	1	0

$= (01111010)_2$

Decimal

$$= 7 \times 16^1 + 10 \times 16^0$$

$$= 112 + 10 = (122)_{10}$$

Question no 3 :

(iii) $(A3)_{16}$

Binary

8	4	2	1	8	4	2	1
1	0	1	0	0	0	1	1

$= (10100011)_2$

Decimal

$$= 10 \times 16^1 + 3 \times 16^0$$

$$= 160 + 3 = (163)_{10}$$

Question no 4 :

(iv) $(FF)_{16}$

Binary

8	4	2	1	8	4	2	1
1	1	1	1	1	1	1	1

$= (11111111)_2$

Decimal

$$= 15 \times 16^1 + 15 \times 16^0$$

$$= 240 + 15 = (255)_{10}$$

Task 4: Octal Conversions

The octal number system (**base 8**) uses digits from 0 to 7. It is often used in computer systems as a compact representation of binary numbers, especially in low-level programming and digital systems.

1. Octal to Binary Conversion

Method

- Convert each octal digit into its 3-bit binary equivalent
- Replace all octal digits with their corresponding binary values
- Combine the binary groups to obtain the final binary number

Explanation

Each octal digit represents 3 binary bits, making the conversion simple and direct without arithmetic calculations.

Importance

- Makes binary data easier to understand
- Useful in low-level programming concepts
- Helps in interpreting system-level data

2. Octal to Decimal Conversion

Method

- Assign powers of 8 to each digit, starting from the rightmost digit (power 0)
- Multiply each octal digit by its corresponding power of 8
- Add all the results to obtain the decimal value

Explanation

Octal numbers follow positional notation similar to decimal numbers, but with base 8 instead of base 10.

Importance

- Helps understand numeric values stored in computers
- Useful for debugging and learning number system conversions

Convert the following octal into binary and decimal

Question no 1 :

Handwritten solution for converting octal (57) to binary:

(1)	(57) ₈	
	Binary	
	5	7
	4 2 1	4 2 1
	1 0 1	1 1 1
	= (10111) ₂	

Decimal

$$= 5 \times 8^1 + 5 \times 8^0$$

$$= 40 + 5 = (45)_{10}$$

Question no 2 :

(ii) $(124)_8$

Binary

1	2	4
4 2 1	4 2 1	4 2 1
0 0 1	0 1 0	1 0 0

$$= (001010100)_2$$

Decimal

$$= 1 \times 8^2 + 1 \times 8^1 + 1 \times 8^0$$

$$= 64 + 8 + 1 = (73)_{10}$$

Question no 3 :

(iii) $(377)_8 \rightarrow \text{Binary}$

3	7	7
4 2 1	4 2 1	4 2 1
0 1 1	1 1 1	1 1 1

$$= (011111111)_2$$

Decimal

$$= 3 \times 8^2 + 3 \times 8^1 + 7 \times 8^0$$

$$= 192 + 24 + 7$$

$$= \cancel{79} = (201)_{10}$$

Question no 4 :

(iv) 645

Binary

6	4	5
4 2 1	4 2 1	4 2 1
1 1 0	1 0 0	1 0 1

$$= (110100101)_2$$

Decimal

$$= 6 \times 8^2 + 4 \times 8^1 + 5 \times 8^0$$

$$= 384 + 32 + 5 = (421)_{10}$$

LAB NO 2

INTRODUCTION TO EMULATOR8086 AND INSTALLATION

EMU8086 is a **microprocessor emulator** designed to simulate the working of the **Intel 8086 microprocessor**. It is widely used for **learning and practicing assembly language programming** in an easy and interactive environment.

EMU8086 provides a **virtual platform** where students can write, assemble, execute, and debug 8086 assembly programs without using actual hardware. It closely follows the **8086 instruction set architecture**, making it ideal for educational purposes.

The emulator allows users to **observe registers, flags, memory, and stack operations in real time**, which helps in understanding how assembly instructions are executed internally. EMU8086 also supports **step-by-step execution**, breakpoints, and memory visualization, making debugging simple and effective.

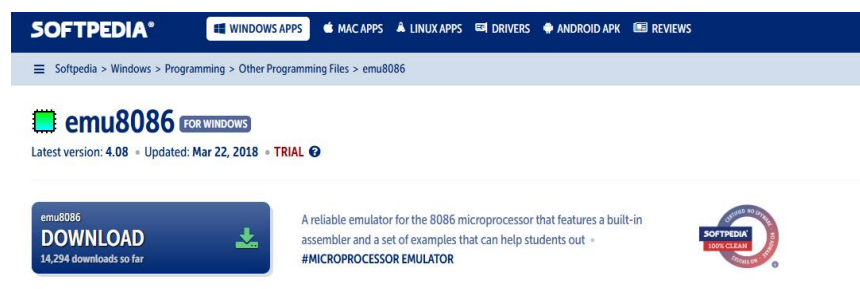
In addition, EMU8086 includes built-in examples, a code editor, and a visual interface for hardware components such as registers and flags. Due to its simplicity and accuracy, EMU8086 is commonly used in **computer organization and assembly language laboratories**.

Step no 1 :

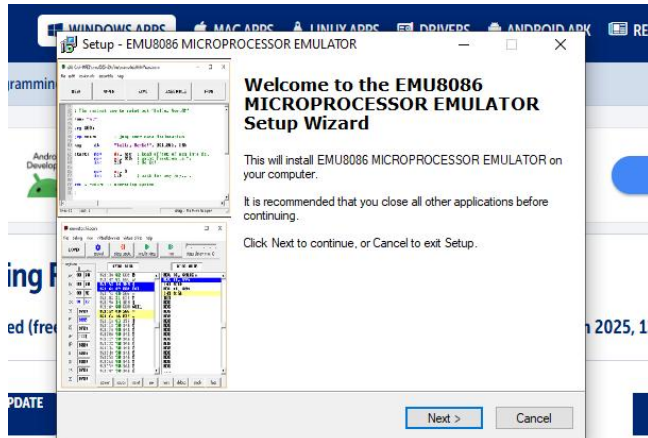
Click here to open website :

<https://www.softpedia.com/get/Programming/Other-Programming-Files/emu8086.shtml#download>

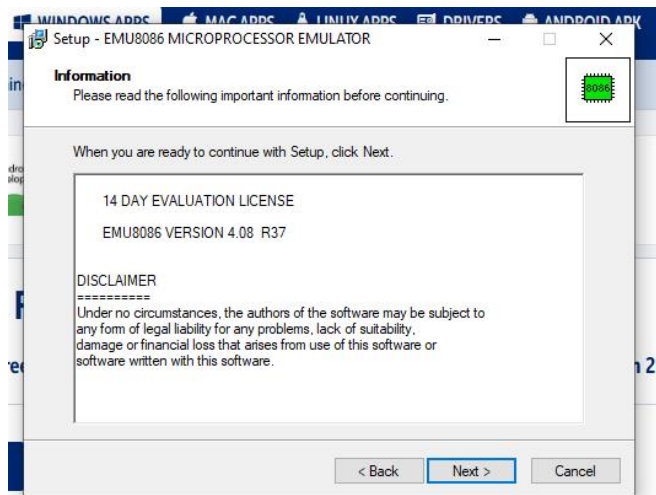
Step no 2 : Click on download button



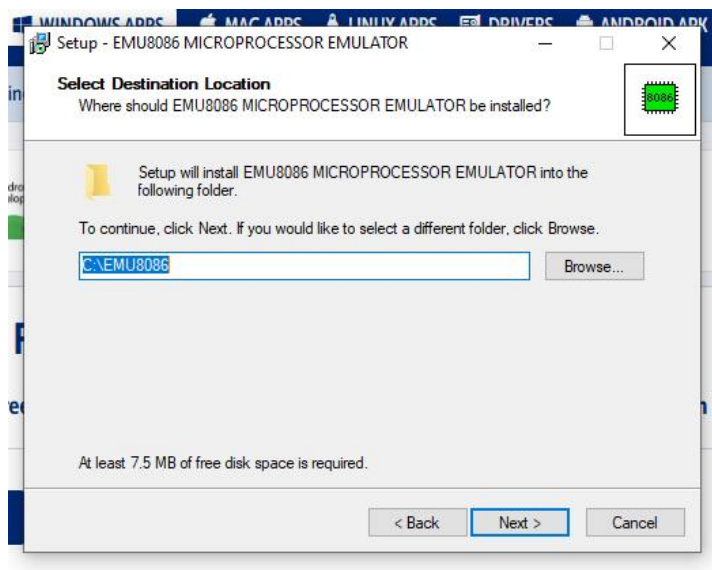
Step no 3 : Click on next



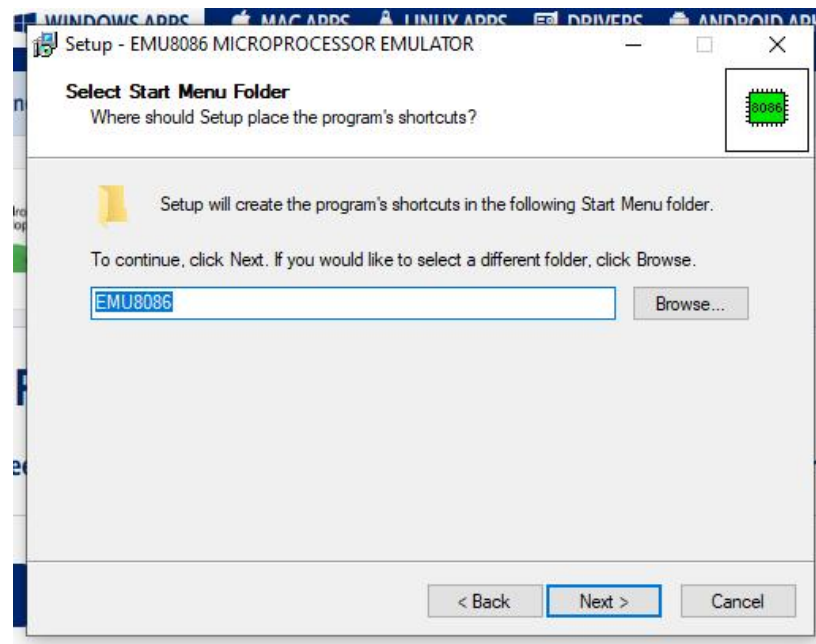
Step no 3 : click on next



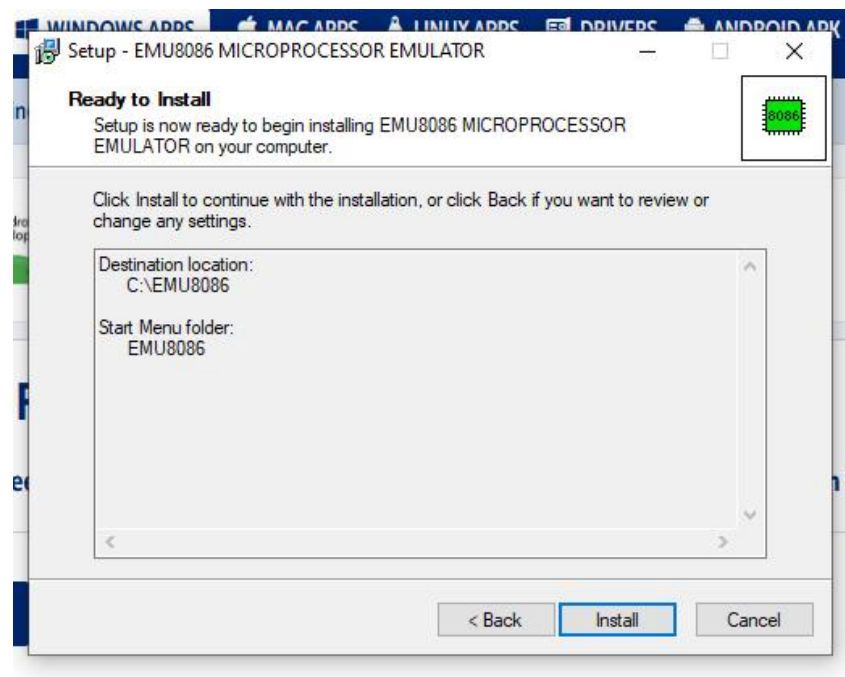
Step no 4 : click on next



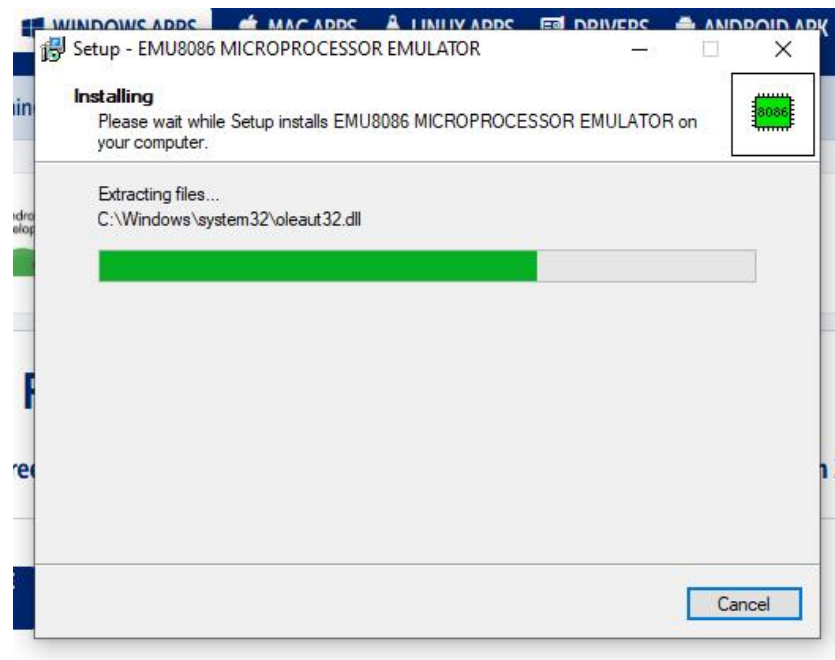
Step no 5 : click on next



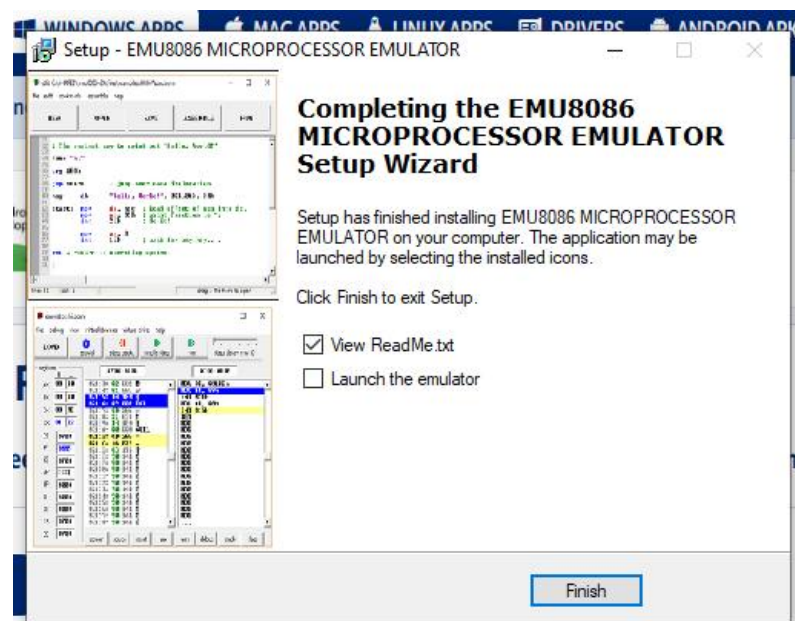
Step no 6 : click on install



Step no 7 : wait for finish

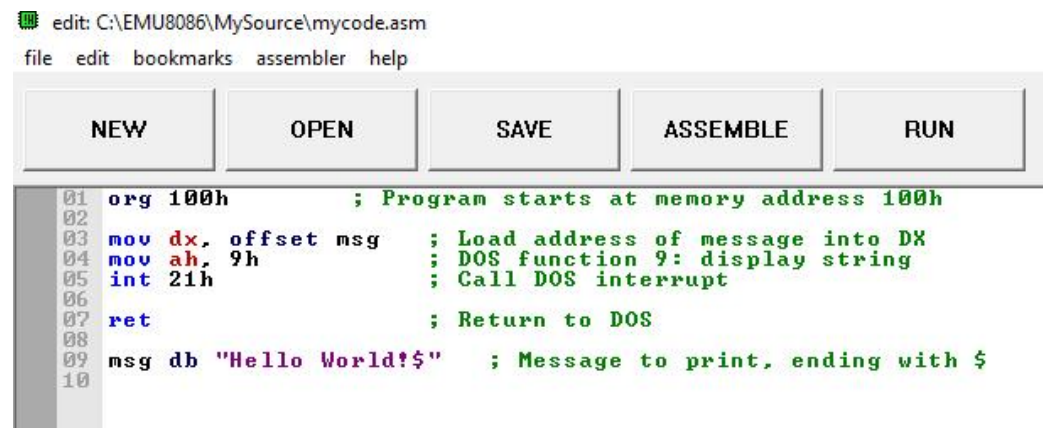


Step no 8 : click on finish

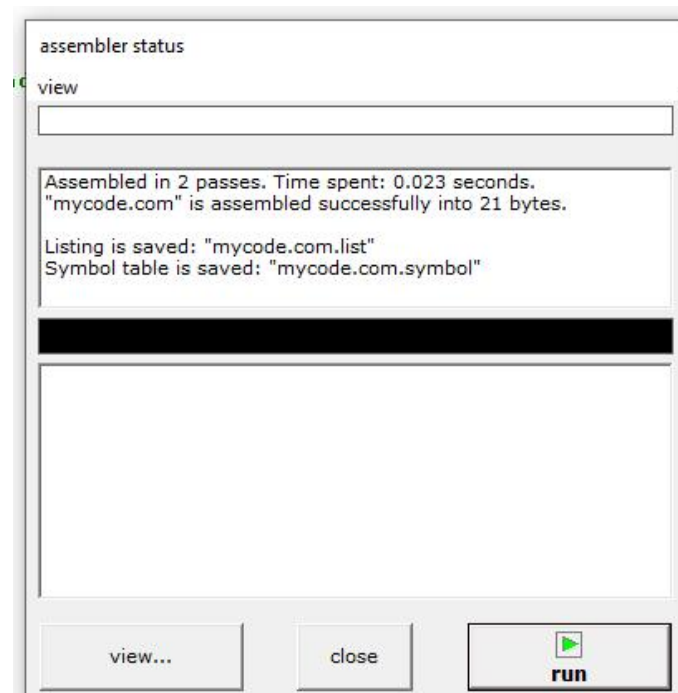


Task no 1 : write program for hello world

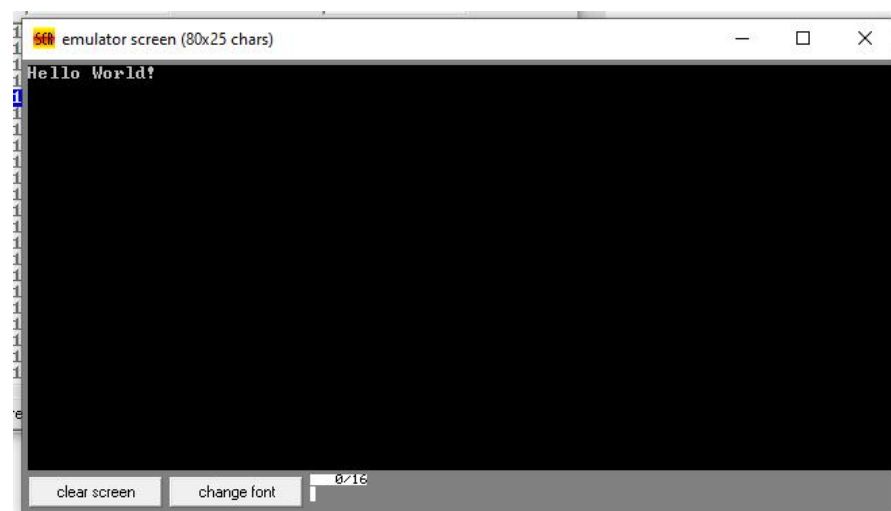
After writing code click on assemble



Click on run



Output :



LAB NO 3

REGISTERS

Registers are small, fast storage units inside the CPU used to hold data, addresses, or results temporarily.

They are faster than RAM and directly used by the processor.

Types of Registers

1. General Purpose / Data Registers

- **AX (Accumulator):** Used in arithmetic, logic, and I/O operations.
- **BX (Base):** Can hold a memory address.
- **CX (Counter):** Used as a counter in loops.
- **DX (Data):** Used in multiplication, division, and I/O.

2. Segment Registers

Store memory addresses for different parts of a program:

- **CS:** Code segment (instructions).
- **DS:** Data segment.
- **ES:** Extra segment (used in string ops).
- **SS:** Stack segment.

3. Pointer and Index Registers

Used for memory addressing and stack operations:

- **IP:** Points to next instruction.
- **SI/DI:** Used in string data movement.
- **SP:** Points to top of stack.
- **BP:** Used to access stack parameters.

Basic Assembly Commands

Command	Function	Example
MOV	Copy data	MOV AX, 10
ADD	Addition	ADD AX, 5
SUB	Subtraction	SUB BX, 2
MUL/IMUL	Multiply	MUL BX
DIV/IDIV	Divide	DIV BL
INC/DEC	Increment/Decrement	INC AX
CMP	Compare	CMP AX, BX
INT	Interrupt (DOS service)	INT 21h

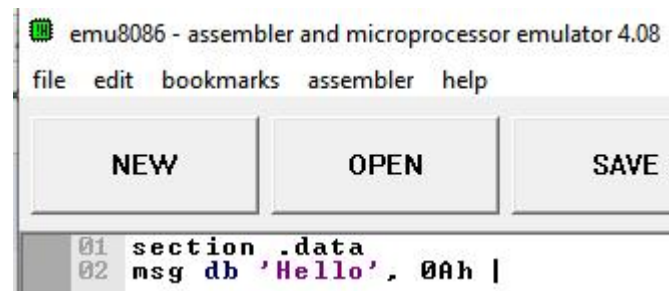
Basic Assembly Program Structure

An assembly program usually has **3 main sections**:

Data Section (section .data / .data)

Used to store **constants or fixed data** (things that don't change).

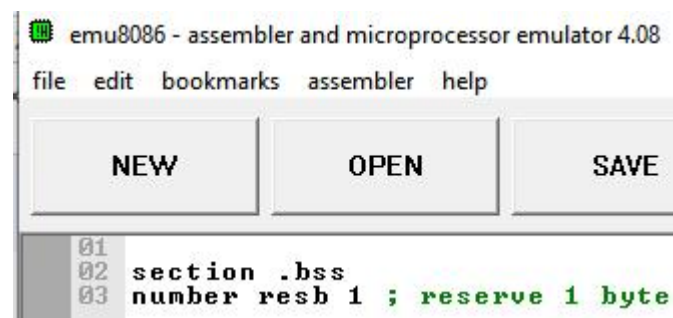
Example: messages, file names, buffer sizes.



BSS Section (section .bss / .bss)

Used to declare **variables** (memory reserved, but not initialized yet).

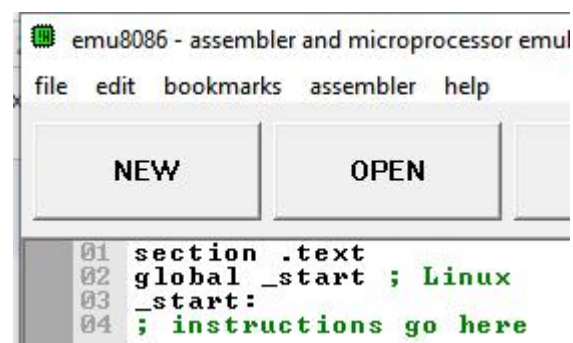
Example:



Text Section (section .text / .code)

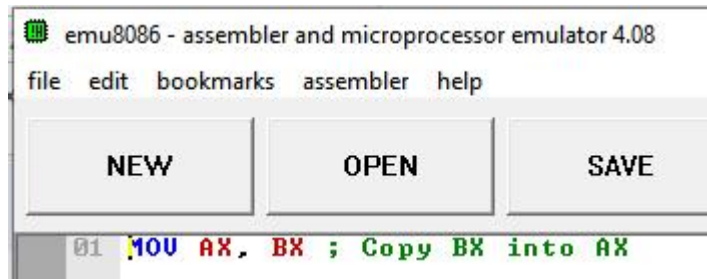
Contains the actual **instructions/code** of the program.

Program execution starts here.



Comments

Comments start with a **semicolon ;**.
They are ignored by the assembler.



Statements in Assembly

Three types:

- **Instructions** → Actual CPU commands (e.g., MOV, ADD, JMP).
- **Directives (pseudo-ops)** → Instructions for the assembler, not the CPU (e.g., .MODEL, .DATA).
- **Macros** → Shortcut blocks of code (like functions).

Directives (Special Instructions for Assembler)

- Tell the assembler how to arrange code and data.
- Do **not** generate machine instructions.

Examples:

- .MODEL → Choose memory model (tiny, small, etc.).
- .STACK → Set stack size.
- .DATA → Define initialized variables.
- .CODE → Begin code section. END → Marks end of program.
- DOSSEG → Align segments properly for DOS.

Memory Models (MASM, old DOS)

This decides how big your program can be and how code/data are stored:

- **TINY** → Code + data fit in 64KB (makes .COM files).
- **SMALL** → One code segment + one data segment (each <64KB).
- **COMPACT** → Code is small (<64KB), data can be bigger.
- **MEDIUM** → Data is small, code can be bigger.
- **LARGE** → Both code and data can be big, but arrays ≤ 64KB.
- **HUGE** → Same as LARGE but arrays can be bigger than 64KB.

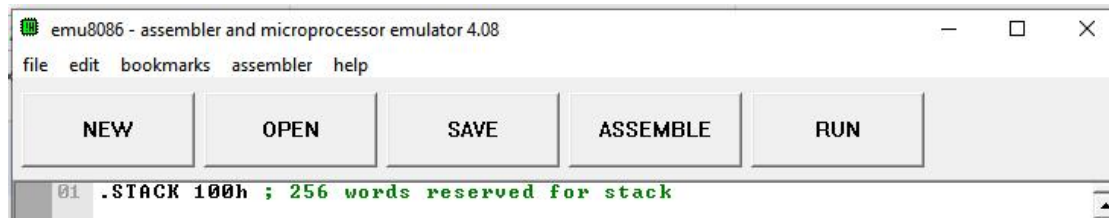
Simple rule:

- Small programs (like Hello World) → **TINY / SMALL model**.
- Big programs with lots of data/code → **LARGE / HUGE model**.

Stack Segment (.STACK)

Defines memory reserved for stack (temporary storage for calls, variables).

Example:



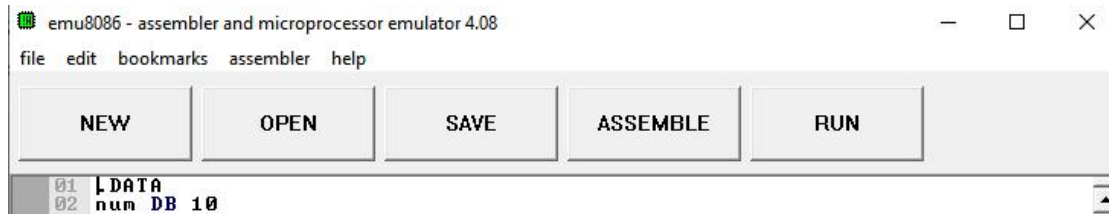
The screenshot shows the emu8086 interface with the following assembly code at line 01:

```
01 .STACK 100h ; 256 words reserved for stack
```

Data Segment (.DATA)

Stores variables/constants that are initialized.

Example:



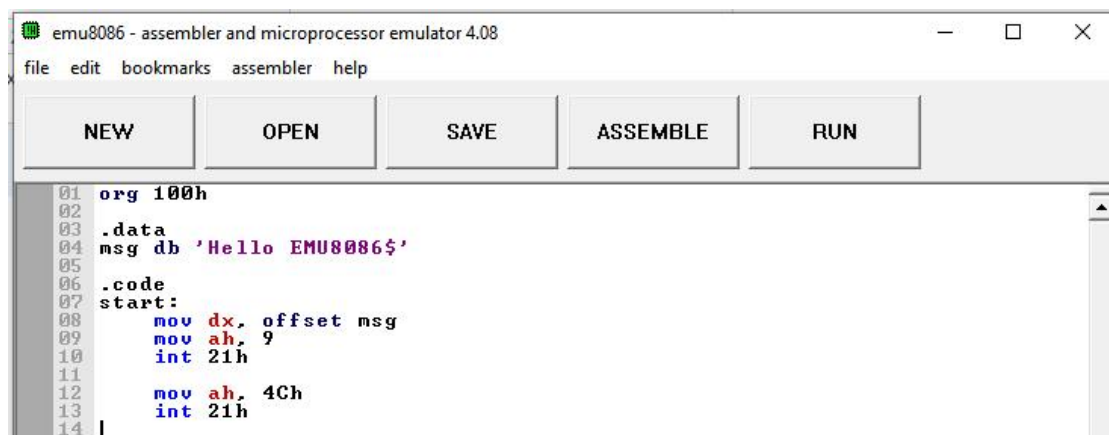
The screenshot shows the emu8086 interface with the following assembly code at lines 01 and 02:

```
01 .DATA
02 num DB 10
```

Code Segment (.CODE)

Where the actual instructions go.

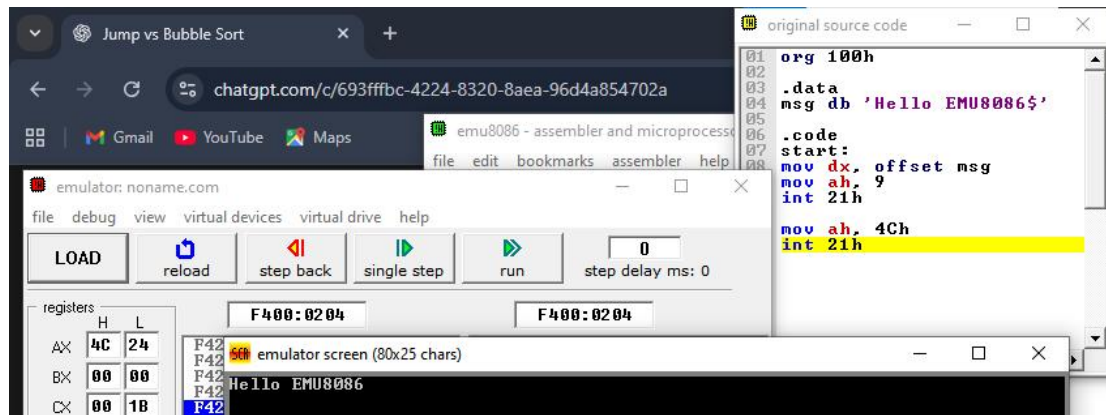
Example:



The screenshot shows the emu8086 interface with the following assembly code at lines 01 through 14:

```
01 org 100h
02
03 .data
04 msg db 'Hello EMU8086$'
05
06 .code
07 start:
08     mov dx, offset msg
09     mov ah, 9
10     int 21h
11
12     mov ah, 4Ch
13     int 21h
14
```

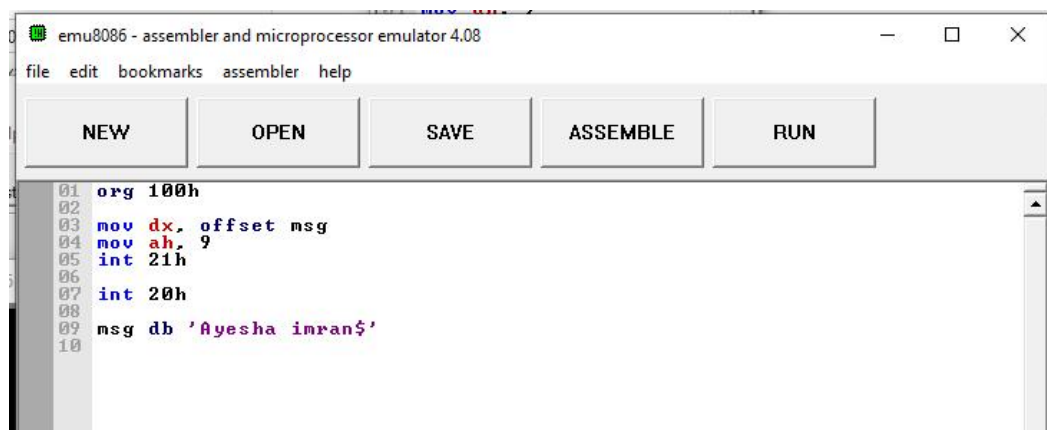
Output :



Lab Exercise

Q1: Print Your Name

Write a program to display your full name on the screen using INT 21h.



Output :



Q2: Compare Two Numbers

Compare two numbers stored in registers.



```

01 org 100h
02
03 mov ax, 5
04 mov bx, 5
05
06 cmp ax, bx
07 je equal
08
09 mov dx, offset not_equal
10 jmp print
11
12 equal:
13 mov dx, offset equal_msg
14
15 print:
16 mov ah, 9
17 int 21h
18 int 20h
19
20 equal_msg db 'EQUAL$'
21 not_equal db 'NOT EQUAL$'
22

```

Output :



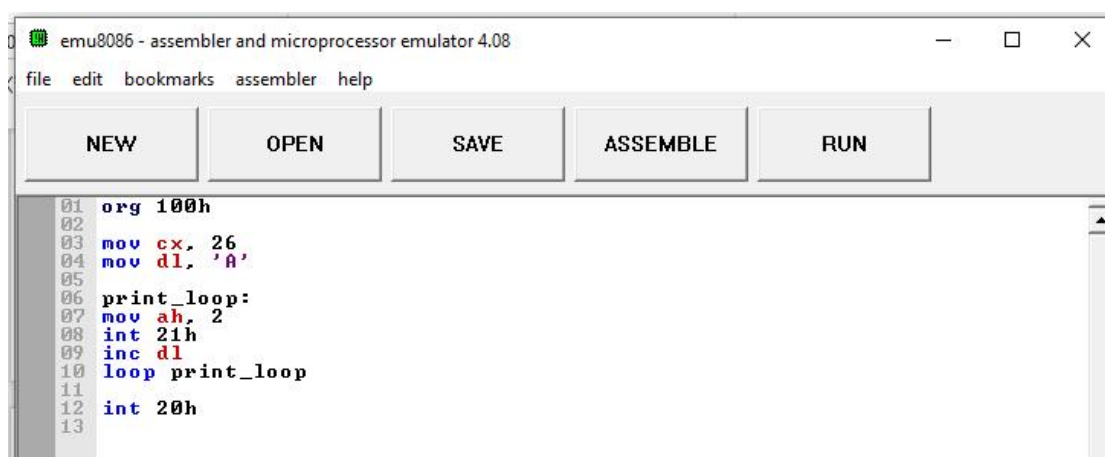
```

emu8086 screen (80x25 chars)
EQUAL

```

Q3: Print A to Z

Write a program to print all uppercase alphabets A–Z using a loop



```

01 org 100h
02
03 mov cx, 26
04 mov dl, 'A'
05
06 print_loop:
07 mov ah, 2
08 int 21h
09 inc dl
10 loop print_loop
11
12 int 20h
13

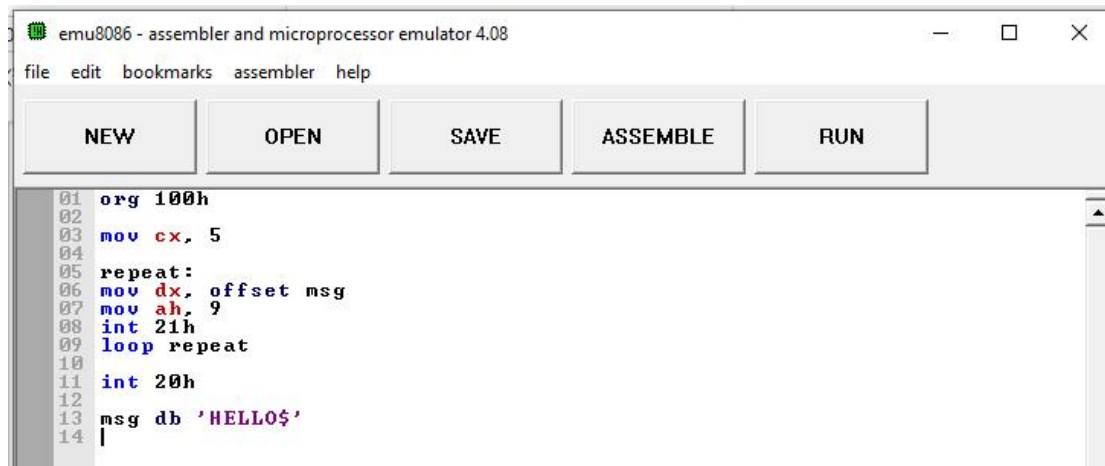
```

Output :



Q4: Print a Message 5 Times

Write a program that prints the message "HELLO" five times using a loop.

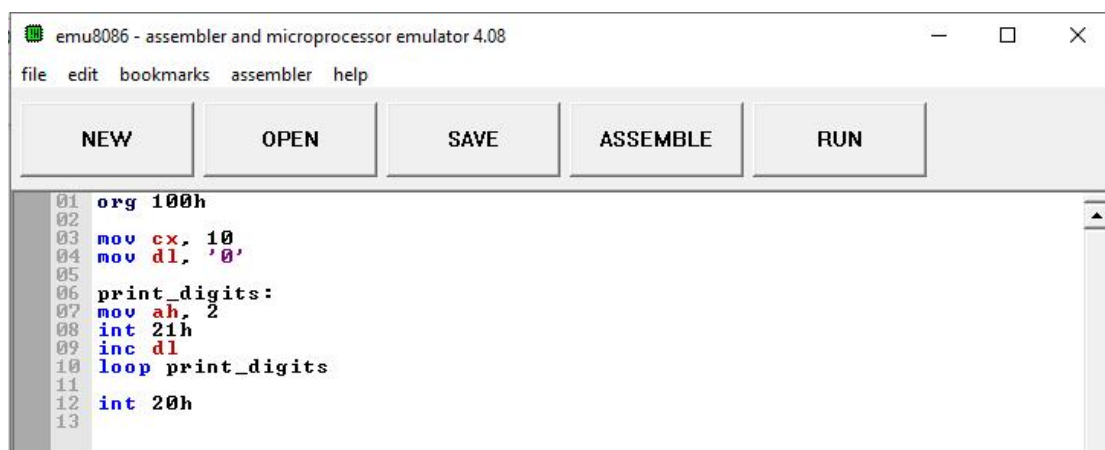


Output :



Q5: Print 0–9 (Digits)

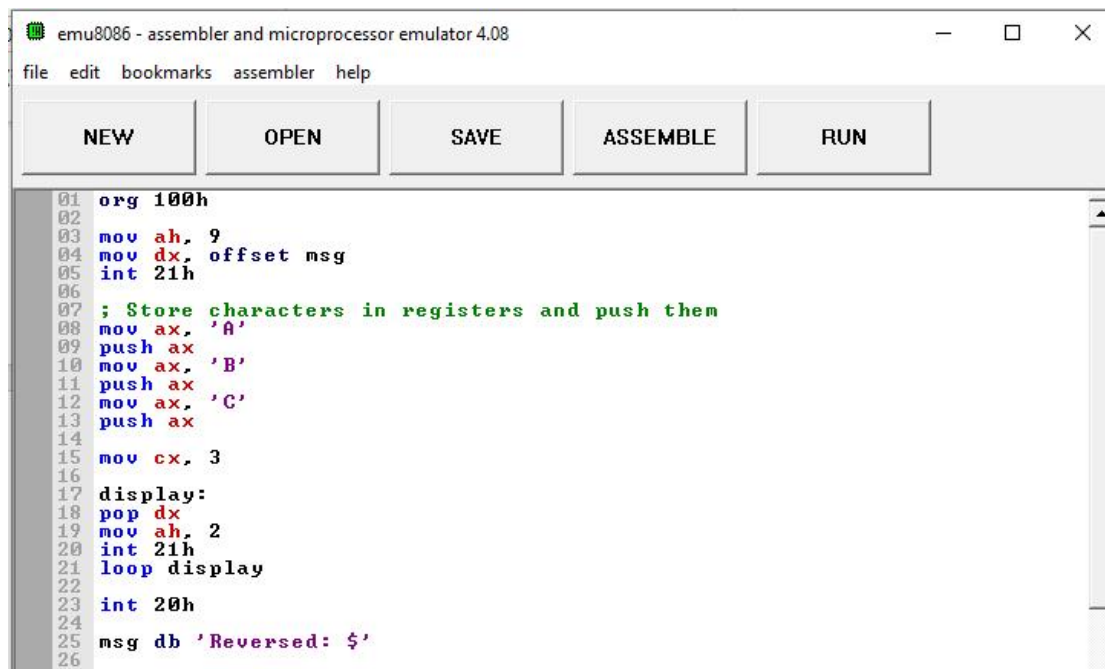
Write a program to display all digits from 0 to 9 in one line



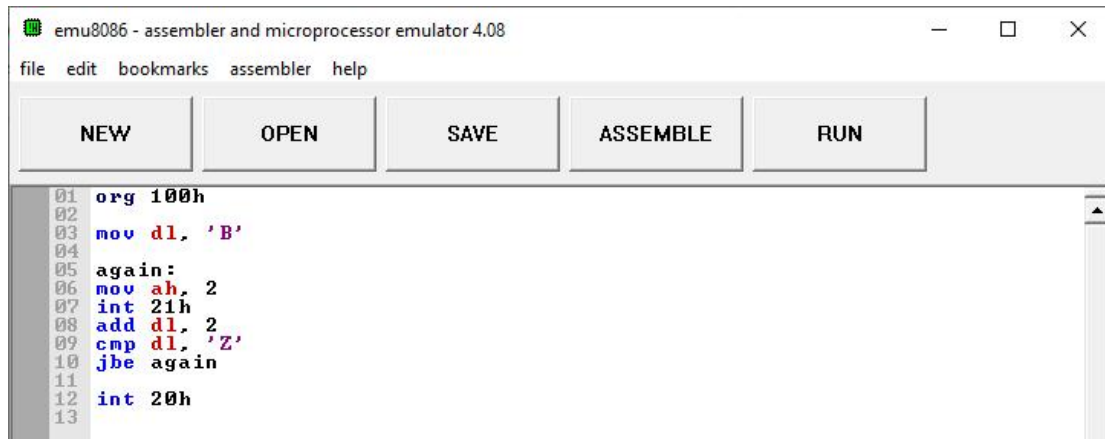
Output :**Q6: Reverse Characters Using Stack**

Push characters 'A', 'B', 'C' onto the stack, then pop them and display them back.

Expected output: CBA

**Output :****Q7: Print Even Letters Only**

Write a program to print only even-position alphabets:



```

01 org 100h
02
03 mov dl, 'B'
04
05 again:
06 mov ah, 2
07 int 21h
08 add dl, 2
09 cmp dl, 'Z'
10 jbe again
11
12 int 20h
13

```

Output :



```

BDFHJLNPRTUXZ

```

Q8: Jump Example

Write a program that checks a number in AX.
If AX contains 1, print "ONE".
Otherwise, print "NOT ONE"



```

01 org 100h
02
03 mov ax, 1
04 cmp ax, 1
05 je one
06
07 mov dx, offset not_one
08 jmp show
09
10 one:
11 mov dx, offset msg1
12
13 show:
14 mov ah, 9
15 int 21h
16
17 int 20h
18
19 msg1 db 'ONE$'
20 not_one db 'NOT ONE$'
21

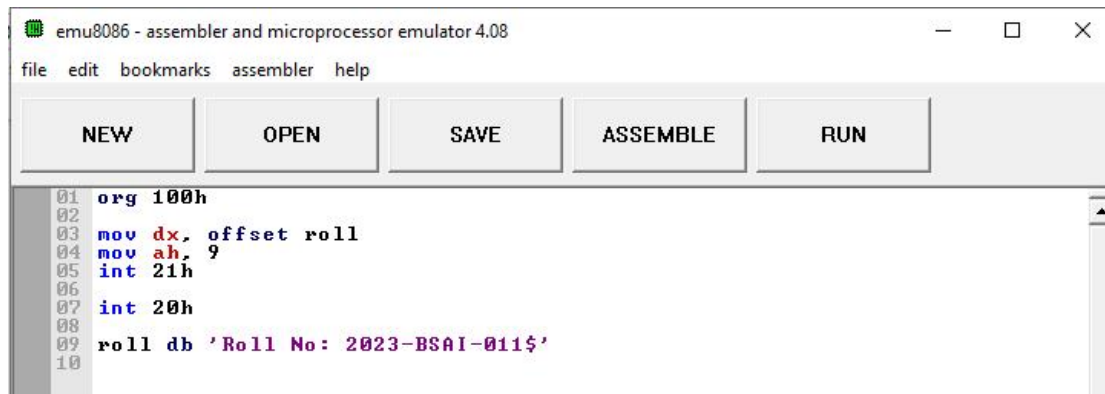
```

Output :



Q9: Print Your Roll Number

Store your roll number in a message and print it using INT 21h.



Output :



Q10: Loop with Delay

Write a program that prints * * * * * with a small pause (using a loop as delay counter)



Output :



LAB NO 4

Arithmetic Instructions in Assembly (x86 – 16-bit)

1. Introduction

Arithmetic instructions perform basic mathematical operations like addition, subtraction, multiplication, and division using CPU registers (AX, BX, CX, DX). These are essential for all computations.

2. ADD (Addition)

Syntax: ADD destination, source

Operation: Destination $\leftarrow$ Destination + Source

Example:

MOV AX, 2500h

MOV BX, 1500h

ADD AX, BX

Result:

AX = 3A00h $\rightarrow$ (2500h + 1500h = 3A00h = 14848 decimal)

3. SUB (Subtraction)

Syntax: SUB destination, source

Operation: Destination $\leftarrow$ Destination - Source

Example:

MOV AX, 4500h

MOV BX, 2500h

SUB AX, BX

Result:

AX = 2000h $\rightarrow$ (4500h - 2500h = 2000h = 8192 decimal)

4. MUL (Unsigned Multiplication)

Syntax: MUL source

Operation: DX:AX $\leftarrow$ AX $\times$ Source

Example:

MOV AX, 0006h

MOV BX, 0005h

MUL BX

Result:

AX = 001Eh (30 decimal), DX = 0000h

(6 $\times$ 5 = 30 fits in 16 bits)

5. DIV (Unsigned Division)

Syntax: DIV source

Operation: (DX:AX) $\div$ Source $\rightarrow$ AX = Quotient, DX = Remainder

Example:

MOV AX, 0030h

MOV DX, 0000h

MOV BX, 0006h

DIV BX

Result:

AX = 0008h (Quotient = 8), DX = 0000h (Remainder = 0)

6. Summary Table

Operation	Instruction	Example	Result Registers	Description
Addition	ADD	ADD AX, BX	AX	AX = AX + BX
Subtraction	SUB	SUB AX, BX	AX	AX = AX - BX
Multiplication	MUL	MUL BX	DX:AX	DX:AX = AX $\times$ BX
Division	DIV	DIV BX	AX (Q), DX (R)	AX = Quotient, DX = Remainder

Lab Exercise

Addition (ADD)

Program no 1: The value 1000h is added to 2000h. The result is stored in AX.

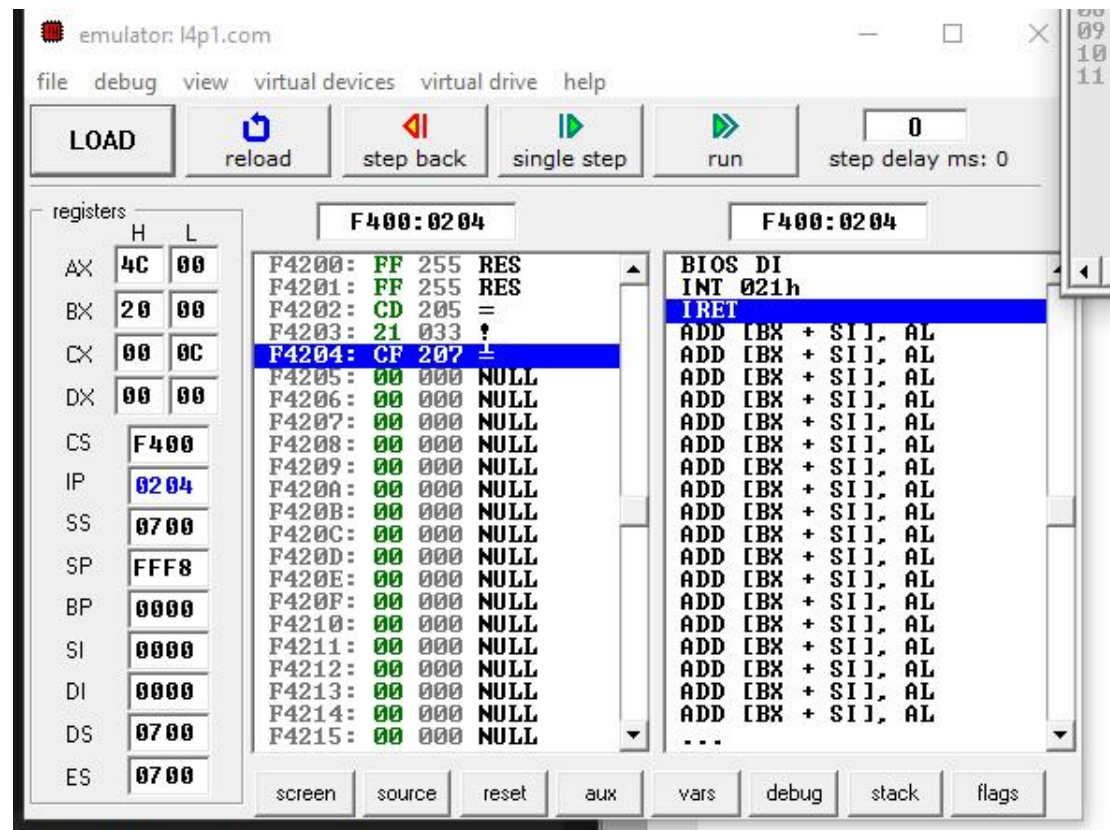
Predicted Result: AX=?

```

01 org 100h
02
03 mov ax, 1000h
04 mov bx, 2000h
05 add ax, bx
06
07 mov ah, 4Ch
08 int 21h
09

```

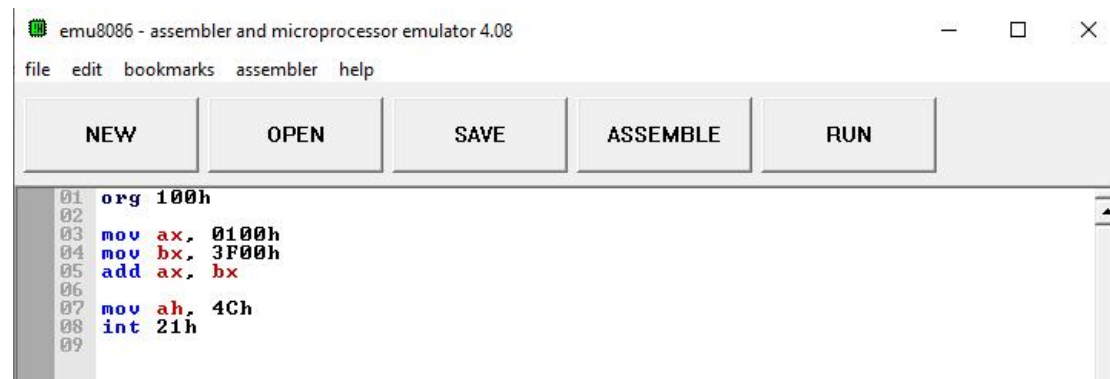
Output :



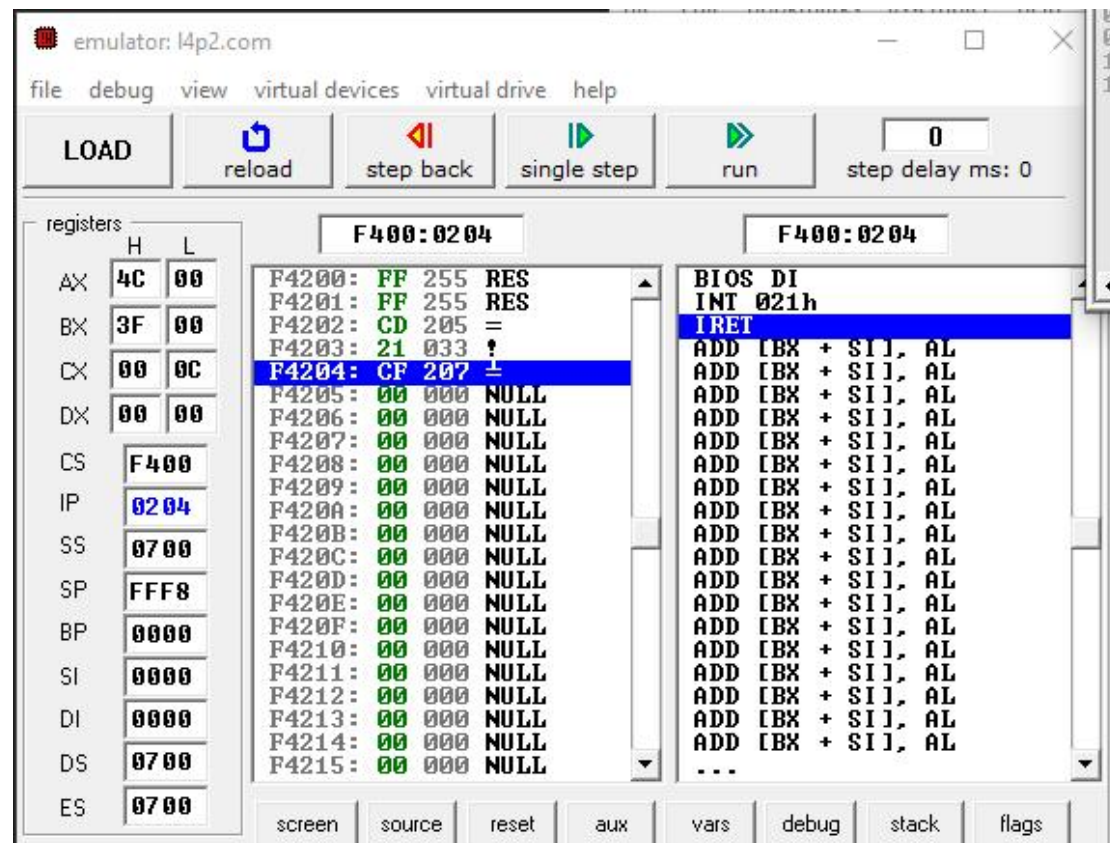
Expected Result: AX = 3000h

Program no 2 : The value 0100h is added to 3F00h. The result is stored in AX.

Predicted Result: $AX=?$



Output :

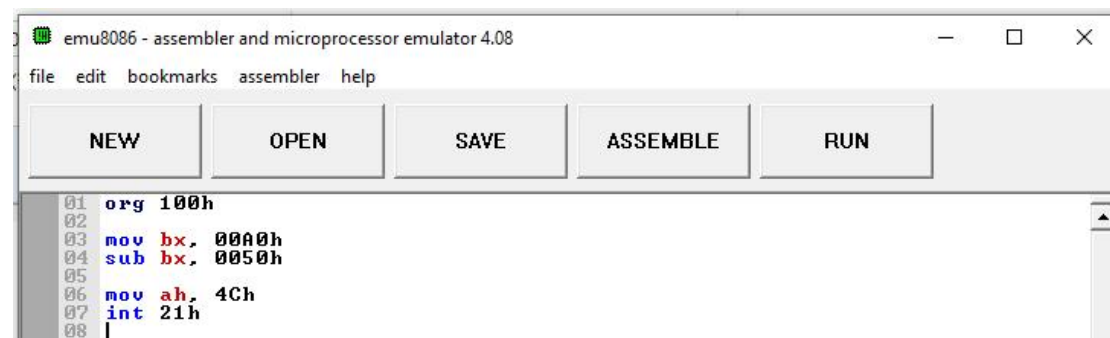


Expected Result: $AX = 4000h$

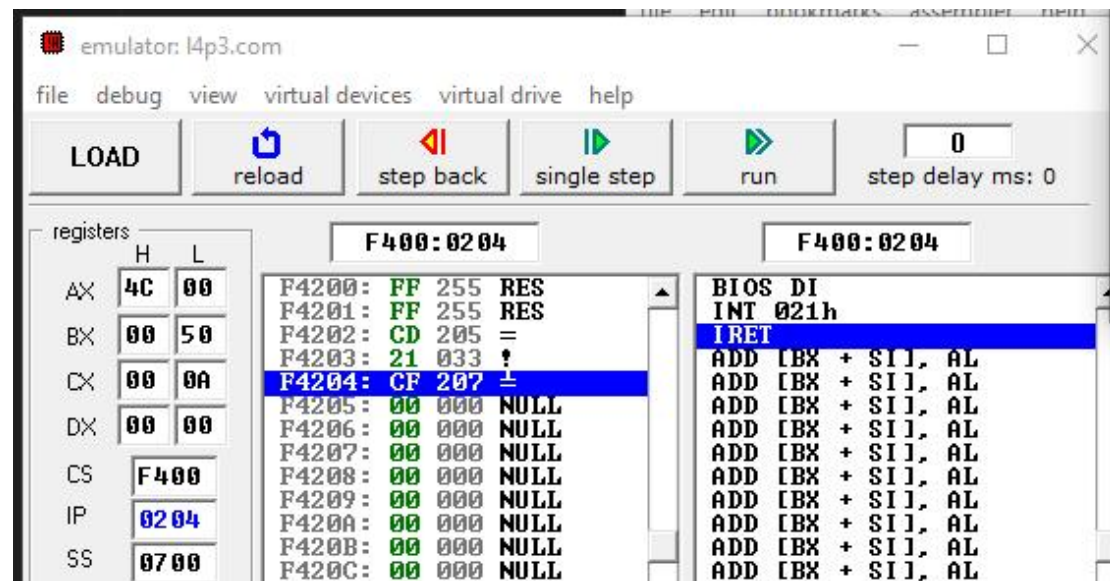
Subtraction (SUB)

Program no 3 : 0050h is subtracted from 00A0h. The result is stored in BX.

Predicted Result: BX=?



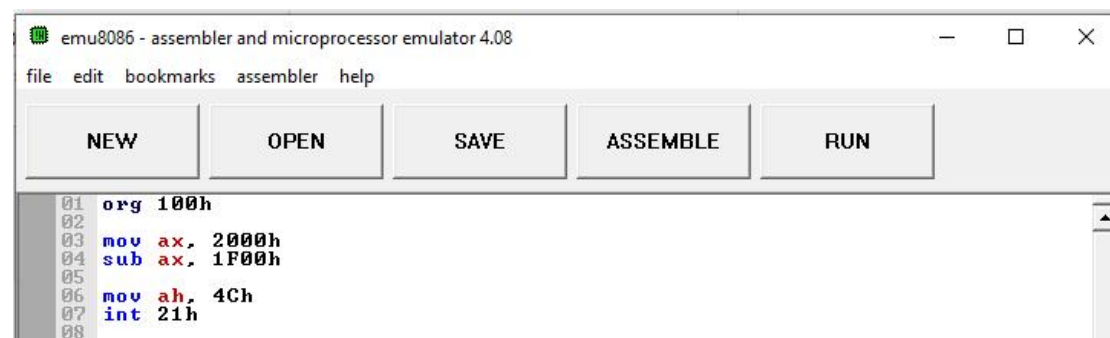
Output :



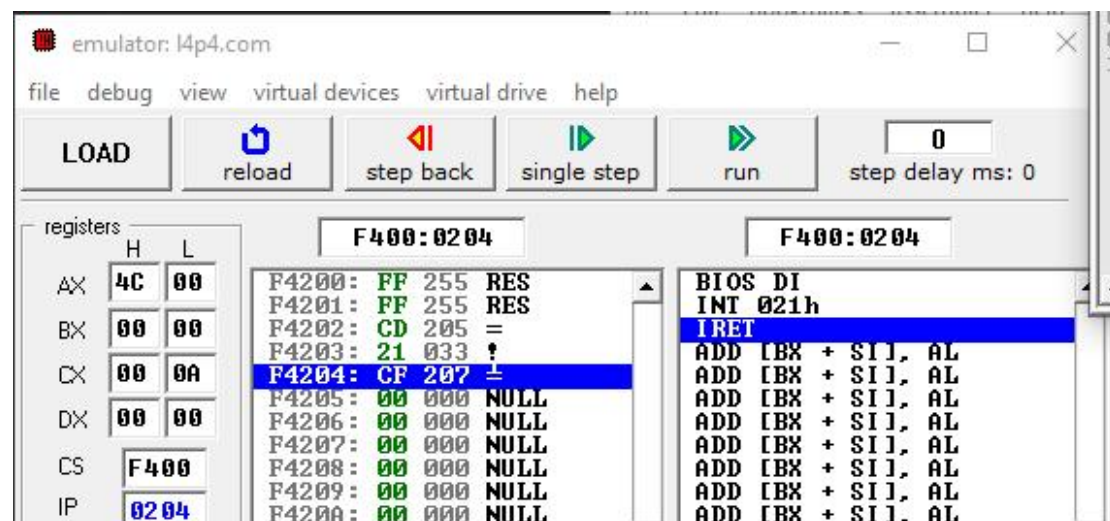
Expected Result: BX = 0050h

Program no 4 : 1F00h is subtracted from 2000h. The result is stored in AX.

Predicted Result: AX=?



Output :



Expected Result: AX = 0100h

Program no 5 : 0001h is subtracted from 0000h. The result is stored in AX (This is -1 in two's complement). **Predicted Result:** AX=?

```

01 org 100h
02
03 mov ax, 0000h
04 sub ax, 0001h
05
06 mov ah, 4Ch
07 int 21h
08

```

Output:

Register	H	L
AX	4C	FF
BX	00	00
CX	00	0A
DX	00	00
CS	F400	
IP	0204	

Address	Hex	Dec	Comment
F4200:	FF	255	RES
F4201:	FF	255	RES
F4202:	CD	205	=
F4203:	21	033	!
F4204:	CF	207	=
F4205:	00	000	NULL
F4206:	00	000	NULL
F4207:	00	000	NULL
F4208:	00	000	NULL
F4209:	00	000	NULL
F420A:	00	000	NULL

Expected Result: AX = FFFFh (-1 in two's complement)

Multiplication (MUL)

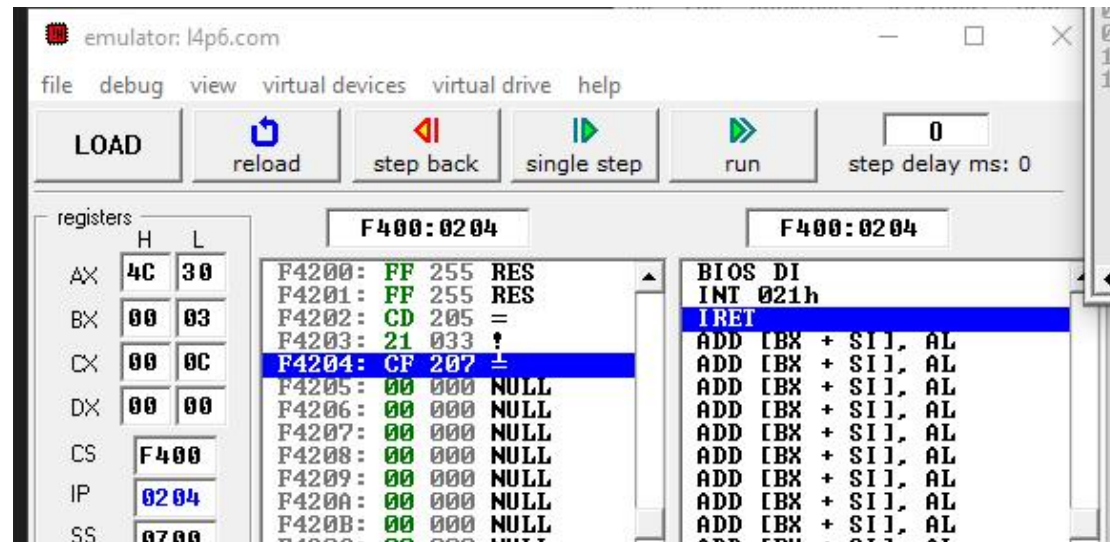
Program no 6 : The number 10h is multiplied by 03h. The 32-bit product is stored in DX:AX. **Predicted Result:** DX=?, AX=?

```

01 org 100h
02
03 mov ax, 10h
04 mov bx, 03h
05 mul bx
06
07 mov ah, 4Ch
08 int 21h
09

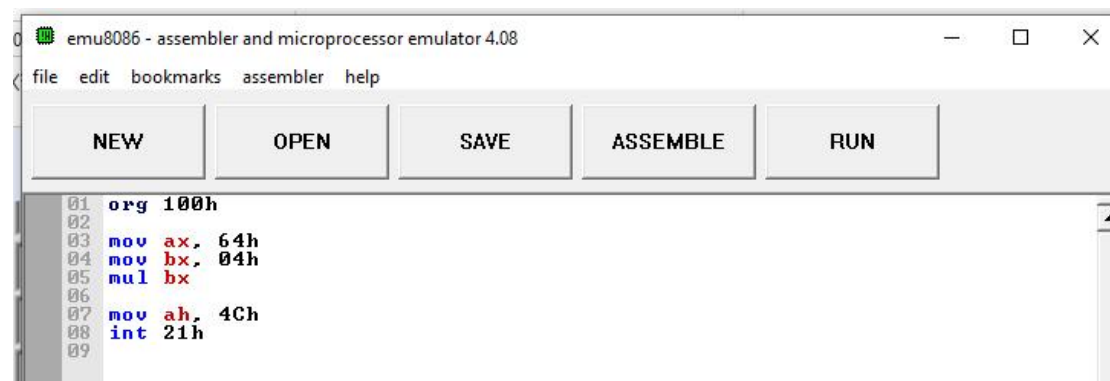
```

Output :

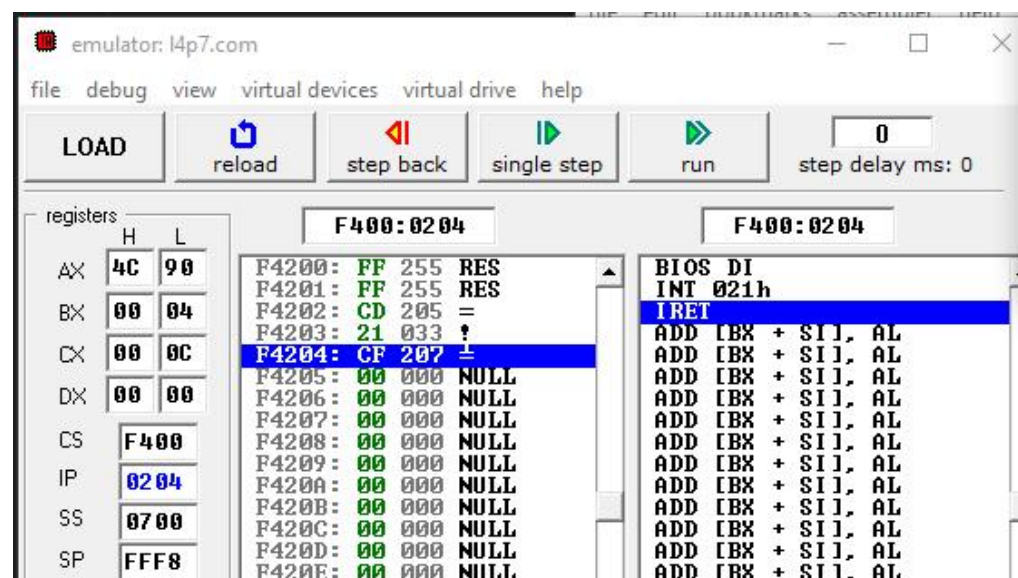


Expected Result: AX = 0030h (48 decimal), DX = 0000h

Program no 7 : The number 100 decimal (64h) is multiplied by 4. The 32-bit product is stored in DX:AX. **Predicted Result:** DX=?, AX=?



Output :



Expected Result: AX = 0190h (400 decimal), DX = 0000h

Division (DIV)

Program no 8 : 20h is divided by 04h. The quotient is stored in AX, and the remainder in DX. (The dividend is 0000:0020h).

Predicted Result: AX=? (Quotient), DX=? (Remainder)

```

01 org 100h
02
03 mov dx, 0000h
04 mov ax, 0020h
05 mov bx, 0004h
06 div bx
07
08 mov ah, 4Ch
09 int 21h
10

```

Output :

emulator: i4p8.com

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers

	H	L
AX	4C	08
BX	00	04
CX	00	0F
DX	00	00
CS	F400	
IP	0204	

F400:0204

F4200:	FF	255	RES
F4201:	FF	255	RES
F4202:	CD	205	=
F4203:	21	033	!
F4204:	CF	207	±
F4205:	00	000	NULL
F4206:	00	000	NULL
F4207:	00	000	NULL
F4208:	00	000	NULL
F4209:	00	000	NULL
F420A:	00	000	NULL
F420B:	00	000	NULL

F400:0204

BIOS DI	
INT 021h	
IRET	
ADD [BX + SI], AL	
ADD [BX + SI], AL	
ADD [BX + SI], AL	
ADD [BX + SI], AL	
ADD [BX + SI], AL	
ADD [BX + SI], AL	
ADD [BX + SI], AL	
ADD [BX + SI], AL	
ADD [BX + SI], AL	
ADD [BX + SI], AL	

Expected Result: AX = 0008h (Quotient = 8), DX = 0000h (Remainder = 0)

Program no 9: 1Ah (26 decimal) is divided by 05h (5 decimal). The quotient is stored in AX, and the remainder in DX.

Predicted Result: AX=? (Quotient), DX=? (Remainder)

```

01 org 100h
02
03 mov dx, 0000h
04 mov ax, 001Ah
05 mov bx, 0005h
06 div bx
07
08 mov ah, 4Ch
09 int 21h
10

```

Output :

Register	H	L
AX	4C	05
BX	00	05
CX	00	0F
DX	00	01
CS	F400	
IP	0204	

Address	Hex	Dec	Comment
F4200	FF	255	RES
F4201	FF	255	RES
F4202	CD	205	=
F4203	21	033	!
F4204	CF	207	IRET
F4205	00	000	NULL
F4206	00	000	NULL
F4207	00	000	NULL
F4208	00	000	NULL
F4209	00	000	NULL
F420A	00	000	NULL

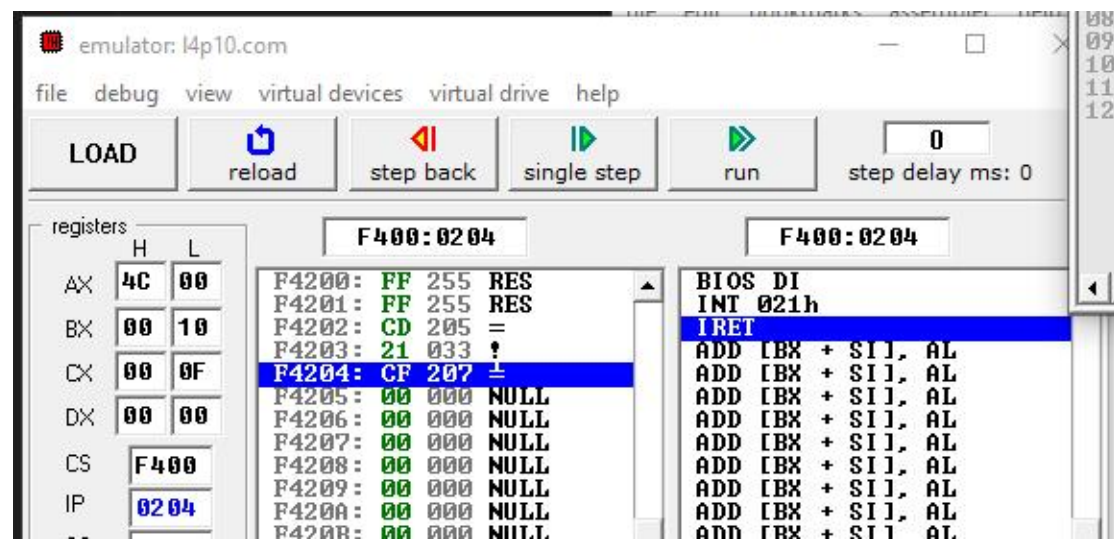
Expected Result: AX = 0005h (Quotient = 5), DX = 0001h (Remainder = 1)

Program no 10 : 1000h is divided by 10h. The quotient is stored in AX, and the remainder in DX. **Predicted Result:** AX=? (Quotient), DX=? (Remainder)

```

01 org 100h
02
03 mov dx, 0000h
04 mov ax, 1000h
05 mov bx, 0010h
06 div bx
07
08 mov ah, 4Ch
09 int 21h
10

```

Output :

Expected Result: AX = 0100h (Quotient = 256), DX = 0000h (Remainder = 0)

LAB NO 5

ADDRESSING MODES

Data Declaration

In 8086 assembly language, data is declared in the **DATA SEGMENT** using different **data types**.

Directive	Meaning	Size
DB	Define byte	1 byte
DW	Define word	2 bytes
DD	Define double word	4 bytes
DQ	Define quad word	8 bytes

Example:



emu8086 - assembler and microprocessor emulator 4.08

file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

```
01 DATA SEGMENT
02 A DB 10H
03 B DB 20H
04 C DW 1234H
05 MSG DB "HELLO$"
06 DATA ENDS
07 |
```

Output :

The screenshot shows the emu8086 emulator window. The title bar reads "emu8086 - assembler and microprocessor simulator". The menu bar includes "file", "edit", "bookmarks", "assembler", and "help". The main window is titled "emulator: noname.bin". Below the title bar is a toolbar with buttons for "LOAD", "reload", "step back", "single step", "run", and a "step delay ms: 0" input field. The "registers" section on the left shows the state of the 8086 registers: AX (00 10), BX (00 00), CX (00 00), DX (00 00), and CS (0100). The main display area shows assembly code with addresses and hex values. The code is as follows:

```

01018: 90 144 E  NOP
01019: 90 144 E  NOP
0101A: 90 144 E  NOP
0101B: 90 144 E  NOP
0101C: 90 144 E  NOP
0101D: 90 144 E  NOP
0101E: F4 244 0  HLT
0101F: 00 000 0  ADD CBX + SI, AL
01020: 00 000 0  ADD CBX + SI, AL
01021: 00 000 0  ADD CBX + SI, AL

```

The "MSG DB 'HELLO\$'" instruction is highlighted in yellow in the original image. The "HLT" instruction is highlighted in blue in the original image.

Direct Addressing

In **direct addressing**, the operand's **memory name** or **offset** is used directly in the instruction.

Example:

```

01 DATA SEGMENT
02 A DB 10H
03 B DB 20H
04 RESULT DB ?
05 DATA ENDS
06
07 CODE SEGMENT
08 ASSUME CS:CODE, DS:DATA
09 START:
10 MOV AX, DATA
11 MOV DS, AX
12
13 MOV AL, A
14 MOV BL, B
15 ADD AL, BL
16 MOV RESULT, AL
17
18 MOV AH, 4CH
19 INT 21H
20 CODE ENDS
21 END START

```

Output :

The screenshot shows the emu8086 emulator interface. The registers window displays the state of the processor, including the AX register (4C 30) and the DS register (F400). The original source code window shows the code being executed, with the instruction `INT 21H` highlighted.

Explanation:

The processor accesses data by directly using its memory label (e.g., A, B, etc.).

Size Mismatch Errors

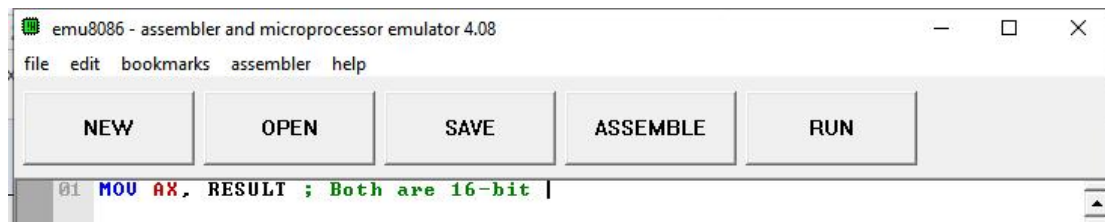
A **size mismatch** occurs when the source and destination operands are of **different sizes**.

Example (Error):

```

01 MOV AL, RESULT ; RESULT is a word, AL is a byte ? size mismatch

```

Correct Version:**Explanation:**

Always ensure that both operands have the **same data size** — either byte or word.

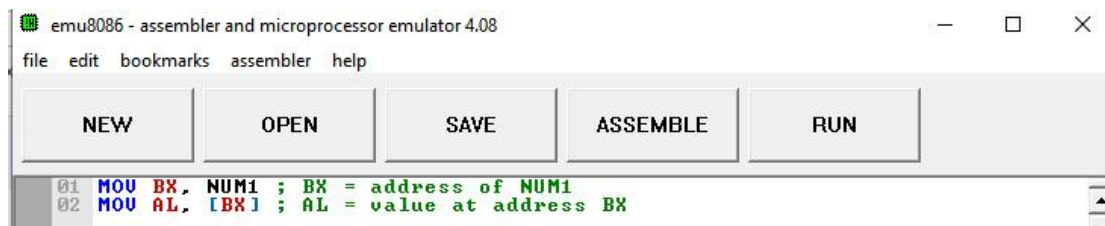
Register Indirect Addressing

In this mode, a **register** holds the **address** of the operand.

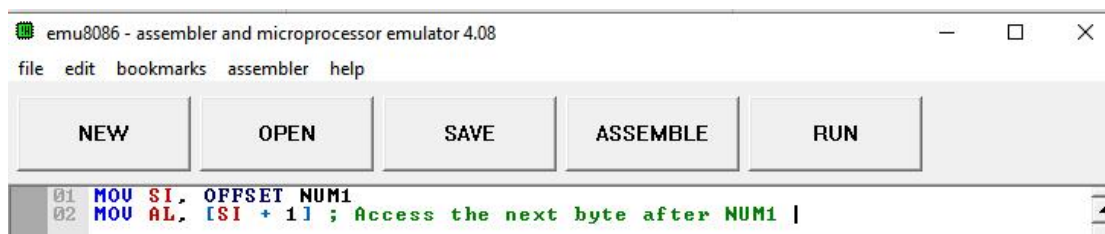
Registers allowed: BX, BP, SI, DI

MOV destination, [register]

means "use the address stored in the register"

Example:**Register + Offset Addressing**

In this mode, an **offset** (constant value) is added to the **address stored in a register**.

Example:**Explanation:**

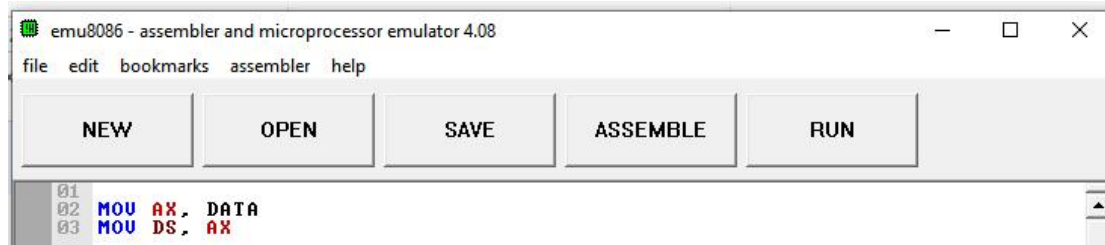
The final address is calculated by adding the value of SI and 1 (offset).

Segment Association

In 8086, memory is divided into **segments** (like DATA, CODE, STACK).

To access variables from the **DATA segment**, it must be associated with DS (Data Segment Register).

Example:



This ensures that all variables like A, B, NUM1 belong to the **current data segment**.

Observation Table

mode	Example	Explanation
Direct	MOV AL, A	Access memory label directly
Register Indirect	MOV AL, [BX]	Access data using address in register
Register + Offset	MOV AL, [SI + 1]	Access data by adding offset
Segment Association	MOV DS, AX	Connect data segment to DS register

Lab Exercise

Program no 1: write a program in 8086 assembly language that uses all addressing modes learned in this lab :

1. Direct addressing
2. Register indirect addressing
3. Register + offset addressing
4. Segment association

Requirements :

- Use arithmetic instructions : ADD, SUB, MUL, DIV
- Store the result of each arithmetic operations in memory
- Display clear comments showing which addressing mode is used

emu8086 - assembler and microprocessor emulator 4.08

file edit bookmarks assembler help

NEW	OPEN	SAVE	ASSEMBLE	RUN
-----	------	------	----------	-----

```

01 .model small
02 .stack 100h
03 .data
04
05 A db 5
06 B db 4
07 RES1 db 0
08
09 ARR db 9,2
10 RES2 db 0
11
12 NUM1 db 6,7
13 RES3 db 0
14
15 RES4 dw 0
16 RES5 db 0
17
18 .code
19 main proc
20     mov ax, @data
21     mov ds, ax
22
23     mov al, A
24     add al, B
25     mov RES1, al
26
27     mov si, OFFSET ARR
28     mov al, [si]
29     sub al, 1
30     mov RES2, al
31
32     mov si, OFFSET NUM1
33     mov al, [si+1]
34     add al, [si]
35     mov RES3, al
36
37     mov al, A
38     mov bl, B
39     mul bl
40     mov RES4, ax
41
42     mov ax, 100
43     mov bl, B
44     div bl
45     mov RES5, al
46
47     mov ah, 4CH
48     mov al, 0
49     int 21h
50
51 main endp
52 end main

```

Output :

emulator: noname.exe

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	01	52
DX	00	00
CS	0721	
IP	0000	
SS	0710	
SP	0100	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0050:0000 0050:0000

00500: 00 000 NULL	MOV AX, 00720h
00501: 00 000 NULL	MOV DS, AX
00502: 00 000 NULL	MOV AL, [00000h]
00503: 00 000 NULL	ADD AL, [00001h]
00504: 00 000 NULL	MOV [00002h], AL
00505: 00 000 NULL	MOV SI, 00003h
00506: 00 000 NULL	MOV AL, [SI]
00507: 00 000 NULL	SUB AL, 01h
00508: 00 000 NULL	MOV [00005h], AL
00509: 00 000 NULL	MOV SI, 00006h
0050A: 00 000 NULL	MOV AL, [SI] + 01h
0050B: 00 000 NULL	ADD AL, [SI]
0050C: 00 000 NULL	MOV [00008h], AL
0050D: 00 000 NULL	MOV AL, [00000h]
0050E: 00 000 NULL	MOV BL, [00001h]
0050F: 00 000 NULL	MUL BL
00510: 00 000 NULL	MOV [00009h], AX
00511: 00 000 NULL	MOV AX, 00064h
00512: 00 000 NULL	MOV BL, [00001h]
00513: 00 000 NULL	DIV BL
00514: 00 000 NULL	...
00515: 00 000 NULL	

screen source reset aux vars debug stack flags

LAB NO 6

DISPLAY A MESSAGE

In Assembly Language (8086), displaying a message on the screen means sending characters or strings to the output device (monitor). This is usually done with the help of DOS interrupt services. The 8086 processor itself cannot directly display text. It uses software interrupts, especially INT 21H, which provides various input/output services.

Common Method to Display a Message

The most commonly used DOS function for displaying a message is:

INT 21H
Function 09H

How it Works:

- The message must be stored in memory
- The message must end with \$
- The offset address of the message is loaded into DX register
- Function number 09H is loaded into AH register
- INT 21H is executed

Registers Used

Register	Purpose
AH	Holds function number (09H)
DX	Holds offset address of message
DS	Data segment register
INT 21H	DOS interrupt for display

Steps to Display a Message

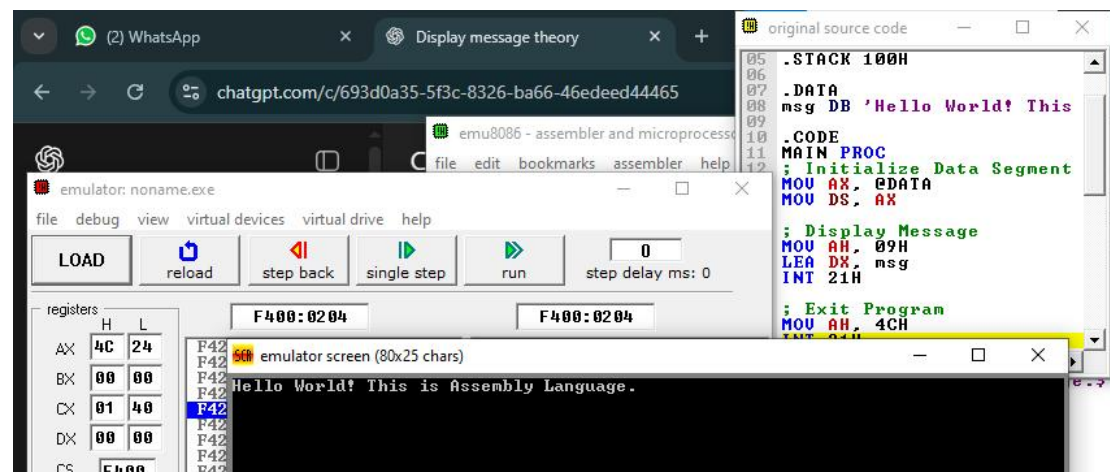
1. Define the message in the DATA segment
2. Initialize the DATA segment
3. Load function number 09H into AH
4. Load message address into DX
5. Call INT 21H
6. Terminate program properly

Lab Exercise

Program no 1 : Display a message



Output :



LAB NO 7

LOGICAL OPERATORS

In Assembly Language, logic operators are used to perform bit-wise operations on data. These operations work bit by bit on binary numbers and are mainly used for:

- Decision making
- Masking bits
- Setting or clearing specific bits
- Comparing values
- Low-level hardware control

In 8086 microprocessor, logic operators do not use arithmetic; instead, they manipulate binary bits directly.

Common Logic Operators in 8086

1. AND Operator (AND)

- Compares each bit of two operands
- Result bit is 1 only if both bits are 1

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

- Used for: Masking bits

2. OR Operator (OR)

- Result bit is 1 if any one bit is 1

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

- Used for: Setting bits

3. XOR Operator (XOR)

- Result bit is 1 if bits are different

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

- Used for: Toggling bits

4. NOT Operator (NOT)

- Inverts all bits ($1 \rightarrow 0, 0 \rightarrow 1$)

A	A NOT
1	0
0	1

- Used for: Bit inversion

Flags Affected

Logic operations affect:

- ZF (Zero Flag)
- SF (Sign Flag)
- PF (Parity Flag)

Carry and overflow flags are cleared.

Syntax

AND destination, source

OR destination, source

XOR destination, source

NOT destination

Lab Exercise

Program no 1 : Demonstration of AND logical operator

```

01 ; Program to demonstrate AND logic operator
02 ; Assembly Language (EMU8086)
03
04 .MODEL SMALL
05 .STACK 100H
06
07 .DATA
08     num1 DB 0FH      ; 00001111
09     num2 DB 03H      ; 00000011
10     result DB ?
11
12 .CODE
13 MAIN PROC
14     ; Initialize Data Segment
15     MOV AX, @DATA
16     MOV DS, AX
17
18     ; Load numbers
19     MOV AL, num1
20     AND AL, num2      ; AND operation
21
22     ; Store result
23     MOV result, AL
24
25     ; Exit program
26     MOV AH, 4CH
27     INT 21H
28 MAIN ENDP
29
30 END MAIN
31

```

Output :

The screenshot shows the emu8086 emulator interface. The registers window displays the following values:

Register	Value
AX	4C 03
BX	00 00
CX	01 23
DX	00 00
IP	02 04
CS	F4 00
SS	07 10
SP	00 FA
BP	00 00
SI	00 00
DI	00 00
DS	07 20
ES	07 00

The memory window shows the following values:

Address	Value
F400:0204	0F 23
F400:0205	00 00
F400:0206	00 00
F400:0207	00 00
F400:0208	00 00
F400:0209	00 00
F400:020A	00 00
F400:020B	00 00
F400:020C	00 00
F400:020D	00 00
F400:020E	00 00
F400:020F	00 00
F400:0210	00 00
F400:0211	00 00
F400:0212	00 00
F400:0213	00 00
F400:0214	00 00
F400:0215	00 00

The original source code window shows the following code:

```

09 num2 DB 03H      ; 00000011
10 result DB ?
11
12 .CODE
13 MAIN PROC
14     ; Initialize Data Segment
15     MOV AX, @DATA
16     MOV DS, AX
17
18     ; Load numbers
19     MOV AL, num1
20     AND AL, num2      ; AND operation
21
22     ; Store result
23     MOV result, AL
24
25     ; Exit program
26     MOV AH, 4CH
27     INT 21H

```

Explanation of Example

- num1 = 0FH → 00001111
- num2 = 03H → 00000011
- AND Result → 00000011 (03H)
- Result is stored in result

LAB NO 8

ARRAYS

In Assembly Language, an array is a collection of elements of the same data type stored in consecutive memory locations.

Arrays are used when we need to store and process multiple values under a single name.

Unlike high-level languages, 8086 Assembly does not provide built-in array support. Instead, arrays are handled using:

- Memory addressing
- Index registers (SI, DI)
- Loop instructions

Each element of an array is accessed by calculating its offset address.

Types of Arrays in Assembly

1. Byte Array

arr DB 10, 20, 30

2. Word Array

arr DW 100, 200, 300

Registers Commonly Used

Register	Purpose
SI	Source Index (array indexing)
DI	Destination Index
CX	Loop counter
DS	Data segment

Accessing Array Elements

- First element → base address
- Next element → base + index

Example:

MOV AL, arr[SI]

Advantages of Arrays

- Stores multiple values efficiently
- Reduces number of variables
- Makes looping and processing easier

Lab Exercise

Program no 1 : sum of elements of an array

emu8086 - assembler and microprocessor emulator 4.08

file edit bookmarks assembler help

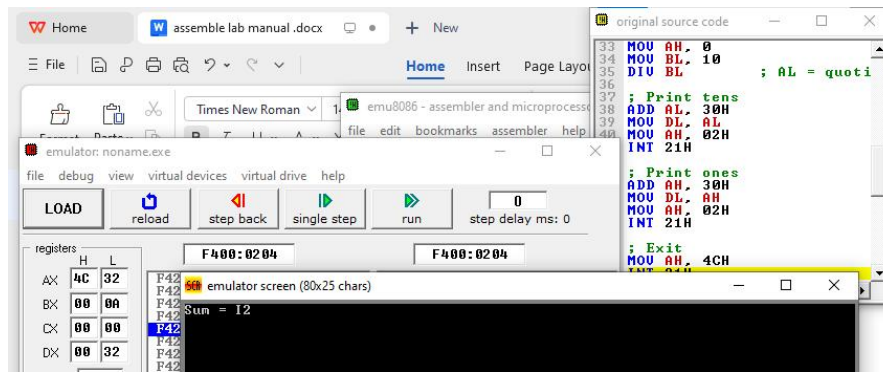
NEW OPEN SAVE ASSEMBLE RUN

```

01 ; Array Sum with Proper DisplayA
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05     arr DB 5, 6, 7, 8
06     count DB 4
07     msg DB 'Sum = $'
08 .CODE
09 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12
13     ; Print message
14     MOV AH, 09H
15     LEA DX, msg
16     INT 21H
17
18     ; Calculate sum
19     MOV SI, 0
20     MOV CL, count
21     MOV AL, 0
22
23 SUM_LOOP:
24     ADD AL, arr[SI]
25     INC SI
26     LOOP SUM_LOOP
27
28     ; Convert to decimal
29     MOV AH, 0
30     MOV BL, 10
31     DIV BL      ; AL = quotient, AH = remainder
32
33     ; Print tens
34     ADD AL, 30H
35     MOV DL, AL
36     MOV AH, 02H
37     INT 21H
38
39     ; Print ones
40     ADD AH, 30H
41     MOV DL, AH
42     MOV AH, 02H
43     INT 21H
44
45     ; Exit
46     MOV AH, 4CH
47     INT 21H
48 MAIN ENDP
49 END MAIN

```

Output :



LAB NO 9

LOOPS

1. AIM

To study and implement **loops** in 8086 assembly language using EMU8086.

2. THEORY

Loop: A set of instructions that repeats a specific number of times.

LOOP instruction:

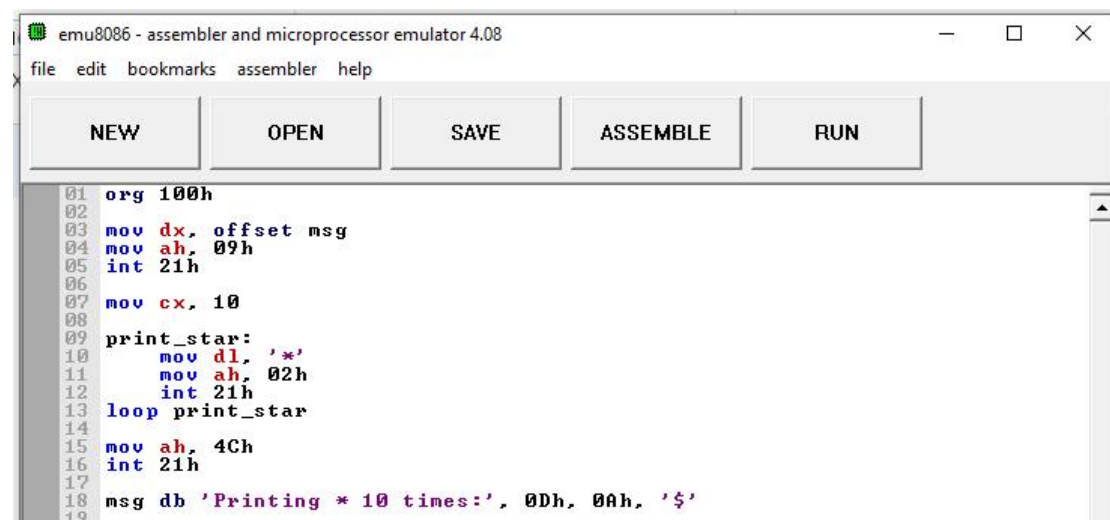
- Uses **CX** as counter.
- Decrements **CX** automatically.
- Jumps to a label if $CX \neq 0$.

Conditional jumps (JE, JNE, JC, JNC, etc.) can also create loops based on flags.

3. ALGORITHM

1. Initialize **CX** (loop counter).
2. Start a loop label.
3. Execute instructions inside the loop.
4. Use **LOOP** or conditional jump to repeat.
5. Exit program when loop finishes.

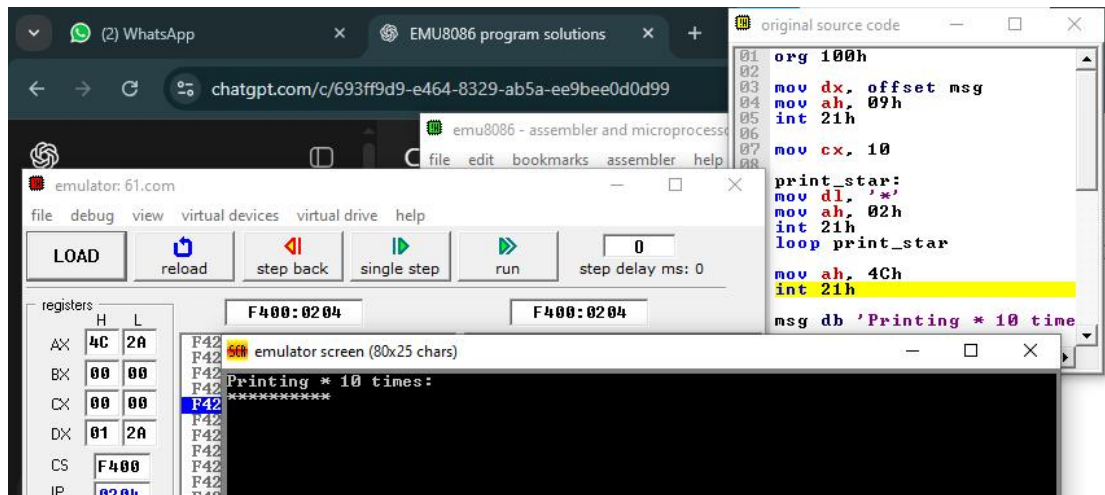
4. Program no 1 : PRINT '*' 10 TIMES



```

01 org 100h
02
03 mov dx, offset msg
04 mov ah, 09h
05 int 21h
06
07 mov cx, 10
08
09 print_star:
10     mov dl, '*'
11     mov ah, 02h
12     int 21h
13     loop print_star
14
15 mov ah, 4Ch
16 int 21h
17
18 msg db 'Printing * 10 times:', 0Dh, 0Ah, '$'
19
  
```

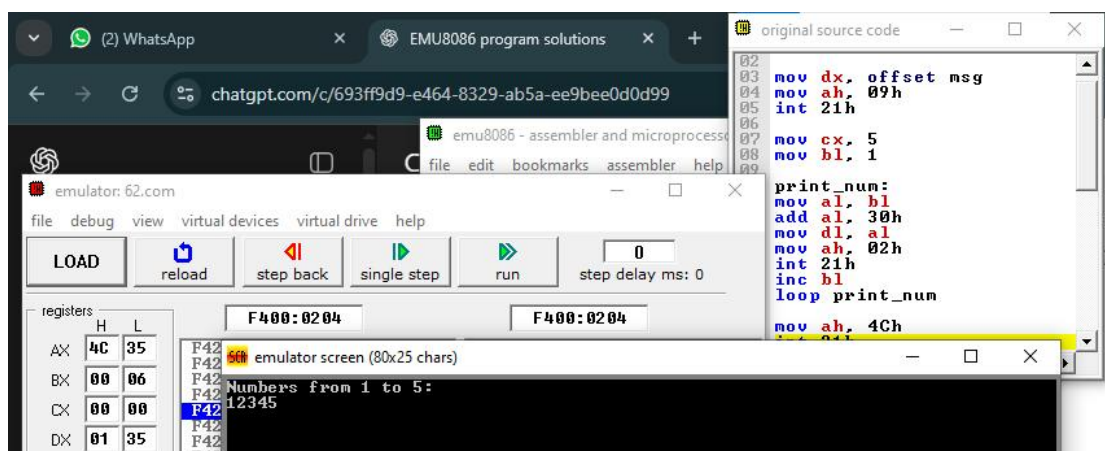
Output :



5. Program no 2 : PRINT NUMBERS 1 TO 5



Output :



LAB EXERCISE

Program no 1 : Print # 15 times using LOOP.

```

01 org 100h
02
03 mov dx, offset msg
04 mov ah, 09h
05 int 21h
06
07 mov cx, 15
08
09 again:
10 mov dl, '#'
11 mov ah, 02h
12 int 21h
13 loop again
14
15 mov ah, 4Ch
16 int 21h
17
18 msg db 'Printing # 15 times:', 0Dh, 0Ah, '$'
19

```

Output :

```

original source code
01 org 100h
02
03 mov dx, offset msg
04 mov ah, 09h
05 int 21h
06
07 mov cx, 15
08
09 again:
10 mov dl, '#'
11 mov ah, 02h
12 int 21h
13 loop again
14
15 mov ah, 4Ch
16 int 21h
17
18 msg db 'Printing # 15 time
19

```

emulator: 63.com

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers

	H	L
AX	4C	23
BX	00	00
CX	00	00
DX	01	23

F400:0204 F400:0204

emulator screen (80x25 chars)

```

Printing # 15 times:
#####

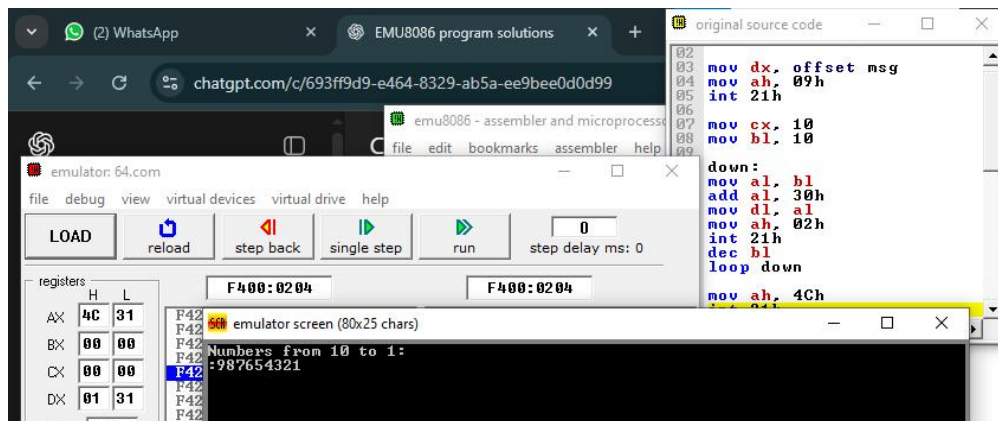
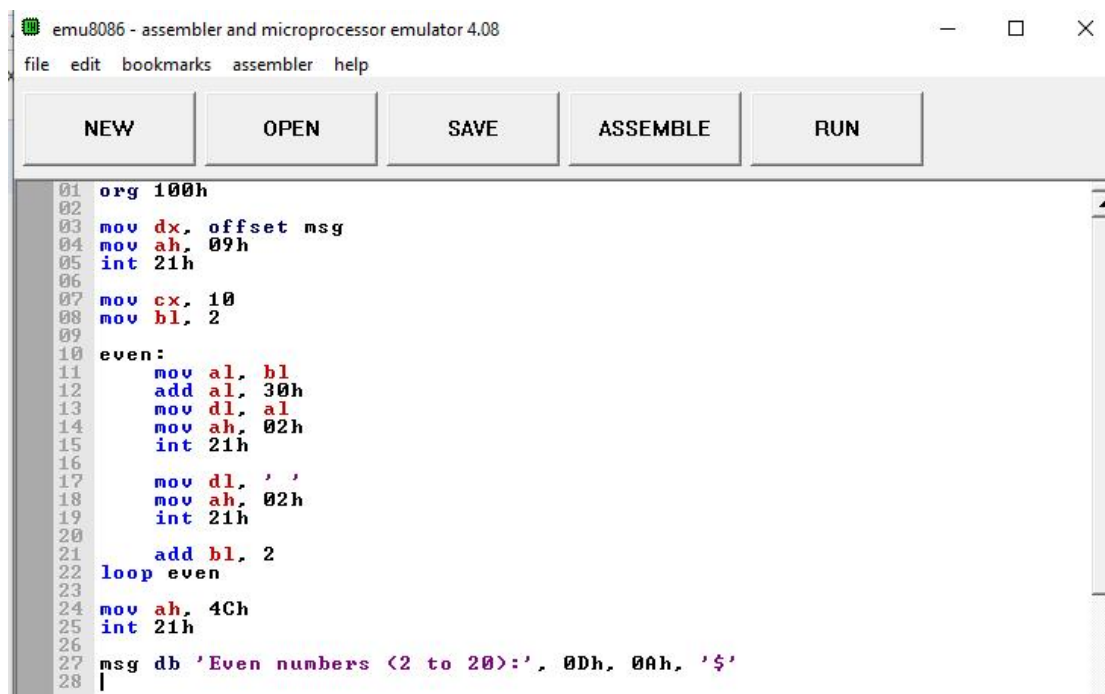
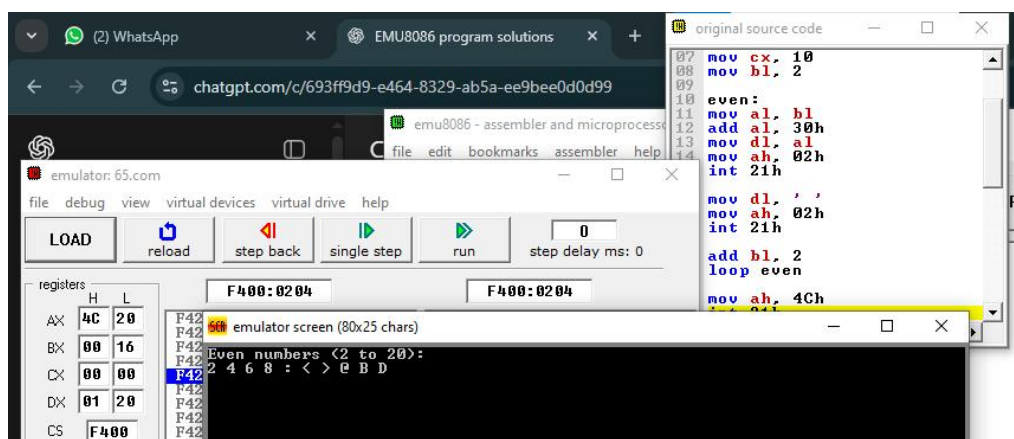
```

Program no 2 : Print numbers 10 to 1 in descending order.

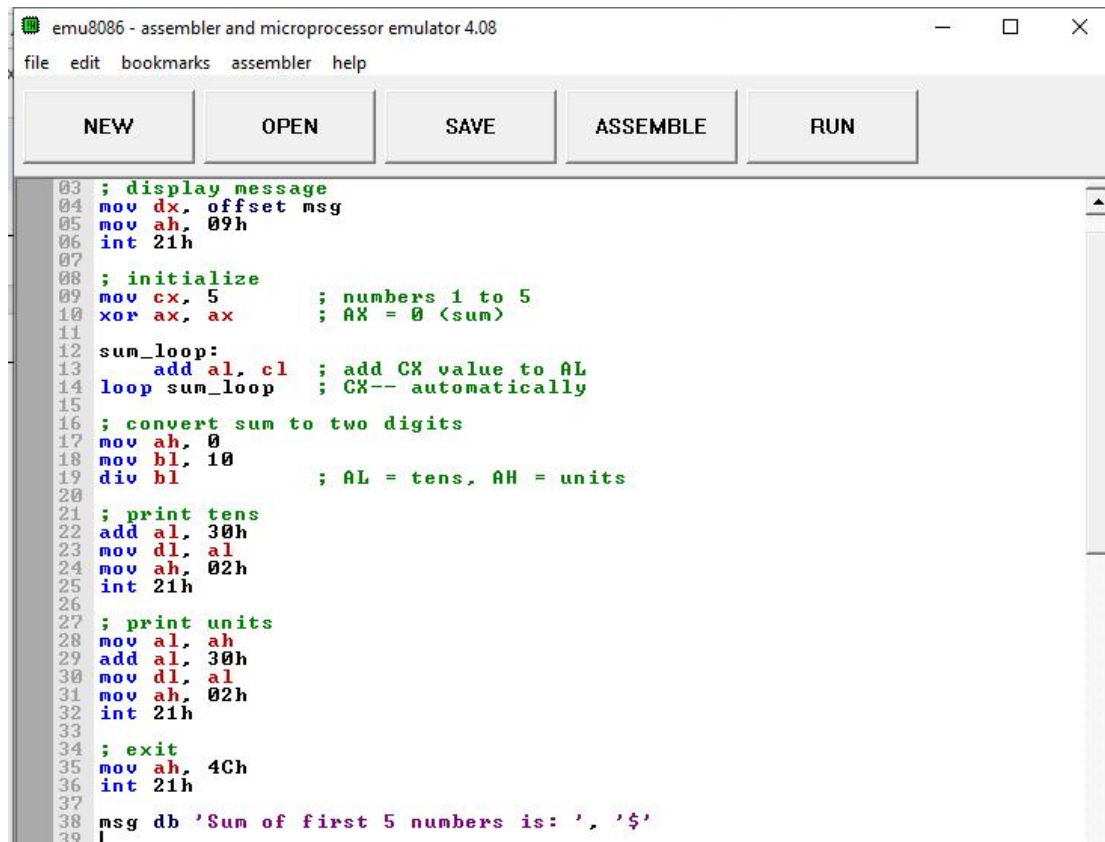
```

01 org 100h
02
03 mov dx, offset msg
04 mov ah, 09h
05 int 21h
06
07 mov cx, 10
08 mov bl, 10
09
10 down:
11 mov al, bl
12 add al, 30h
13 mov dl, al
14 mov ah, 02h
15 int 21h
16 dec bl
17 loop down
18
19 mov ah, 4Ch
20 int 21h
21
22 msg db 'Numbers from 10 to 1:', 0Dh, 0Ah, '$'
23

```

Output :**Program no 3 : Print even numbers from 2 to 20 using a loop.****Output :**

Program no 4 : Add the first 5 numbers (1+2+3+4+5) using a loop and display the sum.

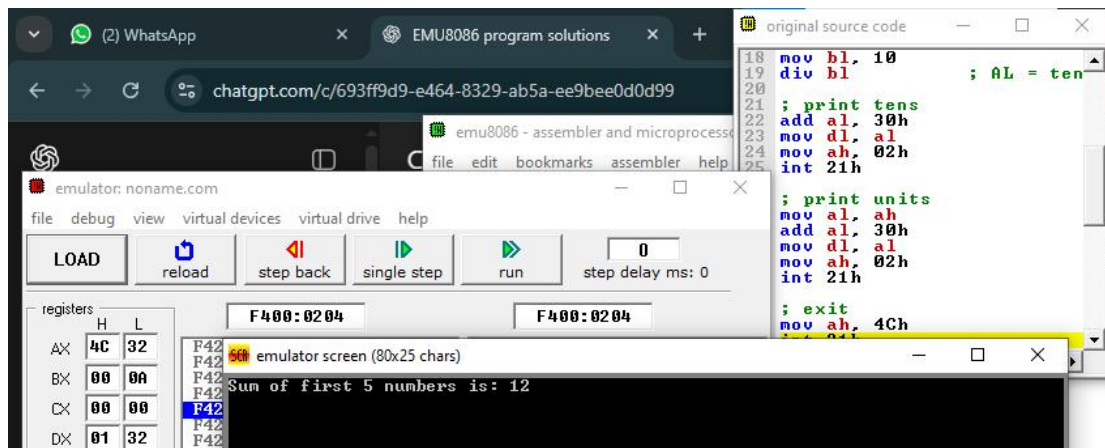


```

03 ; display message
04 mov dx, offset msg
05 mov ah, 09h
06 int 21h
07
08 ; initialize
09 mov cx, 5 ; numbers 1 to 5
10 xor ax, ax ; AX = 0 (sum)
11
12 sum_loop:
13 add al, cl ; add CX value to AL
14 loop sum_loop ; CX-- automatically
15
16 ; convert sum to two digits
17 mov ah, 0
18 mov bl, 10
19 div bl ; AL = tens, AH = units
20
21 ; print tens
22 add al, 30h
23 mov dl, al
24 mov ah, 02h
25 int 21h
26
27 ; print units
28 mov al, ah
29 add al, 30h
30 mov dl, al
31 mov ah, 02h
32 int 21h
33
34 ; exit
35 mov ah, 4Ch
36 int 21h
37
38 msg db 'Sum of first 5 numbers is: ', '$'
39

```

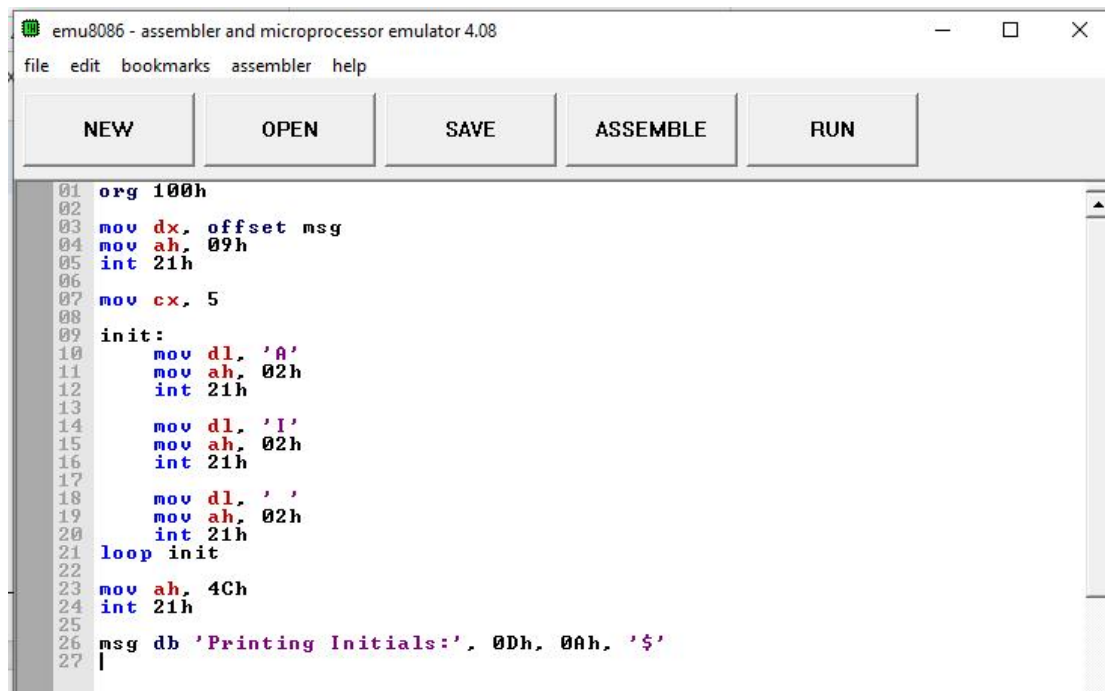
Output :



The screenshot shows the emu8086 emulator interface. The output window displays the text "Sum of first 5 numbers is: 12". The registers window shows the following values:

Register	H	L
AX	4C	32
BX	00	0A
CX	00	00
DX	01	32

Program no 5 : Print your initials multiple times using a loop.

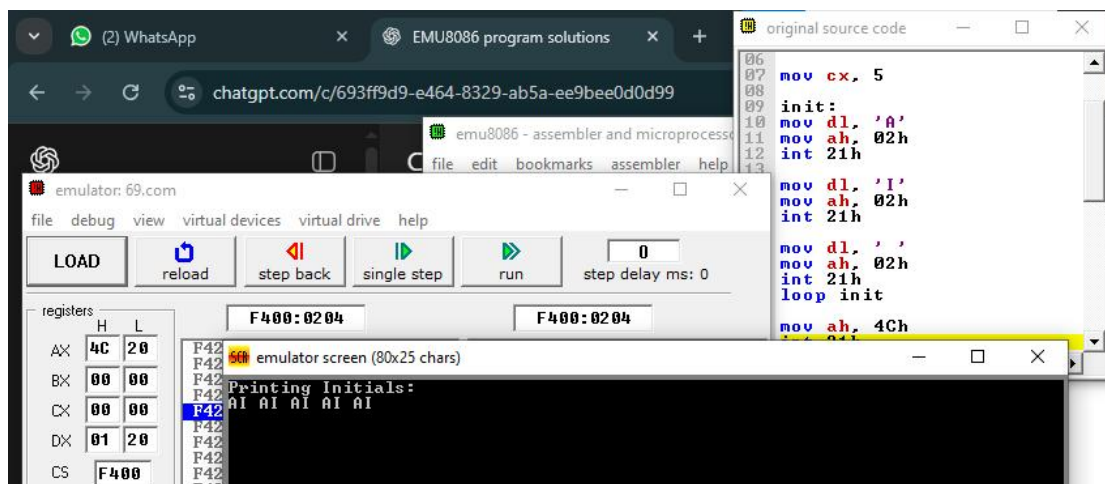


```

01 org 100h
02
03 mov dx, offset msg
04 mov ah, 09h
05 int 21h
06
07 mov cx, 5
08
09 init:
10 mov dl, 'A'
11 mov ah, 02h
12 int 21h
13
14 mov dl, 'I'
15 mov ah, 02h
16 int 21h
17
18 mov dl, ' '
19 mov ah, 02h
20 int 21h
21 loop init
22
23 mov ah, 4Ch
24 int 21h
25
26 msg db 'Printing Initials:', 0Dh, 0Ah, '$'
27

```

Output :



Registers:

	H	L
AX	4C	20
BX	00	00
CX	00	00
DX	01	20
CS	F400	

Output window:

```

Printing Initials:
A I A I A I A I

```

LAB NO 10

JUMPS

In assembly language, jumps are used to control the flow of execution of a program. Instead of executing instructions strictly in sequence, jump instructions allow the program to move execution to a labeled location based on a condition or unconditionally.

Conditional Jumps

Conditional jumps depend on the result of a comparison. The CMP instruction compares two values and sets flags in the flag register. Based on these flags, conditional jumps decide the next execution path.

Common conditional jumps include:

- JE (Jump if Equal)
- JNE (Jump if Not Equal)
- JG (Jump if Greater)
- JL (Jump if Less)
- JBE (Jump if Below or Equal)

These jumps are mainly used in:

- Decision making
- Comparisons
- Loop conditions

Unconditional Jump

The JMP instruction is an unconditional jump, meaning it always transfers control to the specified label without checking any condition. It is commonly used to:

- Create loops
- Skip instructions
- Exit program blocks

Role of Labels

Labels act as reference points in the program. Jump instructions transfer control to these labels, making the program structured and readable.

Importance of Jumps

Jumps are essential for:

- Implementing if-else logic
- Creating loops

- Handling program branching
- Controlling execution flow efficiently

Lab Exercise

Program no 1 : Comparison and Conditions

Aim:

Compare two numbers and display which one is greater, smaller, or equal.

Theory:

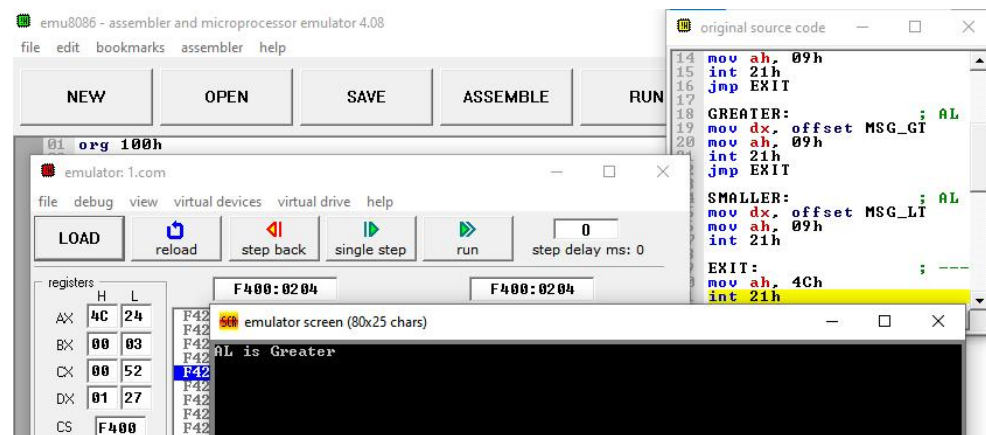
- In assembly, the CMP instruction is used to compare two registers or values.
- Based on the comparison, **conditional jumps** like JG, JL, or JE decide the program flow.
- Labels mark the positions where control will jump after a condition is met.

```

01 org 100h
02
03 START:           ; ---- Program Start ----
04
05     mov al, 5      ; first number
06     mov bl, 3      ; second number
07
08     cmp al, bl     ; compare AL and BL
09     jg GREATER     ; AL > BL
10     jl SMALLER     ; AL < BL
11
12     EQUAL:         ; AL = BL
13     mov dx, offset MSG_EQ
14     mov ah, 09h
15     int 21h
16     jmp EXIT
17
18     GREATER:        ; AL > BL
19     mov dx, offset MSG_GT
20     mov ah, 09h
21     int 21h
22     jmp EXIT
23
24     SMALLER:        ; AL < BL
25     mov dx, offset MSG_LT
26     mov ah, 09h
27     int 21h
28
29     EXIT:           ; ---- Program End ----
30     mov ah, 4Ch
31     int 21h
32
33     MSG_GT db 'AL is Greater$'
34     MSG_LT db 'AL is Smaller$'
35     MSG_EQ db 'Both are Equal$'
36

```

Output :



Label Explanation:

- START: → Program begins.
- GREATER: → Executes when $AL > BL$.
- SMALLER: → Executes when $AL < BL$.
- EQUAL: → Executes when $AL = BL$.
- EXIT: → Ends the program.

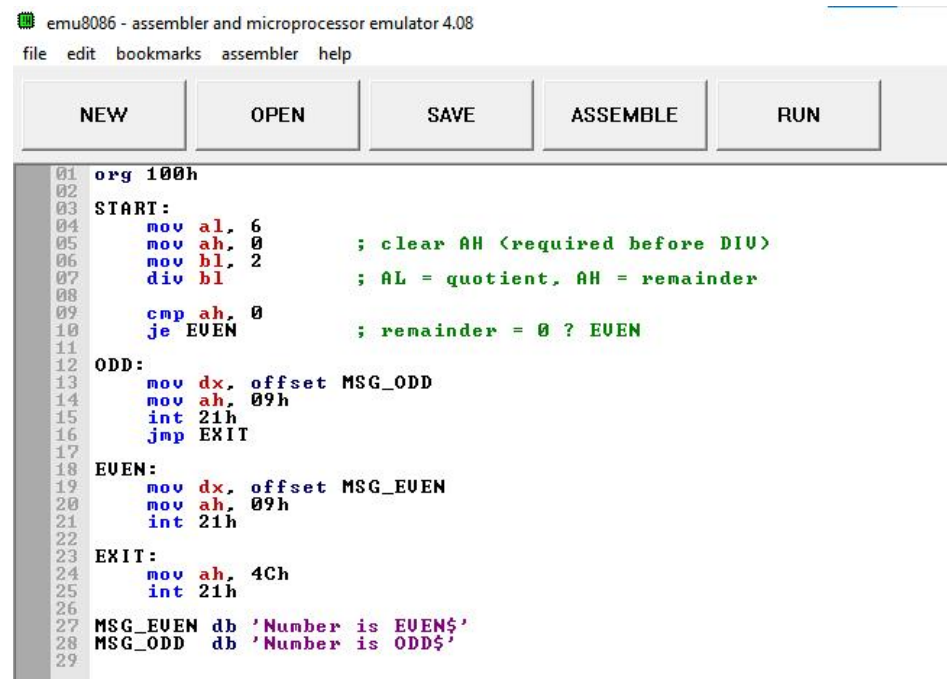
Program no 2 : Conditional Jumps

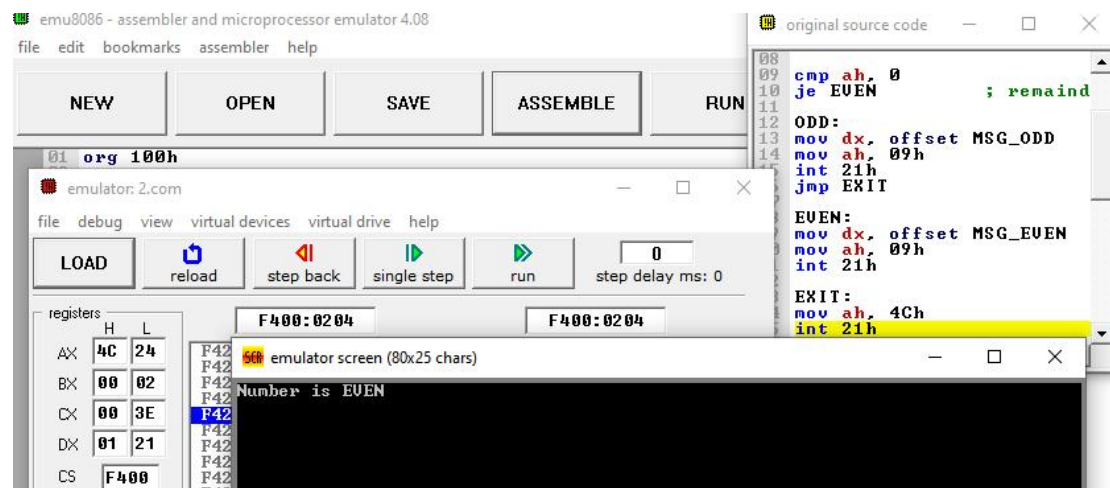
Aim:

Check if number is even or odd using JE and JMP.

Theory:

- DIV divides AL by a number and stores remainder in AH.
- If remainder = 0 → even number, else → odd.
- JE (Jump if Equal) is used to branch to the even block when the remainder is zero



Output :**Label Explanation:**

- START: → Program begins.
- EVEN: → Executes if remainder = 0 (JE condition).
- ODD: → Executes if remainder $\neq$ 0.
- EXIT: → Ends the program.

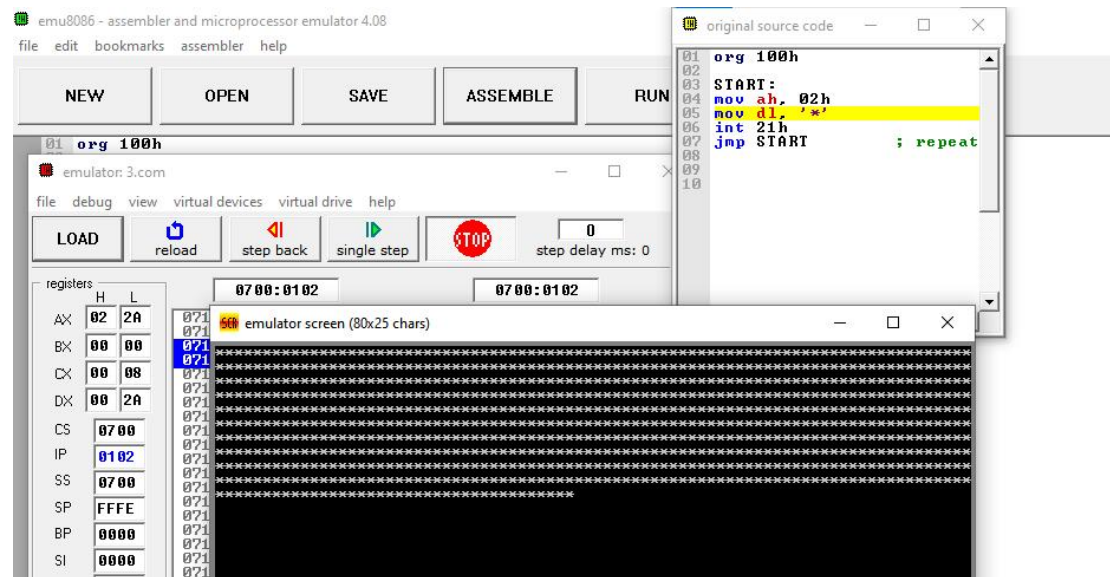
Program no 3 : Unconditional Jump**Aim:**

Display * continuously using JMP (no condition).

Theory:

- JMP is an **unconditional jump** — it always transfers control to the target label.
- This causes the program to repeat endlessly, creating an infinite loop.

**Output :**



Label Explanation:

- START: → The loop start point.
- JMP START → Goes back to the same point repeatedly

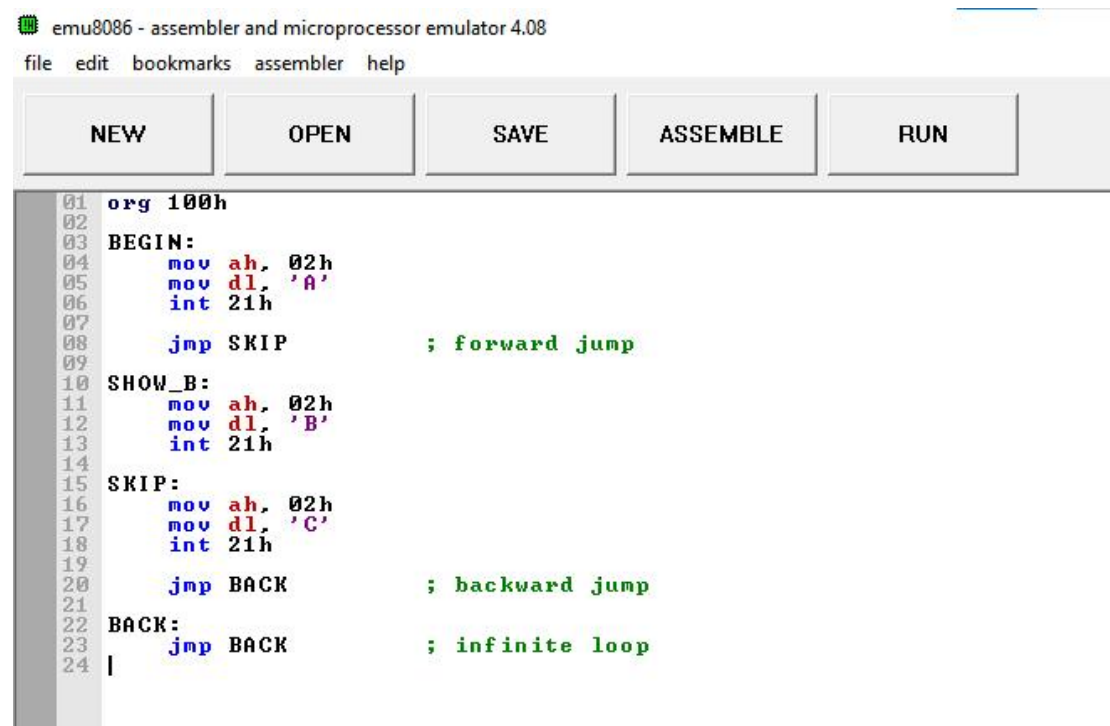
Program no 4 : Relative Addressing

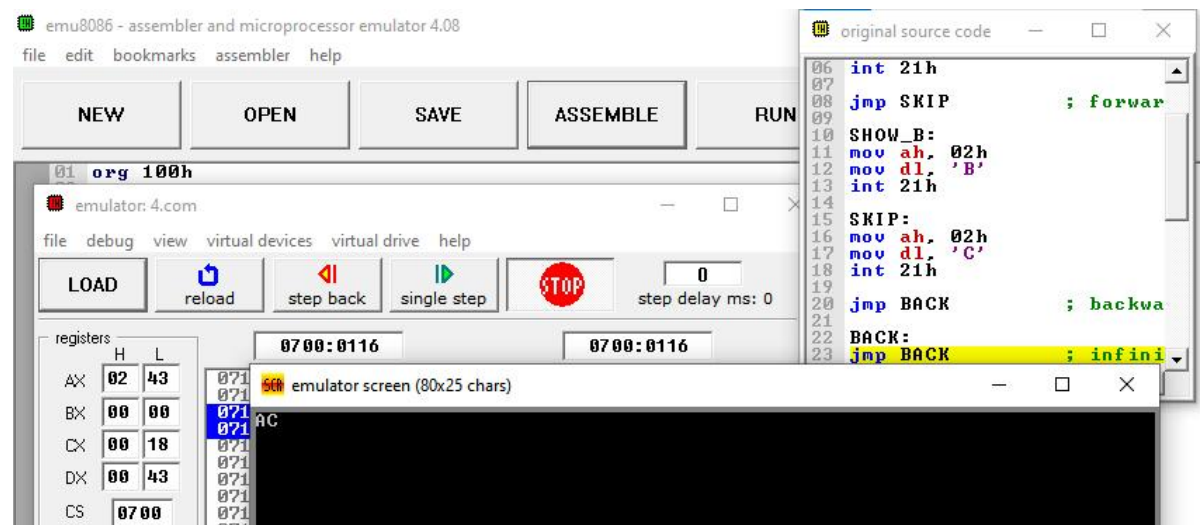
Aim:

Demonstrate short forward and backward jumps.

Theory:

- Relative addressing means the jump target is **nearby** (within ± 128 bytes).
- These jumps move control forward or backward relative to current location.



Output :**Label Explanation:**

- BEGIN: → Program start.
- SKIP: → Target for forward jump.
- BACK: → Backward jump (loop demonstration).
- SHOW_B: → This part is skipped due to jump.

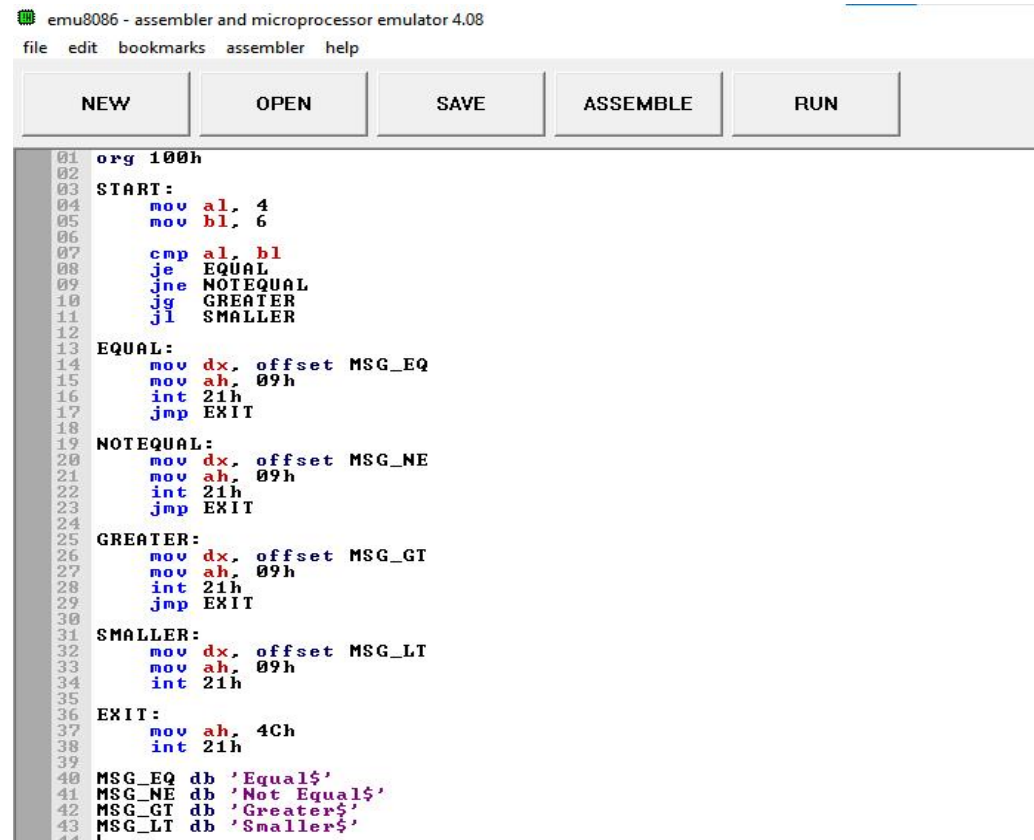
Program no 5 : Types of Jumps**Aim:**

Show multiple jump types (JE, JNE, JG, JL).

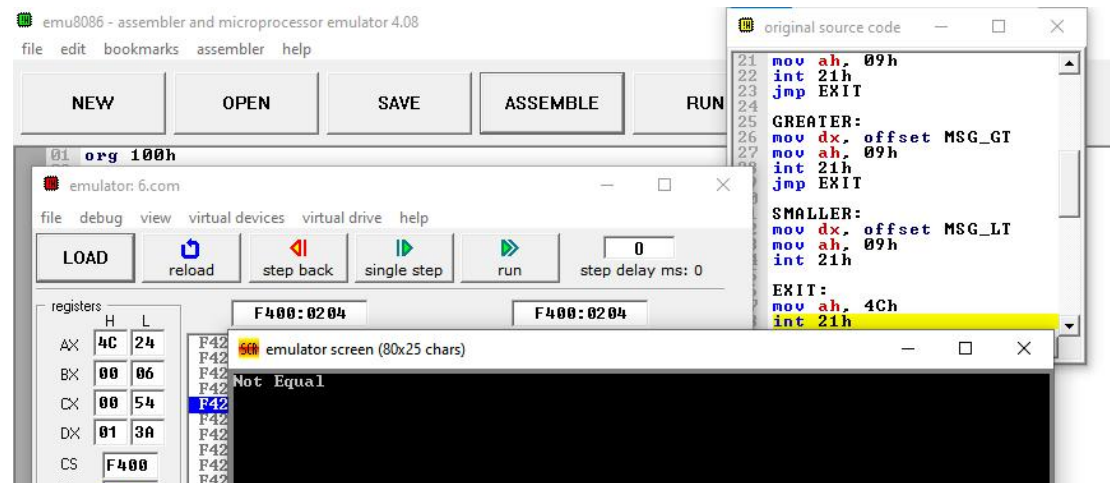
Theory:

Different jumps are used for conditions:

- JE → Jump if equal
- JNE → Jump if not equal
- JG → Jump if greater
- JL → Jump if less



Output :



Label Explanation:

- EQUAL: → Executes if $AL = BL$.
- NOTEQUAL: → Executes if $AL \neq BL$.
- GREATER: → Executes if $AL > BL$.
- SMALLER: → Executes if $AL < BL$.
- EXIT: → Ends program.

LAB NO 11

BUBBLE SORT

Bubble Sort is a simple sorting algorithm used to arrange data in ascending or descending order. In assembly language, bubble sort is implemented using loops, comparisons, and jump instructions.

Working Principle

Bubble sort works by:

1. Repeatedly comparing adjacent elements in an array.
2. Swapping them if they are in the wrong order.
3. Continuing this process until the array becomes sorted.

With each pass, the largest value moves (bubbles) to the end of the list.

Implementation in Assembly Language

In assembly language:

- Data is stored in memory locations
- Registers are used to load and compare values
- CMP is used for comparison
- Conditional jumps like JBE or JG decide whether a swap is needed
- Loops are controlled using LOOP, JNZ, or JMP

Use of Loops

Bubble sort uses:

- **Outer loop** → controls number of passes
- **Inner loop** → compares and swaps adjacent elements

Advantages

- Simple and easy to understand
- Useful for learning loops and jumps in assembly

Limitations

- Inefficient for large datasets
- High number of comparisons and swaps

Lab Exercise

Program no 1 : Sorting Example (Bubble Sort — 3 Numbers)

Aim:

Sort 3 numbers in ascending order using loops and jumps.

Theory:

- Bubble sort repeatedly compares adjacent values and swaps them if needed.
- Here, CMP, JBE, and LOOP are used to control iterations.

emu8086 - assembler and microprocessor emulator 4.08

file edit bookmarks assembler help

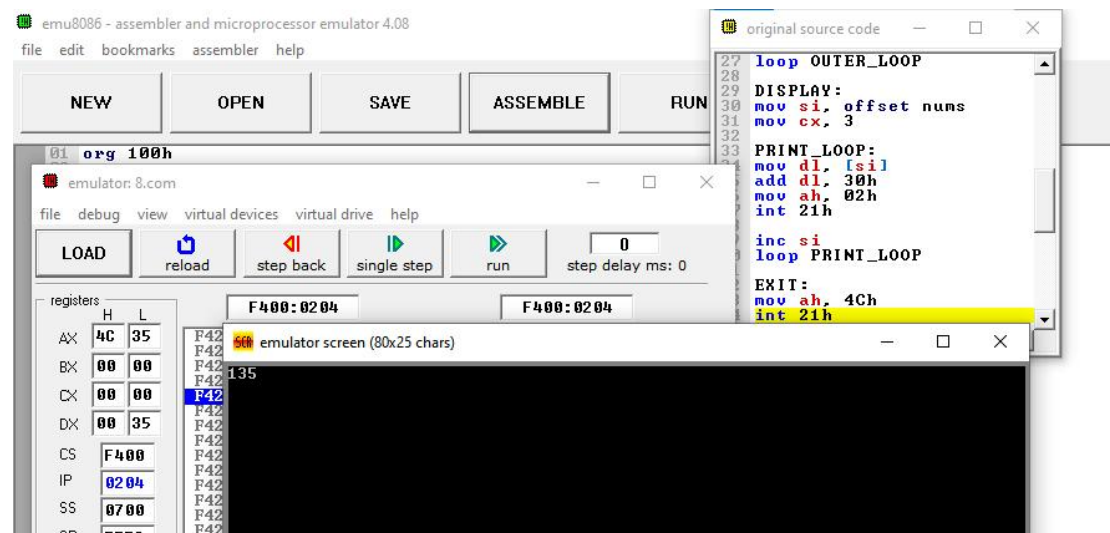
NEW	OPEN	SAVE	ASSEMBLE	RUN
-----	------	------	----------	-----

```

01 org 100h
02
03 DATA_START:
04     nums db 5, 3, 1        ; unsorted numbers
05
06 START:
07     mov cx, 2              ; number of passes
08
09 OUTER_LOOP:
10     mov si, offset nums
11     mov bx, 2              ; 2 comparisons per pass
12
13 INNER_LOOP:
14     mov al, [si]
15     mov dl, [si+1]
16     cmp al, dl
17     jbe NO_SWAP
18
19     mov [si], dl           ; SWAP
20     mov [si+1], al
21
22 NO_SWAP:
23     inc si
24     dec bx
25     jnz INNER_LOOP
26
27     loop OUTER_LOOP
28
29 DISPLAY:
30     mov si, offset nums
31     mov cx, 3
32
33 PRINT_LOOP:
34     mov dl, [si]
35     add dl, 30h
36     mov ah, 02h
37     int 21h
38
39     inc si
40     loop PRINT_LOOP
41
42 EXIT:
43     mov ah, 4Ch
44     int 21h
45

```

Output :



Label Explanation:

- DATA_START: → Start of data (array).
- OUTER_LOOP: → Outer loop for number of passes.
- INNER_LOOP: → Compares and swaps numbers.
- NO_SWAP: → Skip if no swap needed.
- DISPLAY: → Prints sorted numbers.
- EXIT: → Program end.

LAB NO 12

BIT SHIFTING IN 8086 ASSEMBLY

Shift Instructions

SHL (Shift Left)

1. Moves bits to the left
2. Fills rightmost bit with 0
3. Leftmost bit goes into CF
4. Same as multiply by 2 each shift

SHR (Shift Right)

1. Moves bits to the right
2. Fills leftmost bit with 0
3. Rightmost bit goes into CF
4. Same as divide by 2 each shift

Shift Left (SHL)

```

01 org 100h
02 mov al, 05h
03 ; 00000101b
04 shl al, 1
05 ; becomes 00001010b <= 0Ah> mov ah, 4Ch
06 int 21h |
  
```

Output :

```

01 org 100h
02 mov al, 05h
03 ; 00000101b
04 shl al, 1
05 ; becomes 00001010b <= 0Ah> mov ah, 4Ch
06 int 21h |
  
```

Shift Right (SHR)

```

01 org 100h
02 mov bl, 80h
03 ; 10000000b
04 shr bl, 1
05 ; 01000000b (= 40h)
06 mov ah, 4Ch
07 int 21h

```

Output :

Registers:

Register	H	L
AX	4C	00
BX	00	40
CX	00	08
DX	00	00
CS	F400	
IP	0204	

Memory Dump (F400:0204):

Address	Value	Comment
F4200:	FF 255	RES
F4201:	FF 255	RES
F4202:	CD 205	=
F4203:	21 033	!
F4204:	CF 207	!
F4205:	00 000	NULL
F4206:	00 000	NULL
F4207:	00 000	NULL
F4208:	00 000	NULL
F4209:	00 000	NULL
F420A:	00 000	NULL

BIOS DI: INT 021h

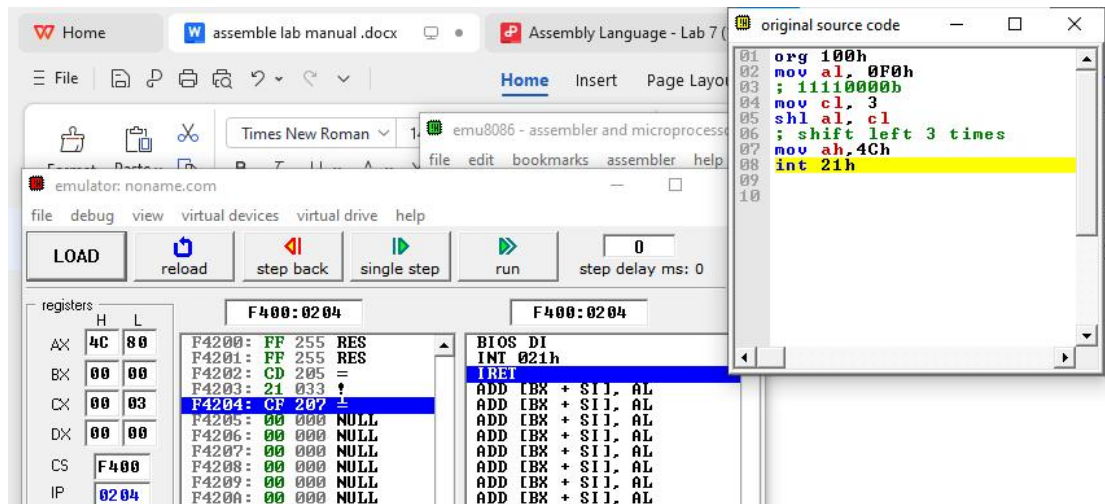
Multi- Bit Shift Using CL

```

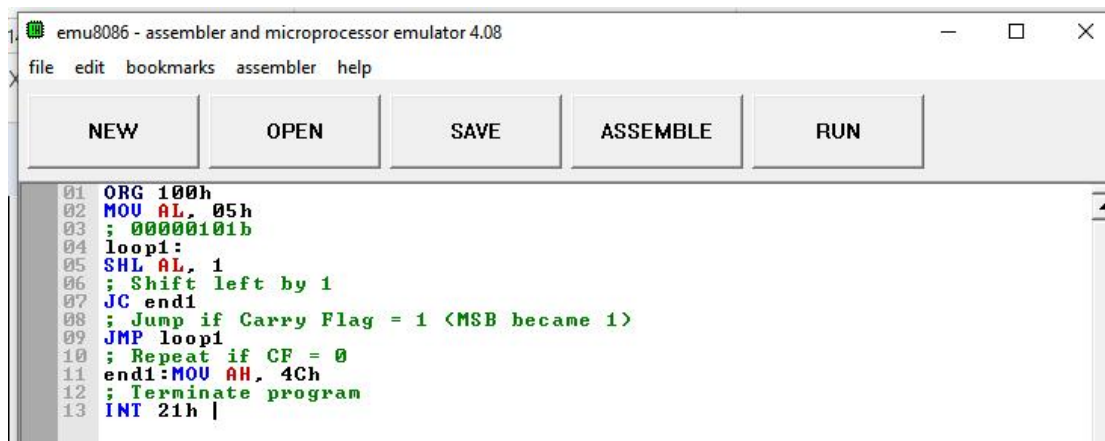
01 org 100h
02 mov al, 0F0h
03 ; 11110000b
04 mov cl, 3
05 shl al, cl
06 ; shift left 3 times
07 mov ah, 4Ch
08 int 21h

```

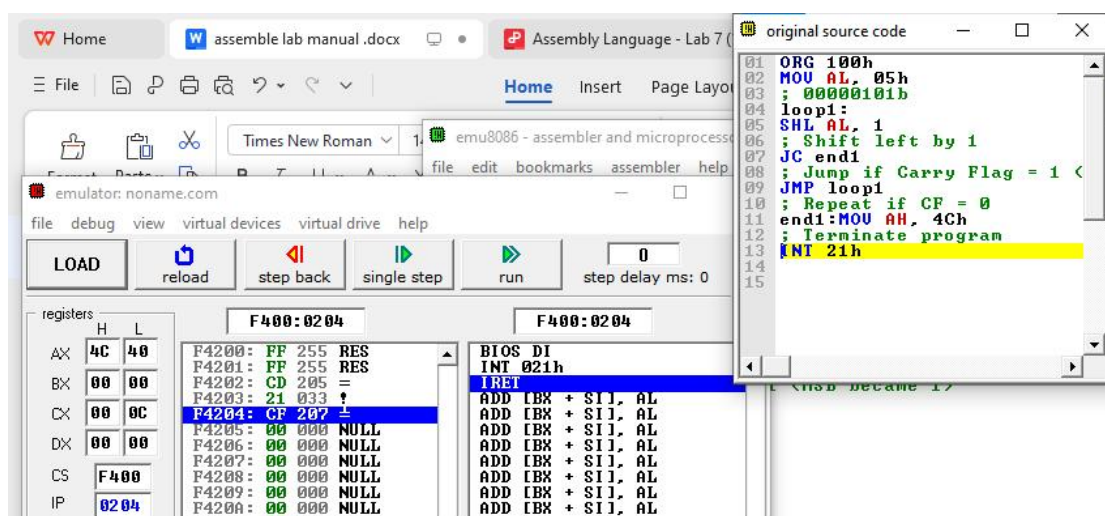
Output :



Shift Left Until Carry (CF) and Jump



Output :



Shift Right Until Zero and Jump

```

01 ORG 100h
02 MOV AL, 10h
03 ; 00010000b
04 loop2:
05 SHR AL, 1
06 ; Shift right by 1
07 JZ end2
08 ; Jump if AL = 0
09 JMP loop2
10 ; Repeat if AL != 0
11 end2:
12 MOV AH, 4Ch
13 ; Terminate program
14 INT 21h

```

Output :

emulator: noname.com

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers

	H	L
AX	4C	00
BX	00	00
CX	00	0C
DX	00	00

F400:0204

F4200: FF 255 RES
F4201: FF 255 RES
F4202: CD 205 =
F4203: 21 033 ?
F4204: CF 207 ±
F4205: 00 000 NULL
F4206: 00 000 NULL
F4207: 00 000 NULL

BIOS DI
INT 021h
IRET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL

Lab Exercise

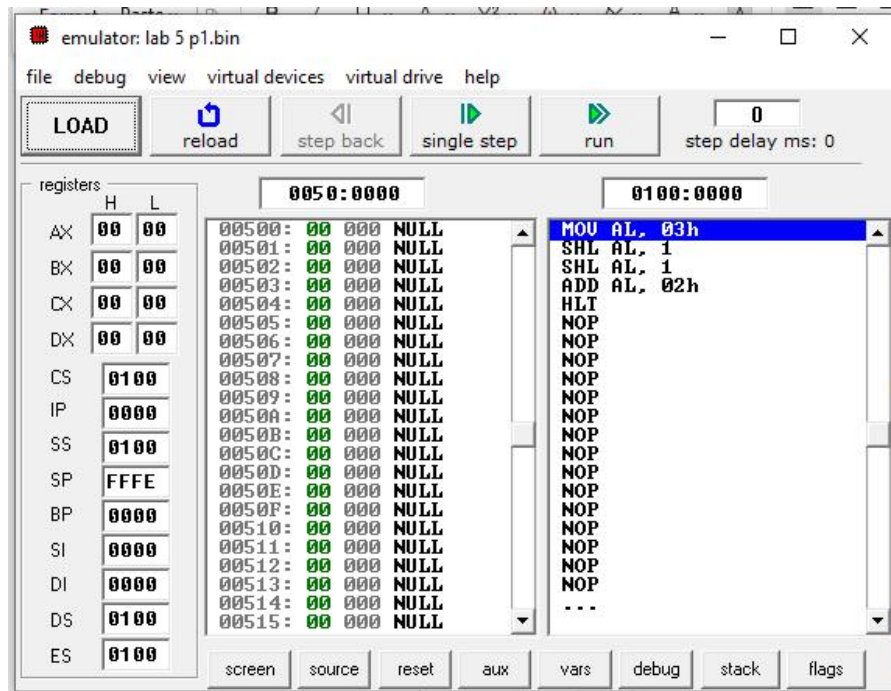
Program no 1 : Multiply 03h by 4 using SHL, then add 02h to the result and store it in AL.

```

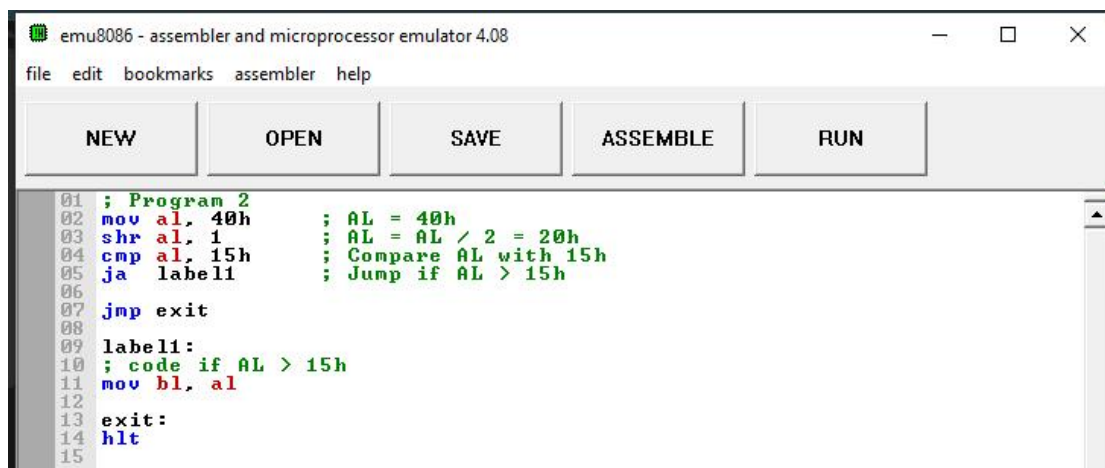
01 ; Program 1
02 mov al, 03h ; AL = 03h
03 shl al, 1 ; AL = AL * 2 (06h)
04 shl al, 1 ; AL = AL * 4 (0Ch)
05 add al, 02h ; AL = 0Ch + 02h = 0Eh
06 hlt
07

```

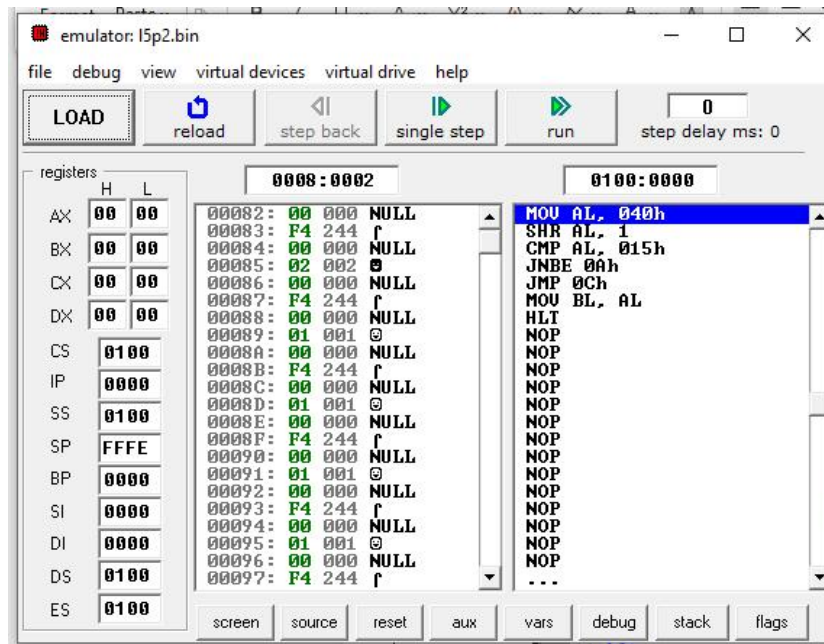
Output :



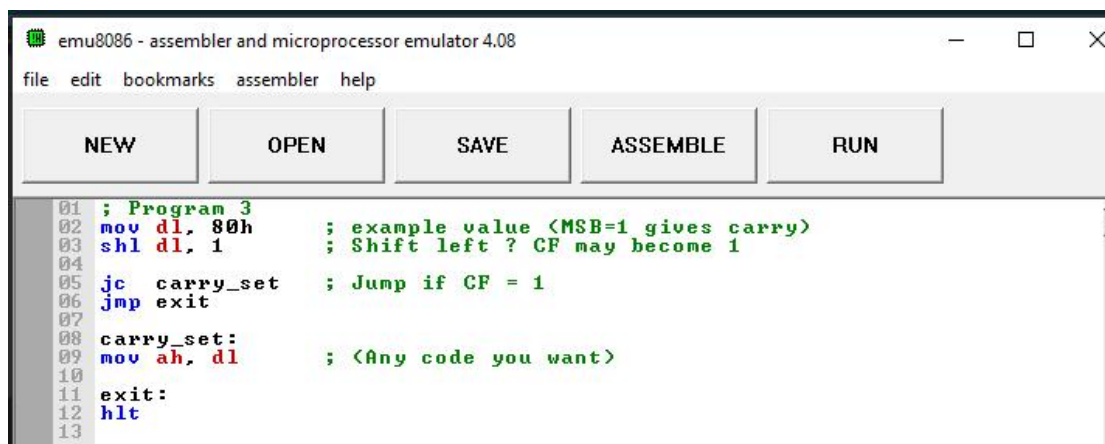
Program no 2 : Divide 40h by 2 using SHR, then compare the result with 15h and jump to label1 if it is greater.



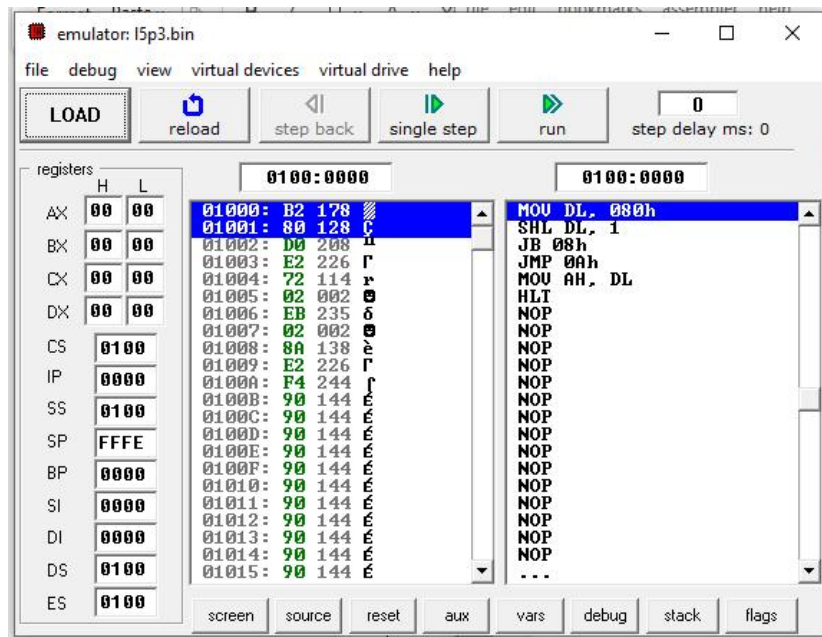
Output :



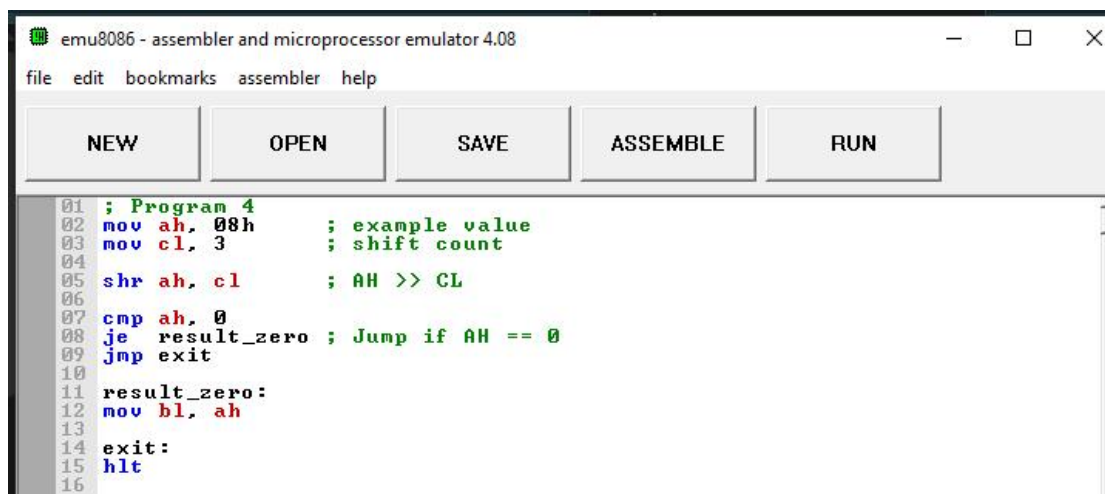
Program no 3 :Shift DL left by 1 and jump to a label if the Carry Flag (CF) is set.



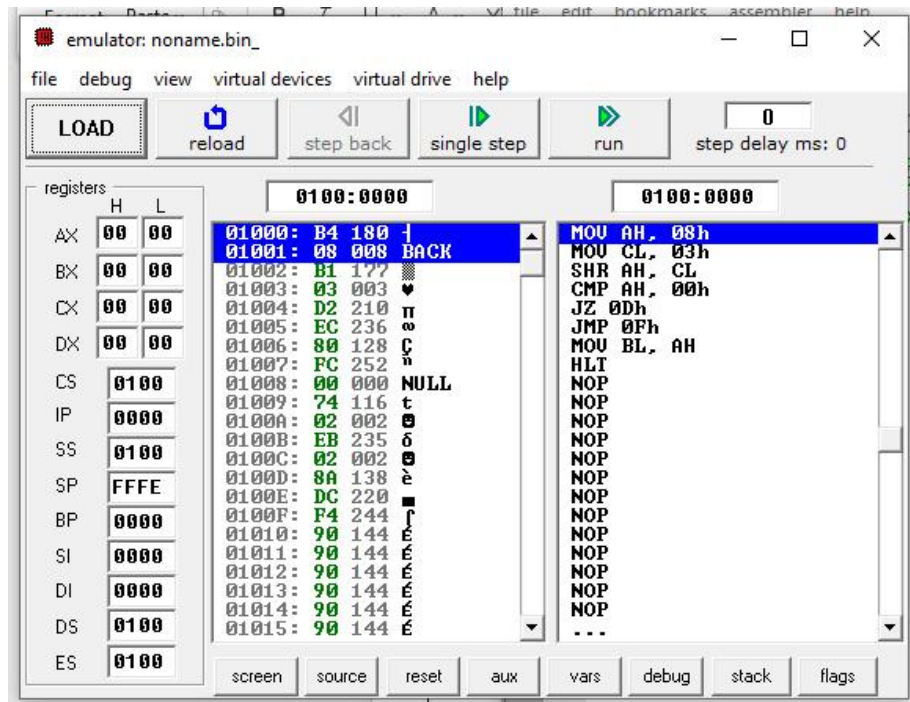
Output :



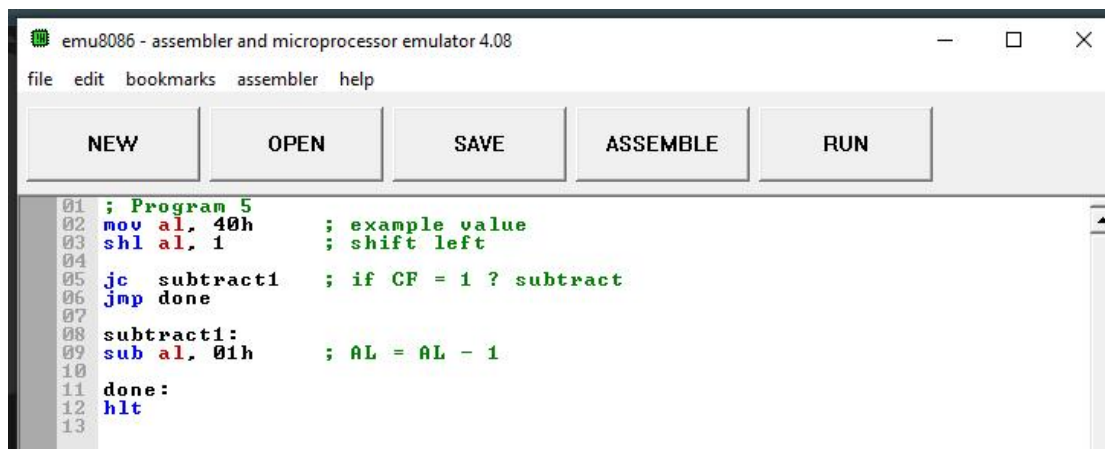
Program no 4 : Shift AH right by the number in CL using SHR and jump to a label if the result is zero.



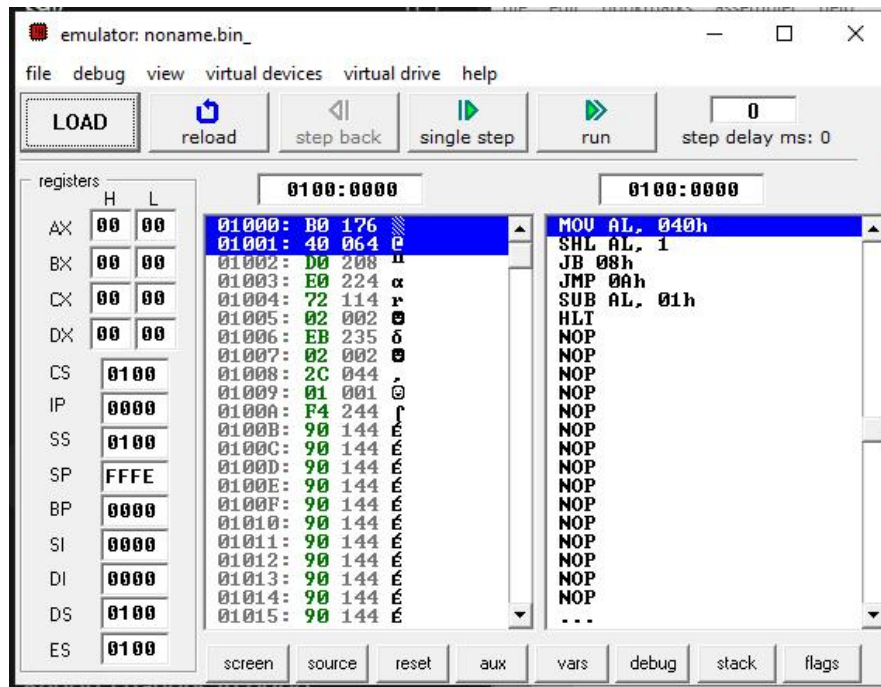
Output :



Program no 5 : Shift AL left by 1. If CF = 1, subtract 01h from AL and store the result; otherwise leave AL unchanged.



Output :



LAB NO 13

ROTATION IN BIT MANIPULATION

ROTATE OPERATIONS

Rotate operations move bits in a **circular manner**, unlike shift operations where bits are pushed out. In rotation, **no bit is lost**; instead, bits leaving one end of the register re-enter from the opposite end.

You can imagine **ROTATE as turning a wheel**, where bits continuously move around in a loop.

Just like shift operations, rotate instructions also update the **Carry Flag (CF)**, which stores the last rotated bit.

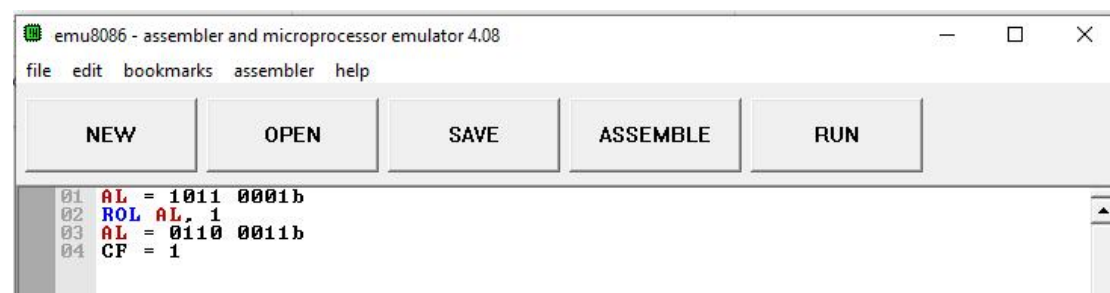
TYPES OF ROTATE INSTRUCTIONS

1) ROL – Rotate Left

The **ROL (Rotate Left)** instruction shifts all bits to the left in a circular way.

- The **most significant bit (MSB)** moves to the **least significant bit (LSB)** position.
- The old MSB is copied into the **Carry Flag (CF)**.
- No bit is discarded.

Example:



```

01 AL = 1011 0001b
02 ROL AL, 1
03 AL = 0110 0011b
04 CF = 1
  
```

Here, the MSB was 1, so it appears both in the CF and at the LSB position after rotation.

Usage:

ROL is commonly used when you need to examine or preserve the MSB during repeated rotations.

2) ROR – Rotate Right

The **ROR (Rotate Right)** instruction rotates bits to the right in a circular fashion.

- The **least significant bit (LSB)** moves to the **most significant bit (MSB)**.
- The old LSB is stored in the **Carry Flag (CF)**.
- All bits remain within the register.

Example:

The screenshot shows the emu8086 interface with the following assembly code and register state:

```

01 AL = 1011 0001b
02 ROR AL, 1
03 AL = 1101 1000b
04 CF = 1

```

Usage:

ROR is useful when checking or manipulating the LSB repeatedly.

3) RCL – Rotate Left Through Carry

The **RCL (Rotate Left through Carry)** instruction is an extended form of rotate left.

- Rotation includes both the register and the **Carry Flag**.
- The Carry Flag acts like an **extra 9th bit**.
- Bit flow is:
 $CF \rightarrow LSB \rightarrow \dots \rightarrow MSB \rightarrow CF$

This instruction is very helpful when working with **multi-byte numbers**, where bits must move across registers.

Example:

The screenshot shows the emu8086 interface with the following assembly code and register state:

```

01 AL = 1011 0001b
02 CF = 1
03 RCL AL, 1
04 AL = 0110 0011b
05 CF = 1

```

The old MSB moves into the Carry Flag, and the old CF moves into the LSB.

4) RCR – Rotate Right Through Carry

The **RCR (Rotate Right through Carry)** instruction rotates bits to the right while also including the Carry Flag in the rotation.

- The Carry Flag again acts as an extra bit.

- Bit flow is:
CF → MSB → ... → LSB → CF

Example:

```

01  AL = 1011 0001b
02  CF = 1
03  RCR AL, 1
04  AL = 1101 1000b
05  CF = 1

```

Usage:

RCR is especially useful for **right-side multi-byte rotations** or when the carry flag is treated as part of the data.

Example no 1 :ROL

```

01  org 100h
02
03  mov al, 10110001b      ; AL = 1011 0001
04  rol al, 1              ; Rotate left by 1
05                          ; AL = 0110 0011
06                          ; CF = 1 <old MSB>
07
08  mov ah, 4Ch             ; Terminate program
09  int 21h
10

```

Output :

The screenshot shows the emu8086 emulator interface. The registers window displays the following values:

Register	Value
AX	4C 63
BX	00 00
CX	00 08
DX	00 00
CS	F400
IP	0204
SS	0700
SP	FFF8
BP	0000
SI	0000
DI	0000
DS	0700
ES	0700

The memory window shows the following values:

Address	Value
F4200	FF 255 RES
F4201	FF 255 RES
F4202	CD 205 =
F4203	21 033 ↑
F4204	CF 207 ↑
F4205	00 000 NULL
F4206	00 000 NULL
F4207	00 000 NULL
F4208	00 000 NULL
F4209	00 000 NULL
F420A	00 000 NULL
F420B	00 000 NULL
F420C	00 000 NULL
F420D	00 000 NULL
F420E	00 000 NULL
F420F	00 000 NULL
F4210	00 000 NULL
F4211	00 000 NULL
F4212	00 000 NULL
F4213	00 000 NULL
F4214	00 000 NULL
F4215	00 000 NULL

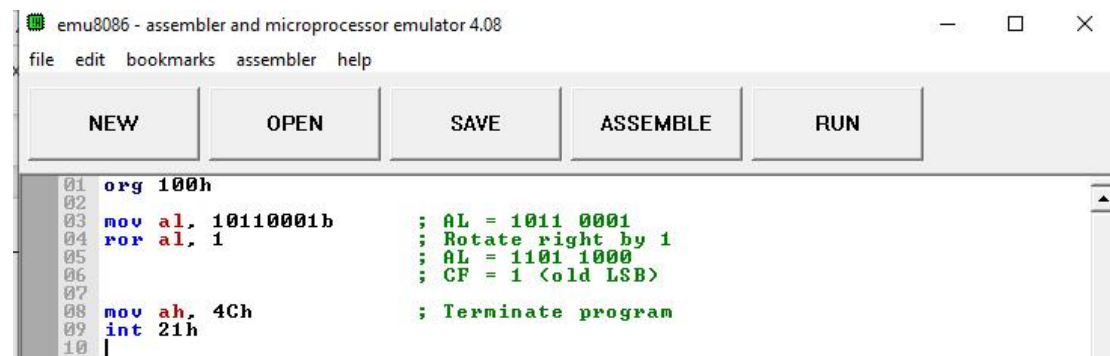
The source code window shows the following code:

```

01  org 100h
02
03  mov al, 10110001b      ; A
04  rol al, 1              ; R
05                          ; AL = 0110 0011
06                          ; CF = 1 <old MSB>
07
08  mov ah, 4Ch             ; T
09  int 21h
10

```

Example no 2 : ROR

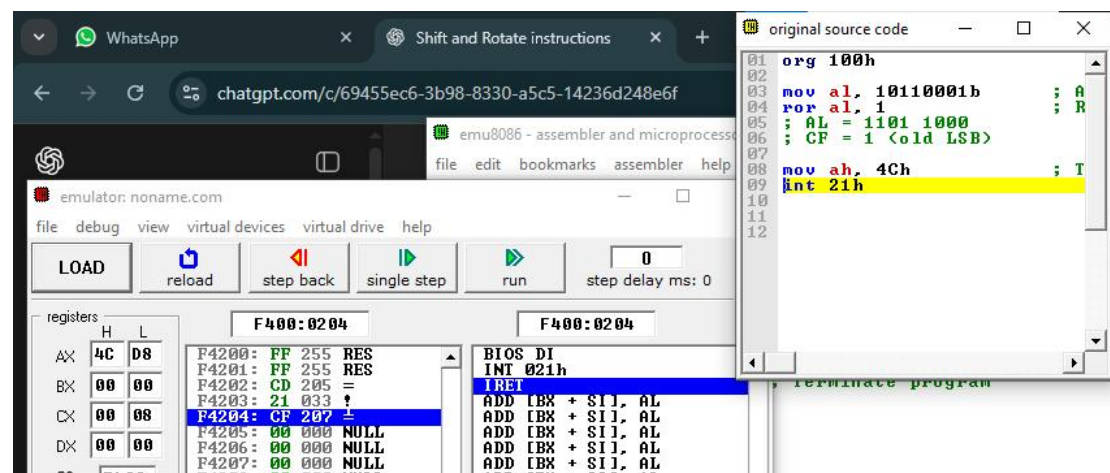


```

01 org 100h
02
03 mov al, 10110001h    ; AL = 1011 0001
04 ror al, 1            ; Rotate right by 1
05                     ; AL = 1101 1000
06                     ; CF = 1 (old LSB)
07
08 mov ah, 4Ch          ; Terminate program
09 int 21h
10

```

Output :



original source code

```

01 org 100h
02
03 mov al, 10110001h    ; A
04 ror al, 1            ; R
05                     ; AL = 1101 1000
06                     ; CF = 1 (old LSB)
07
08 mov ah, 4Ch          ; T
09 int 21h
10
11
12

```

emulator: noname.com

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

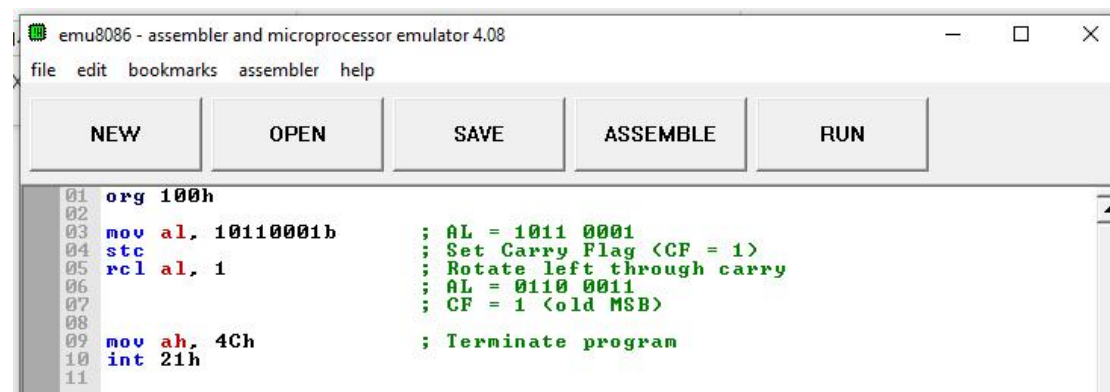
registers	H	L
AX	4C	08
BX	00	00
CX	00	08
DX	00	00
CS	F000	0000

F400:0204 F400:0204

F4200: FF 255 RES
F4201: FF 255 RES
F4202: CD 205 =
F4203: 21 033
F4204: CF 207
F4205: 00 000 NULL
F4206: 00 000 NULL
F4207: 00 000 NULL
F4208: 00 000 NULL

BIOS DI
INT 021h
IRET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL

Example no 3 : RCL

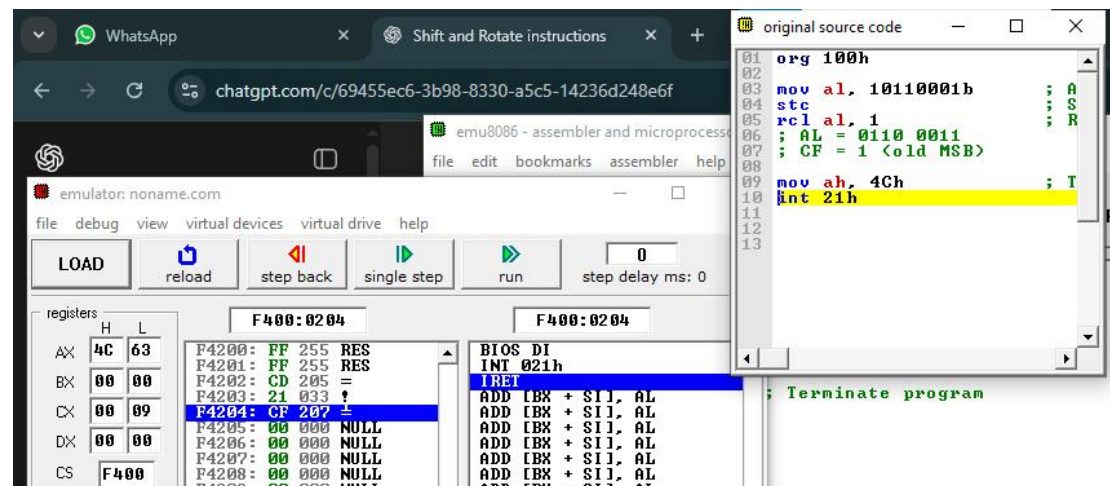


```

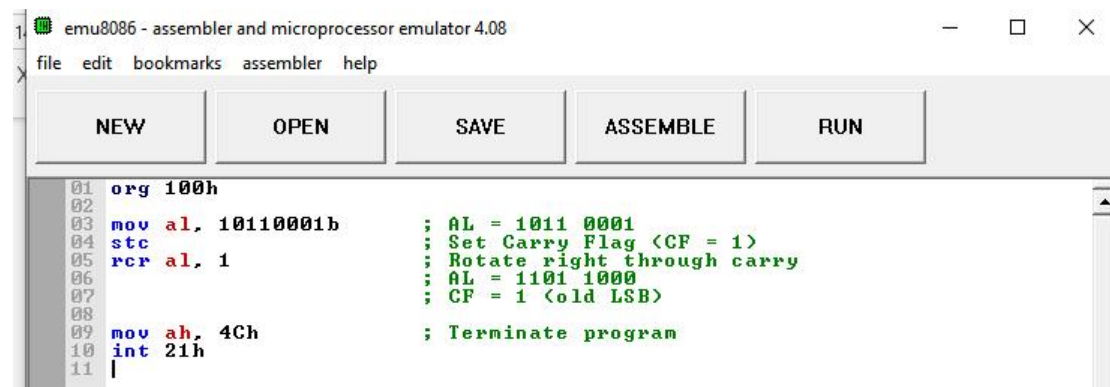
01 org 100h
02
03 mov al, 10110001h    ; AL = 1011 0001
04 stc                 ; Set Carry Flag (CF = 1)
05 rcl al, 1            ; Rotate left through carry
06                     ; AL = 0110 0011
07                     ; CF = 1 (old MSB)
08
09 mov ah, 4Ch          ; Terminate program
10 int 21h
11

```

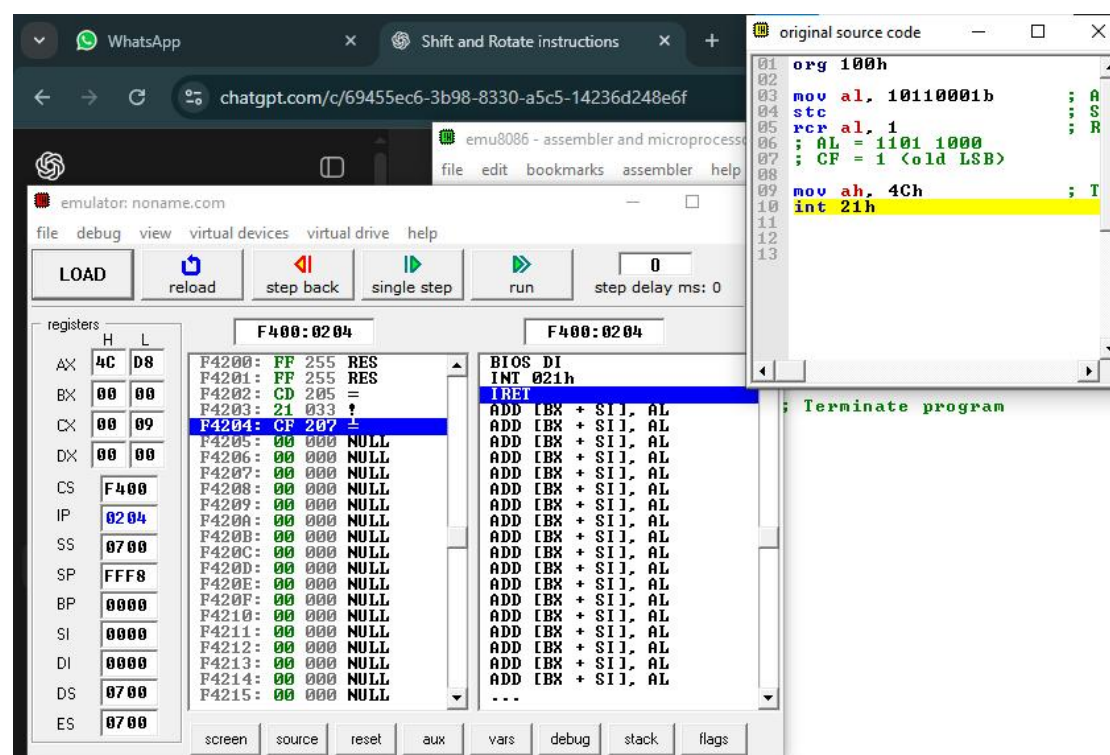
Output :



Example no 4 : RCR



Output :



Example no 5 : rotate with jump

```

01 org 100h
02
03 mov al, 80h      ; AL = 1000 0000h
04 rol al, 1        ; Rotate left
05                 ; MSB goes into CF
06
07 jc jump_label    ; Jump if CF = 1
08
09 jmp end_program   ; Otherwise skip
10
11 jump_label:
12 mov bl, 1         ; BL = 1 indicates jump occurred
13
14 end_program:
15 mov ah, 4Ch       ; Terminate program
16 int 21h
17 |
  
```

Output :

Registers:

	H	L
AX	4C	01
BX	00	01
CX	00	0E
DX	00	00
CS	F400	
IP	0204	
SS	0700	
SP	FFFE	

Memory (F400:0204):

Address	Value	Comment
F4200	FF 255	RES
F4201	FF 255	RES
F4202	CD 205	=
F4203	21 033	?
F4204	CF 207	?
F4205	00 000	NULL
F4206	00 000	NULL
F4207	00 000	NULL
F4208	00 000	NULL
F4209	00 000	NULL
F420A	00 000	NULL
F420B	00 000	NULL
F420C	00 000	NULL
F420D	00 000	NULL

Original source code:

```

01 org 100h
02
03 mov al, 80h      ; A
04 rol al, 1        ; R
05                 ; MSB goes into CF
06
07 jc jump_label    ; J
08
09 jmp end_program   ; O
10
11 jump_label:
12 mov bl, 1         ; B
13
14 end_program:
15 mov ah, 4Ch       ; T
16 int 21h
17
18
19
  
```

Lab Exercise

Output logic:

1. If jump occurs → prints YES
2. If jump does not occur → prints NO

Program no 1 : Load AL with 1 and perform SHR AL, 1. When AL becomes zero, should JZ be taken?

```

01 org 100h
02
03 mov al, 1          ; AL = 0000 0001
04 shr al, 1          ; AL = 0000 0000
05
06 jz zero_jump       ; Jump if Zero Flag = 1
07
08 ; NO case
09 mov dx, offset msg_no
10 jmp show
11
12 zero_jump:
13 mov dx, offset msg_yes
14
15 show:
16 mov ah, 09h
17 int 21h
18
19 mov ah, 4Ch
20 int 21h
21
22 msg_yes db 'YES$'
23 msg_no  db 'NO$'
24

```

Output :

emulator: noname.com

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers H L

AX	4C	24
BX	00	00
CX	00	1D
DX	01	16
CS	F400	
IP	0204	

F400:0204 F400:0204

emu8086 screen (80x25 chars)

```

jmp show
zero_jump:
mov dx, offset msg_yes

show:
mov ah, 09h
int 21h
mov ah, 4Ch
int 21h

```

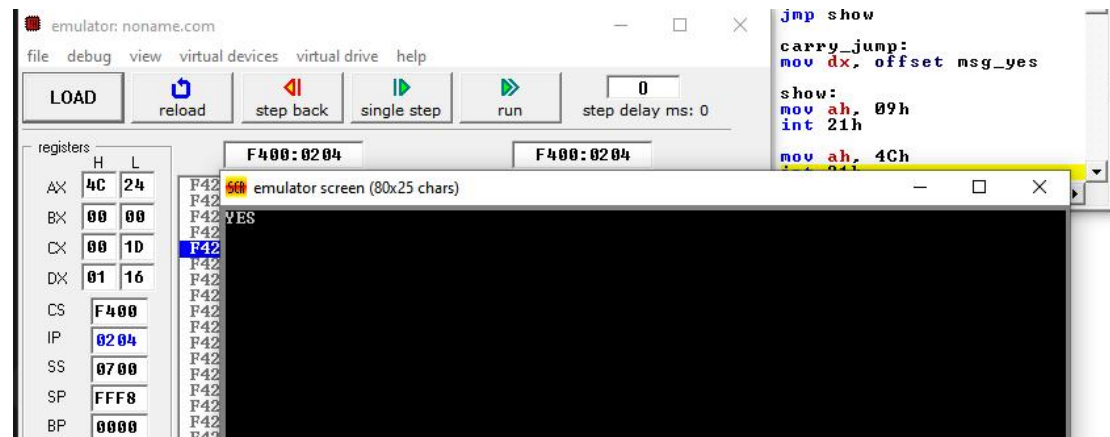
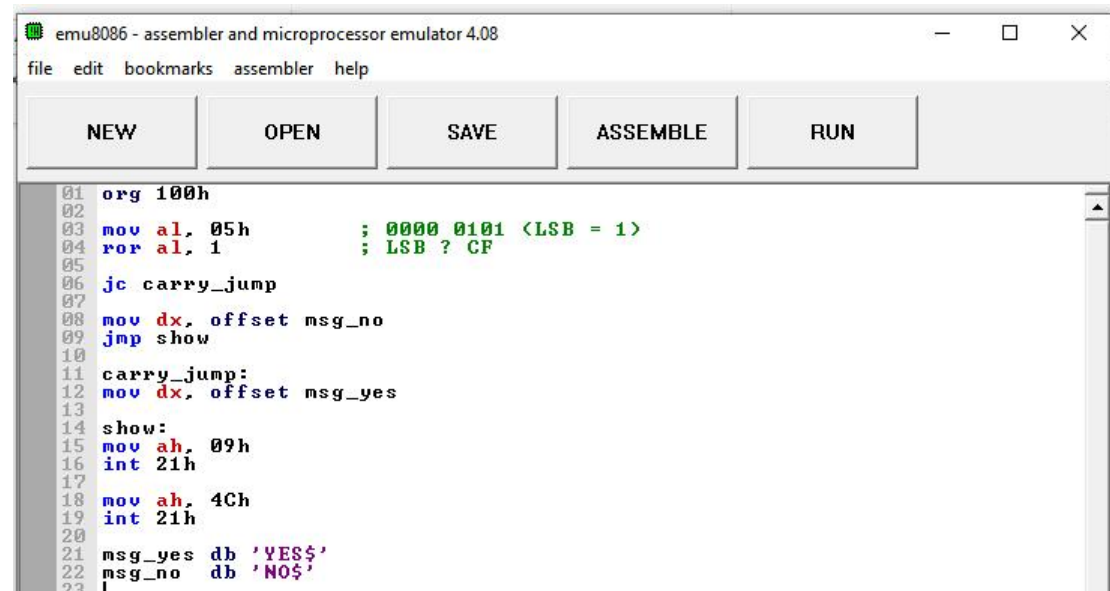
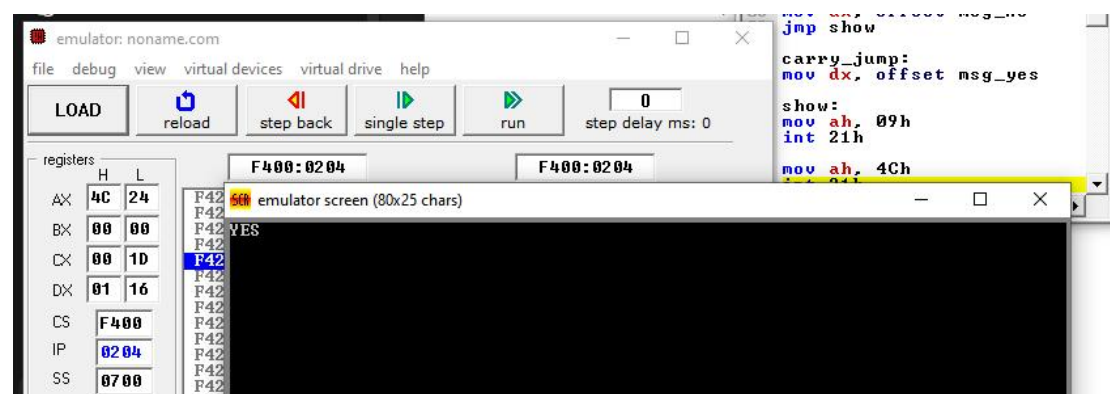
YES

Program no 2 : Load AL with 80h and perform ROL AL, 1. Will JC be triggered?

```

01 org 100h
02
03 mov al, 80h          ; 1000 0000
04 rol al, 1            ; MSB ? CF
05
06 jc carry_jump
07
08 mov dx, offset msg_no
09 jmp show
10
11 carry_jump:
12 mov dx, offset msg_yes
13
14 show:
15 mov ah, 09h
16 int 21h
17
18 mov ah, 4Ch
19 int 21h
20
21 msg_yes db 'YES$'
22 msg_no  db 'NO$'
23

```


Output :**Program no 3 : Load AL with any value and perform ROR AL, 1. If LSB = 1, should JC occur?****Output :**

Program no 4 : Use RCL AL, 1 when CF = 1. After rotating, should a JC jump happen?

emu8086 - assembler and microprocessor emulator 4.08

file edit bookmarks assembler help

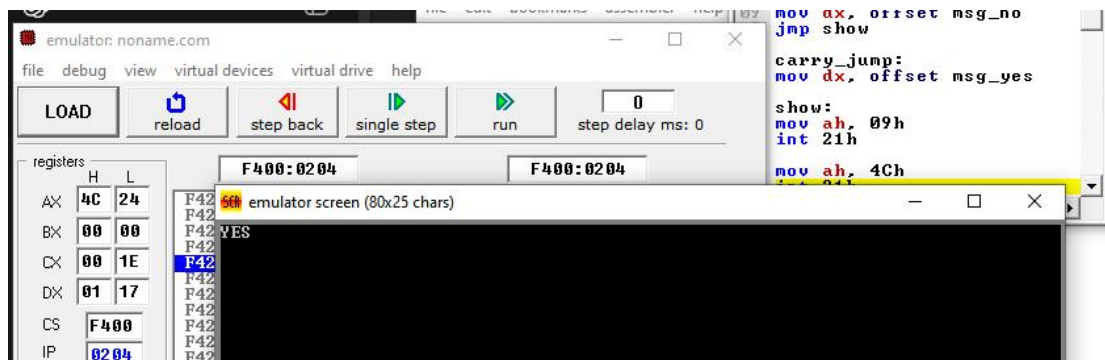
NEW OPEN SAVE ASSEMBLE RUN

```

01 org 100h
02
03 mov al, 80h          ; 1000 0000
04 stc                 ; CF = 1
05 rcl al, 1           ; Rotate through carry
06
07 jc carry_jump
08
09 mov dx, offset msg_no
10 jmp show
11
12 carry_jump:
13 mov dx, offset msg_yes
14
15 show:
16 mov ah, 09h
17 int 21h
18
19 mov ah, 4Ch
20 int 21h
21
22 msg_yes db 'YES$'
23 msg_no db 'NO$'
24 !

```

Output :



Program no 5 : Perform RCR AL, 1 on a value ending in bit 1. Will that bit rotate into CF and activate JC?

```

01 org 100h
02
03 mov al, 03h      ; 0000 0011 <LSB = 1>
04 stc             ; Optional: CF included
05 rcr al, 1       ; Rotate right through carry
06
07 jc carry_jump
08
09 mov dx, offset msg_no
10 jmp show
11
12 carry_jump:
13 mov dx, offset msg_yes
14
15 show:
16 mov ah, 09h
17 int 21h
18
19 mov ah, 4Ch
20 int 21h
21
22 msg_yes db 'YES$'
23 msg_no  db 'NO$'
24 |

```

Output :

emulator noname.com

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers

	H	L
AX	4C	24
BX	00	00
CX	00	1E
DX	01	17

F400:0204 F400:0204

emulator screen (80x25 chars)

```

jmp show
carry_jump:
mov dx, offset msg_yes

show:
mov ah, 09h
int 21h

mov ah, 4Ch
int 21h

```

YES

LAB NO 14

SUBROUTINES

Subroutine (Procedure)

A subroutine, also known as a procedure or function, is a self-contained block of instructions designed to perform a specific task within a program. Instead of writing the same code again and again, a subroutine allows the programmer to write the code once and reuse it whenever needed.

Subroutines are executed only when they are called by the main program or by another subroutine. After completing their task, they return control back to the point from where they were called. This mechanism makes programs more organized, efficient, and easier to manage.

1. CALL Instruction

The CALL instruction is used to transfer control from the main program to a subroutine. When the processor encounters a CALL instruction, it performs two important actions:

First, it saves the return address on the stack. The return address is the location of the next instruction that should be executed after the subroutine finishes.

- In a NEAR CALL, only the Instruction Pointer (IP) is pushed onto the stack.
- In a FAR CALL, both the Code Segment (CS) and Instruction Pointer (IP) are pushed onto the stack.

Second, the processor jumps to the starting address of the subroutine. From this point onward, instructions inside the subroutine are executed. In this way, the CALL instruction temporarily transfers program control to another part of the program.

2. RET Instruction

The RET (Return) instruction is used at the end of a subroutine to send control back to the calling program. When RET executes, the processor performs the following actions:

- It pops the return address from the stack.
- The popped address is loaded back into the instruction pointer.
- Program execution continues from the instruction immediately after the CALL instruction.

The RET instruction ensures that the program resumes execution from the correct location, maintaining proper program flow and preventing execution errors.

Purpose of Using CALL and RET

The use of CALL and RET instructions provides several important advantages:

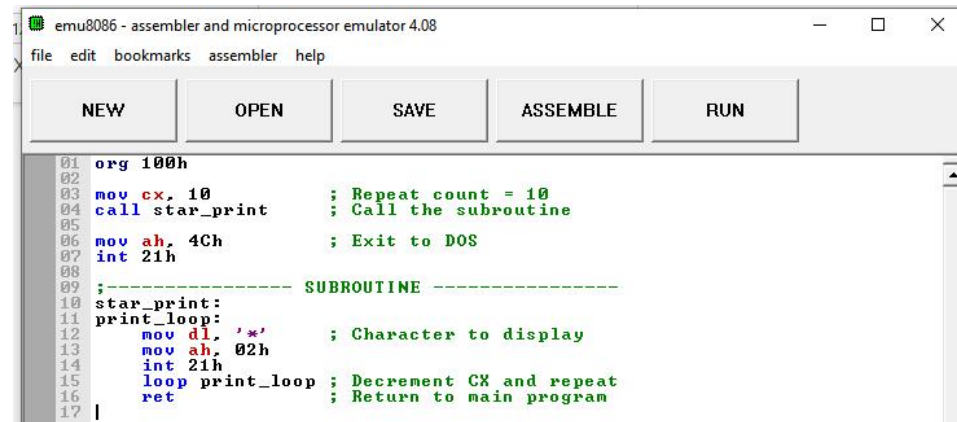
- They reduce code repetition by allowing the same subroutine to be used multiple times.
- They make large programs modular, dividing them into smaller and manageable sections.
- They improve readability, as each subroutine performs a clearly defined task.
- They make programs easier to maintain and debug, since changes can be made in one place without affecting the entire program.

Overall, CALL and RET play a vital role in structured and efficient assembly language programming.

ALGORITHM (Using Subroutine)

1. Start the program execution.
2. Initialize or load required register values.
3. Use the CALL instruction to invoke the subroutine.
4. The subroutine executes its assigned task (for example, displaying a character or performing a calculation).
5. The RET instruction is executed to return control to the main program.
6. The program continues execution and finally terminates.

Example no 1 : 10 stars printed on screen

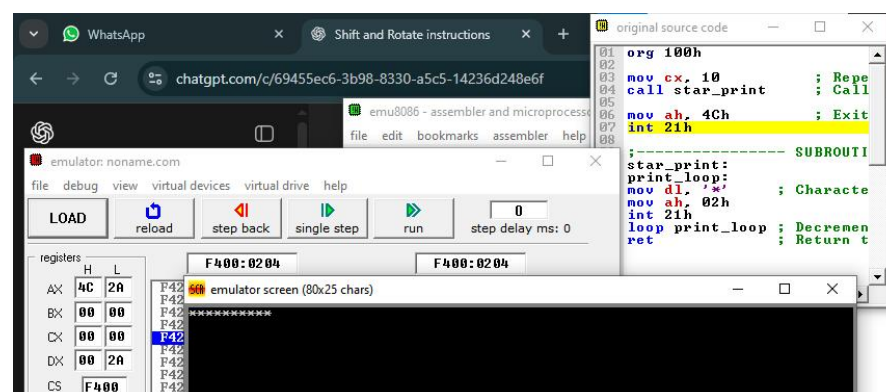


```

01 org 100h
02
03 mov cx, 10      ; Repeat count = 10
04 call star_print ; Call the subroutine
05
06 mov ah, 4Ch     ; Exit to DOS
07 int 21h
08
09 ;----- SUBROUTINE -----
10 star_print:
11 print_loop:
12     mov dl, '*'  ; Character to display
13     mov ah, 02h
14     int 21h
15     loop print_loop ; Decrement CX and repeat
16     ret          ; Return to main program
17

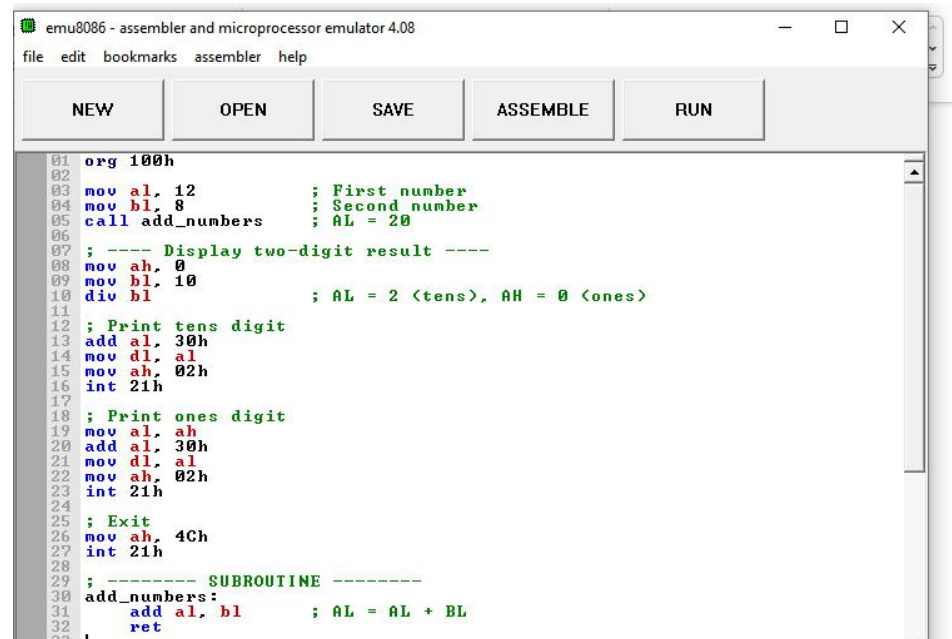
```

Output :



The screenshot shows the emu8086 emulator interface. The assembly code from the previous block is visible in the background. The foreground shows the emulator's registers and the output window. The output window displays 10 stars, confirming the program's execution.

Example no 2 : Subroutine to Add Two Numbers

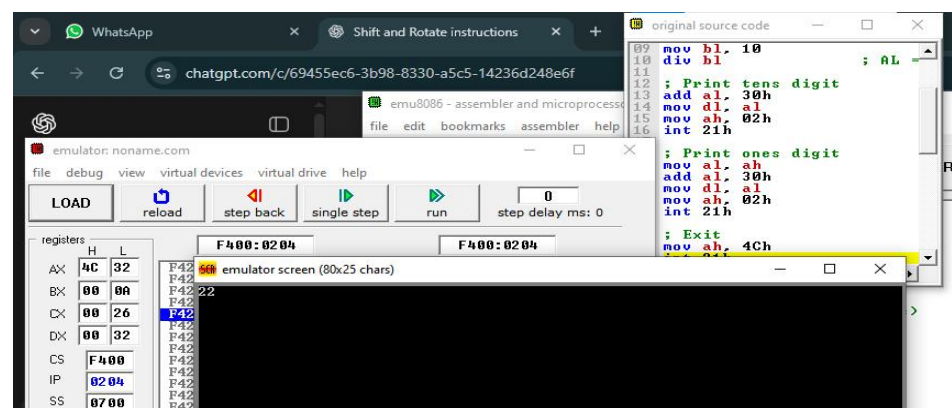


```

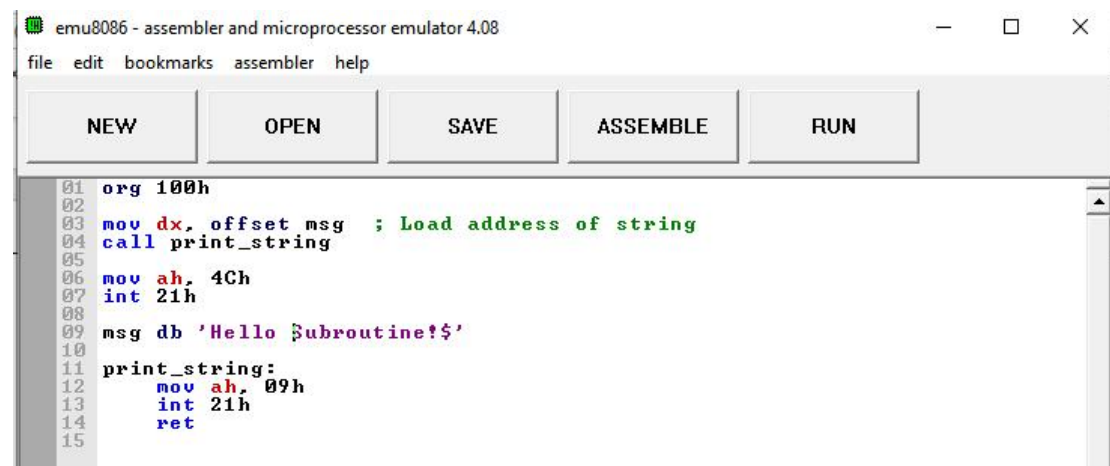
01 org 100h
02
03 mov al, 12      ; First number
04 mov bl, 8       ; Second number
05 call add_numbers ; AL = 20
06
07 ; ---- Display two-digit result ----
08 mov ah, 0
09 mov bl, 10
10 div bl          ; AL = 2 (tens), AH = 0 (ones)
11
12 ; Print tens digit
13 add al, 30h
14 mov dl, al
15 mov ah, 02h
16 int 21h
17
18 ; Print ones digit
19 mov al, ah
20 add al, 30h
21 mov dl, al
22 mov ah, 02h
23 int 21h
24
25 ; Exit
26 mov ah, 4Ch
27 int 21h
28
29 ; ----- SUBROUTINE -----
30 add_numbers:
31     add al, bl      ; AL = AL + BL
32     ret
33

```

Output :



Example no 3 : Subroutine to Display a String

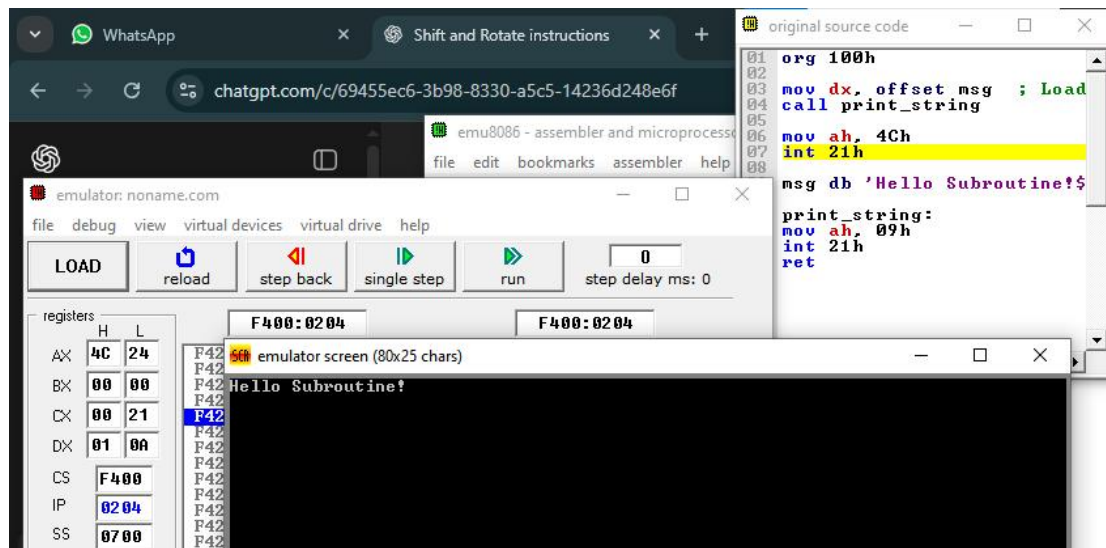


```

01 org 100h
02
03 mov dx, offset msg ; Load address of string
04 call print_string
05
06 mov ah, 4Ch
07 int 21h
08
09 msg db 'Hello Subroutine!$'
10
11 print_string:
12     mov ah, 09h
13     int 21h
14     ret
15

```

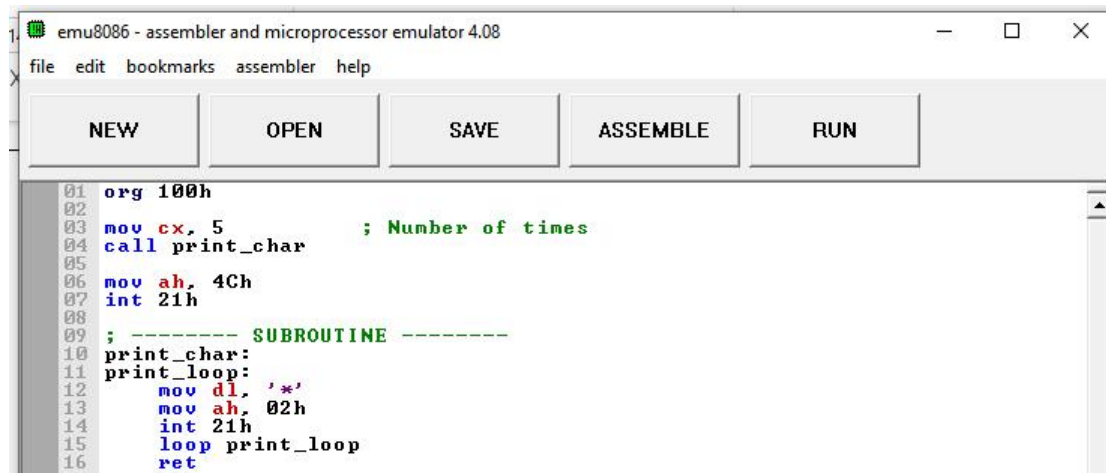
Output :



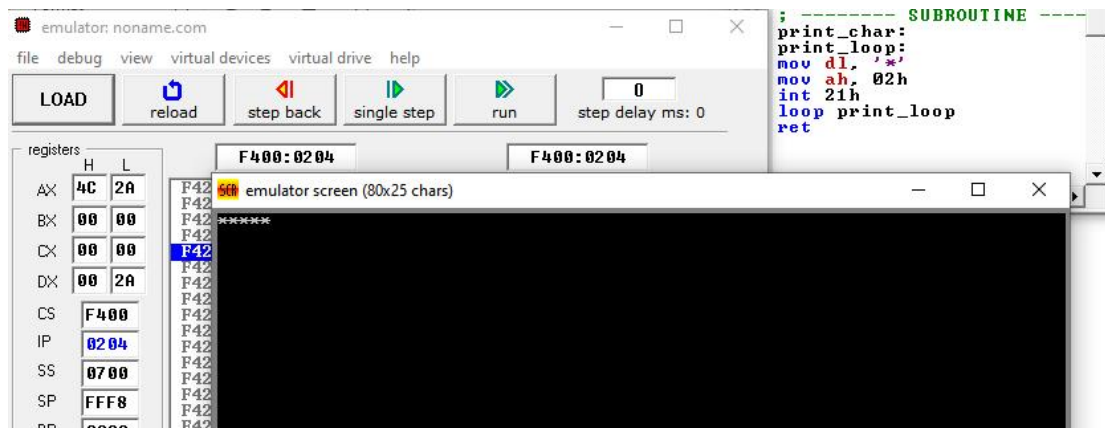
This lab focuses on the use of **subroutines (CALL and RET)** in Assembly Language. The objective is to understand how tasks can be separated into reusable procedures to improve program structure, readability, and efficiency.

Each program demonstrates calling a subroutine to perform a specific operation and returning control to the main program.

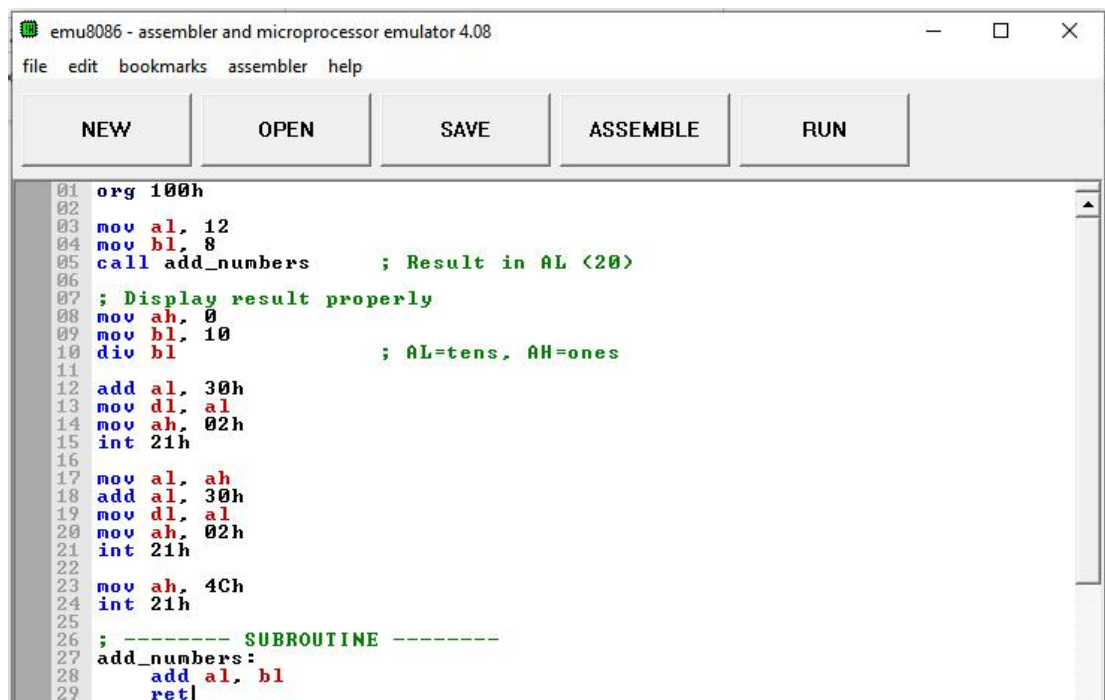
Program no 1 : Print a Character N Times Using Subroutine



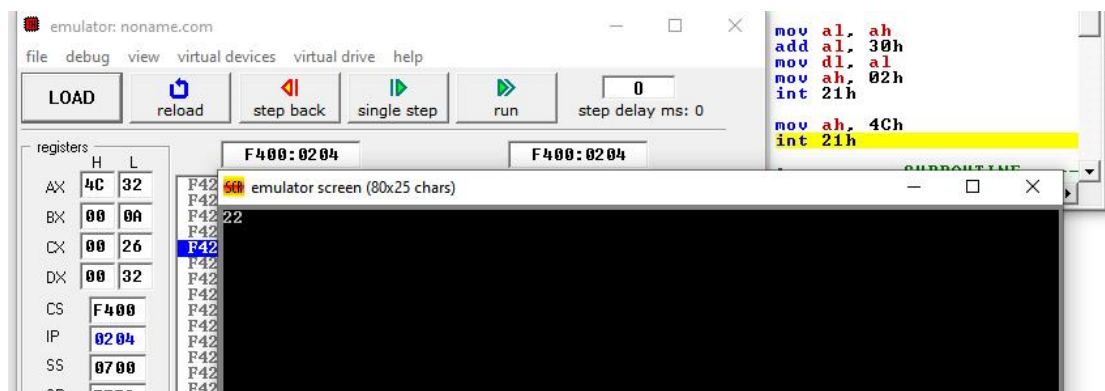
Output



Program no 2 : Add Two Numbers Using Subroutine (Proper Numeric Output)



Output



Program no 3 : Display a String Using Subroutine



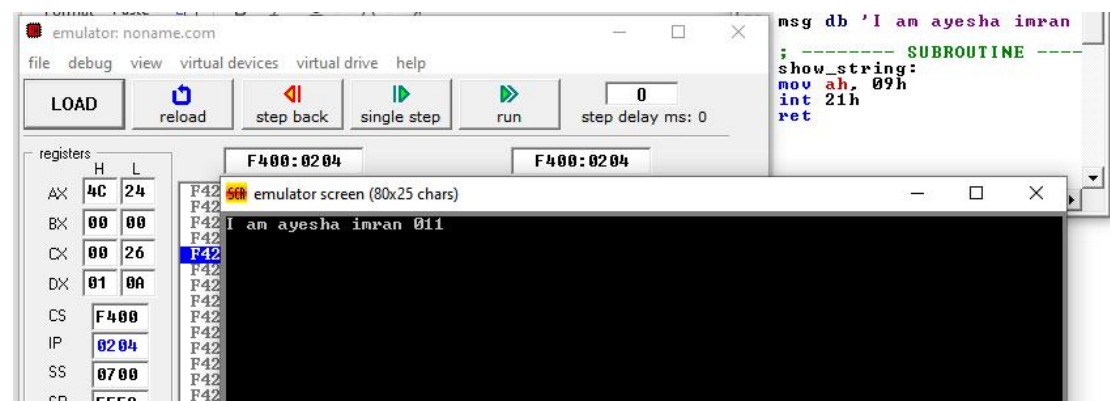
emu8086 - assembler and microprocessor emulator 4.08

file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

```
01 org 100h
02
03 mov dx, offset msg
04 call show_string
05
06 mov ah, 4Ch
07 int 21h
08
09 msg db 'I am ayesha imran 011 $'
10
11 ; ----- SUBROUTINE -----
12 show_string:
13     mov ah, 09h
14     int 21h
15     ret
```

Output



Program no 4 : Find the Largest of Three Numbers Using Subroutine



emu8086 - assembler and microprocessor emulator 4.08

file edit bookmarks assembler help

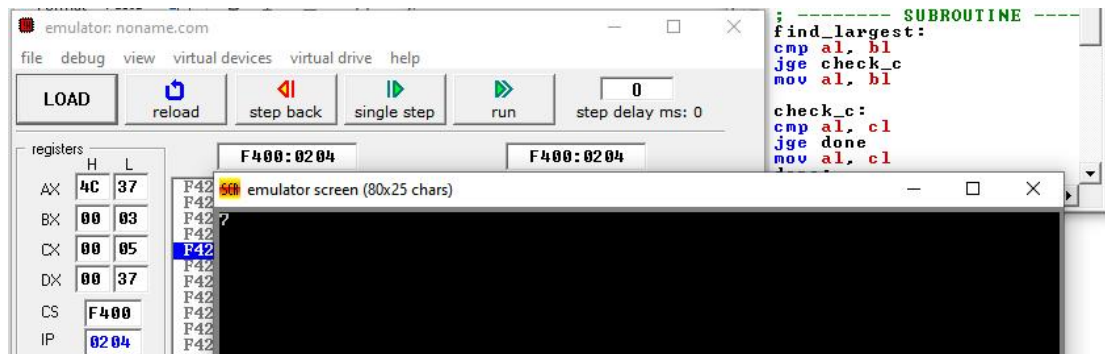
NEW OPEN SAVE ASSEMBLE RUN

```

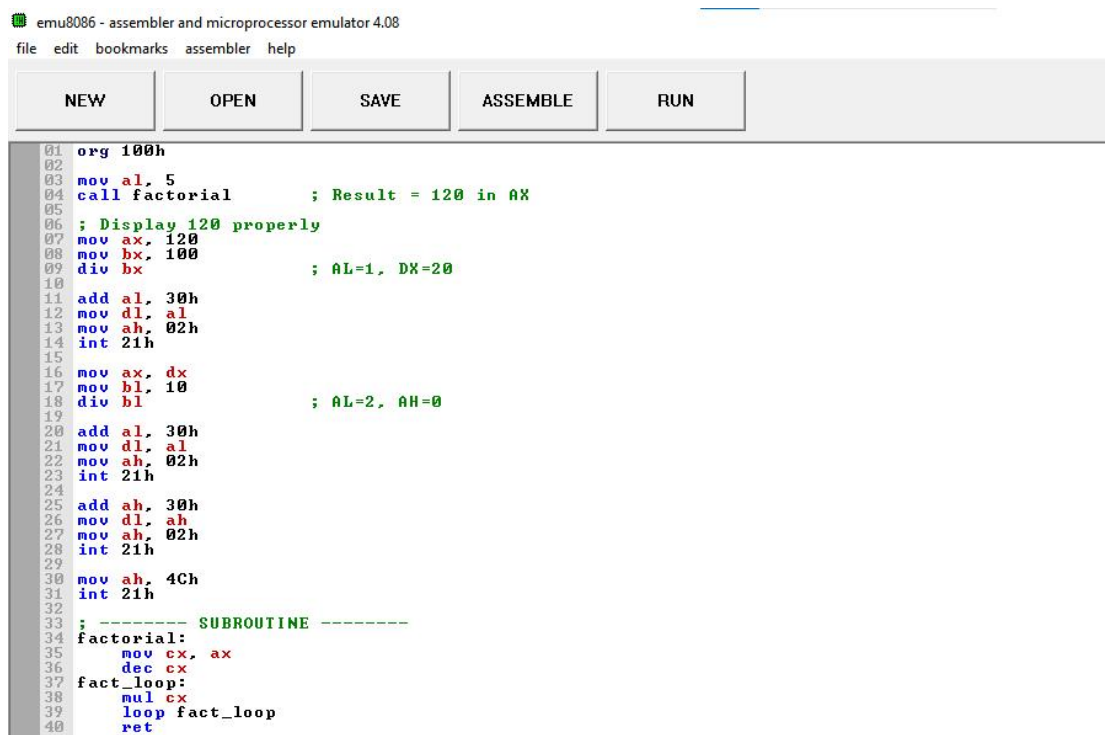
01 org 100h
02
03 mov al, 7
04 mov bl, 3
05 mov cl, 5
06 call find_largest      ; Largest in AL
07
08 ; Display result (single digit)
09 add al, 30h
10 mov dl, al
11 mov ah, 02h
12 int 21h
13
14 mov ah, 4Ch
15 int 21h
16
17 ; ----- SUBROUTINE -----
18 find_largest:
19     cmp al, bl
20     jge check_c
21     mov al, bl
22
23 check_c:
24     cmp al, cl
25     jge done
26     mov al, cl
27 done:
28     retl

```

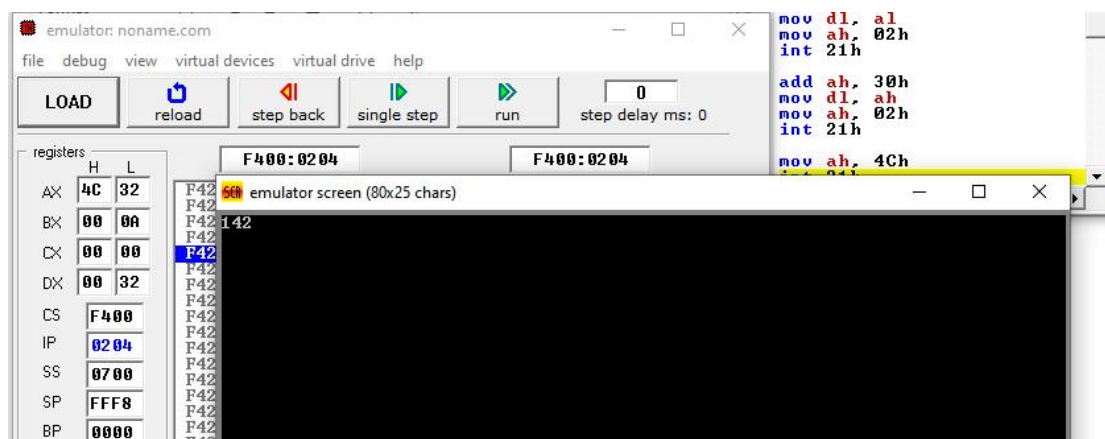
Output



Program no 5 : Compute Factorial Using Subroutine (Factorial of 5)



Output



LAB NO 15

SUBROUTINES